

它定义TCP/IP协议的物理多播地址。余下的23位可用来定一个组。要将IP多播地址转换为以太网地址，多播路由器就要提取D类IP地址中的最低的23位，并将其插入到以太网物理地址中（如图21.21所示）。

但是，D类IP地址的组标识符是28位长，这就是说有5位未使用。这就表示了32 (2^5)个IP级的多播地址映射到一个的多播地址。换言之，映射是多对一，而不是一对一。如果D类地址中的组标识符的最左5位不全为0，主机就可能接收到并不是真正属于它所涉及到的组的分组。为此，主机必须检查IP地址，并丢弃任何不属于它的分组。

其他局域网也支持同样的概念，但映射的方法可以不同的。

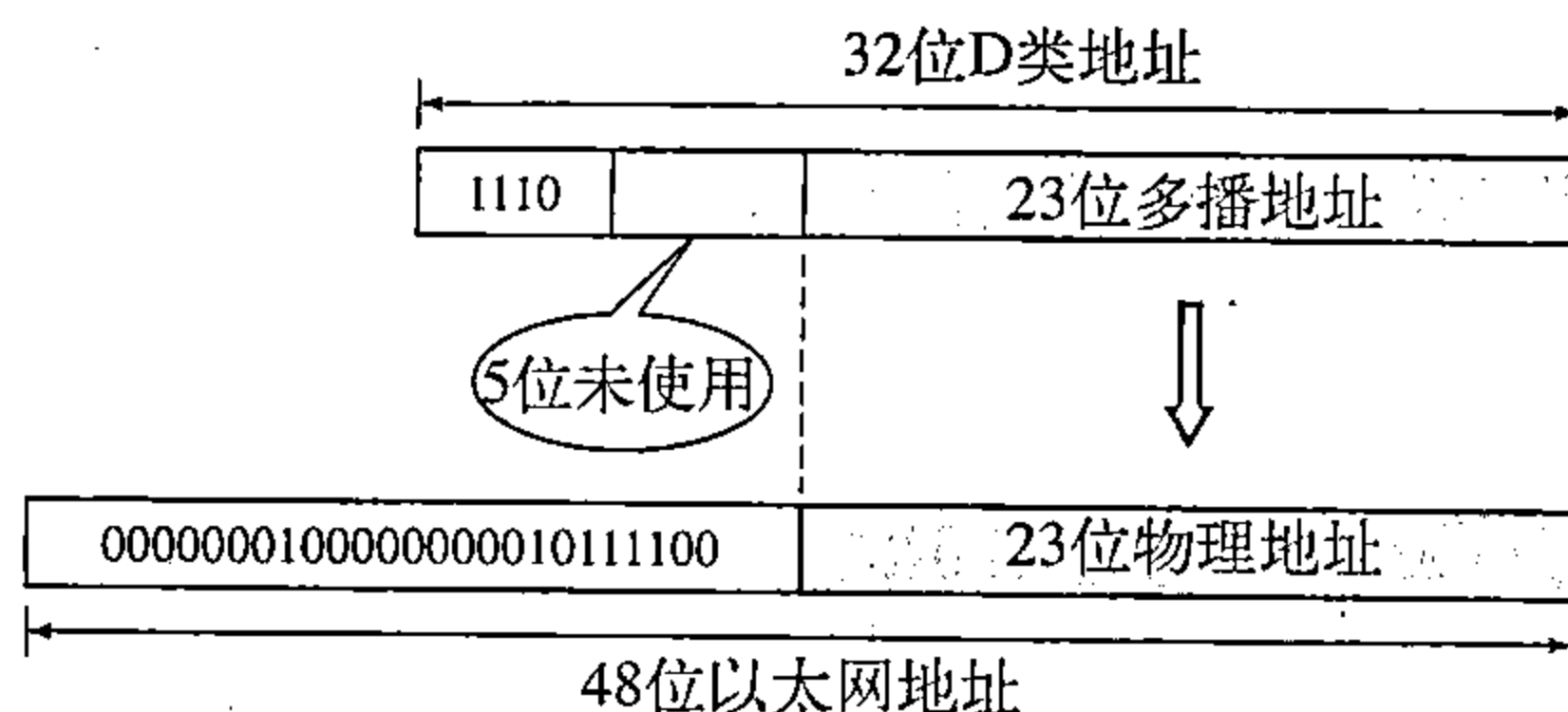


图21.21 将D类地址映射到以太网物理地址

以太网的多播物理地址是在01: 00: 5E: 00: 00: 00到01: 00: 5E: 7F: FF: FF范
围中。

例21.7

将多播IP地址230.43.14.7转换成以太网多播物理地址。

解：

这可用2个步骤来完成：

- a. 我们用十六进制写出IP地址的最右23位。这可将最右边3个字节变换成十六进制，然后如果最左边的数字大于或等于8，则将该数减去8。在本例中，其结果是2B: 0E: 07。
- b. 将a步所得的结果加到开始的以太网多播地址01: 00: 5E: 00: 00: 00，其结果是01: 00: 5E: 2B: 0E: 07。

例21.8

将多播IP地址238.212.24.9转换成以太网多播地址。

解：

- a. 用十六进制表示的最右边3个字节是D4: 18: 09。我们需将最左边的数字减去8，其结果是54: 18: 09。
- b. 将a步所得的结果加到以太网多播的开始地址，其结果是01: 00: 5E: 54: 18: 09。

无物理多播地址支持 大多数广域网不支持物理多播地址。要通过这样的网络发送多播分组，就要使用称为隧道技术的过程。在隧道技术（tunneling）中，多播分组封装成单播分组并通过网络发送，然后在另一端，这个分组又转换成多播分组（如图21.22所示）。

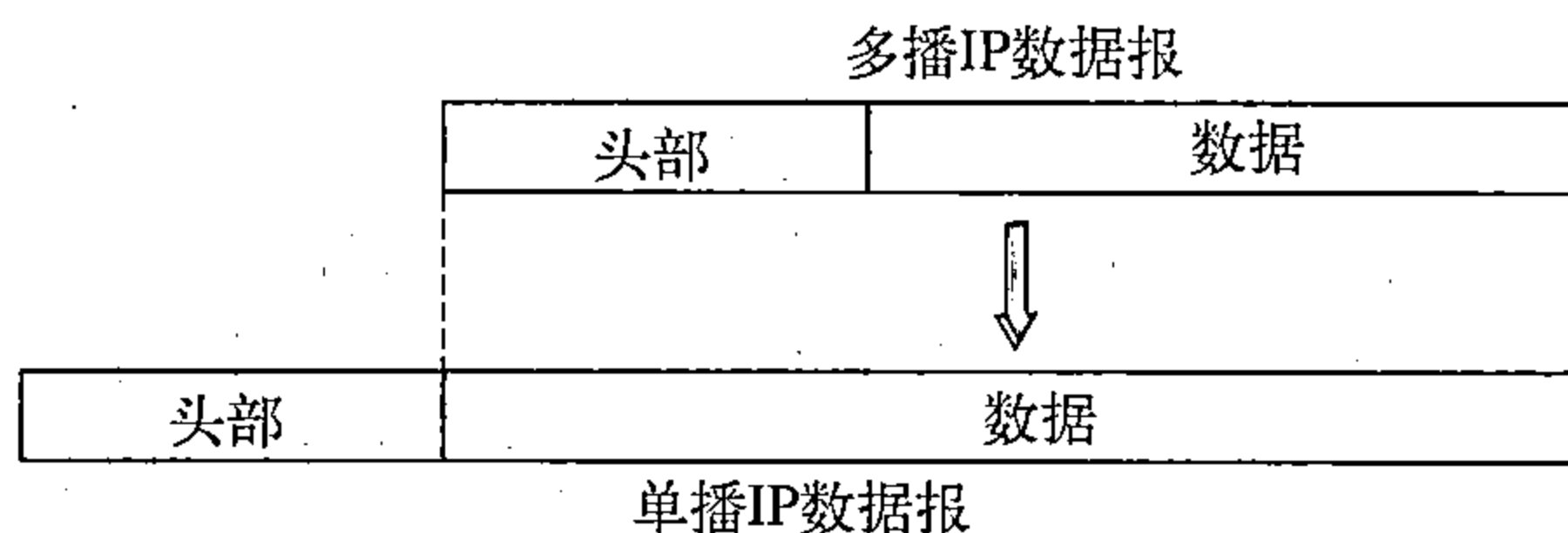


图21.22 隧道技术

21.3.6 Netstat应用程序

netstat应用程序可以用来得到接口所支持的多播地址。

例21.9

我们使用带有三个选项的netstat命令。选项-n是以数字形式显示的IP地址、选项-r显示

路由表、选项-a显示所有的地址（单播和多播地址）。注意：这里仅给出与我们讨论有关的项目。“Destination”定义目的地址，“Gateway”定义路由器，“Iface”定义接口，“Mask”定义掩码，“Flag”定义标记。标记U表该路由可使用，标志G表示该路由是一个网关（路由器）。

```
$ netstat -nra
Kernel IP routing table
Destination      Gateway          Mask            Flags           Iface
153.18.16.0      0.0.0.0          255.255.240.0   U               eth0
169.254.0.0      0.0.0.0          255.255.0.0     U               eth0
127.0.0.0        0.0.0.0          255.0.0.0       U               lo
224.0.0.0        0.0.0.0          224.0.0.0       U               eth0
0.0.0.0          153.18.31.254   0.0.0.0         UG              eth0
```

注意：多播地址用灰色表示。具有从240.0.0.0到239.255.255.255多播地址的任何分组都被屏蔽，并传递给以太网接口。

21.4 ICMPv6

我们在第20章中讨论过IPv6。在TCP/IP协议族的版本6中被修改的另一个协议是ICMP (IMCpv6)。这个新版本与版本4的策略和目的一样，修改ICMPv4使得它更加适合IPv6。此外，在版本4中的一些独立的协议现在成为网际控制报文协议（Internet-working Control Message Protocol, ICMPv6）的一部分，图21.23比较了版本4和版本6的网络层。

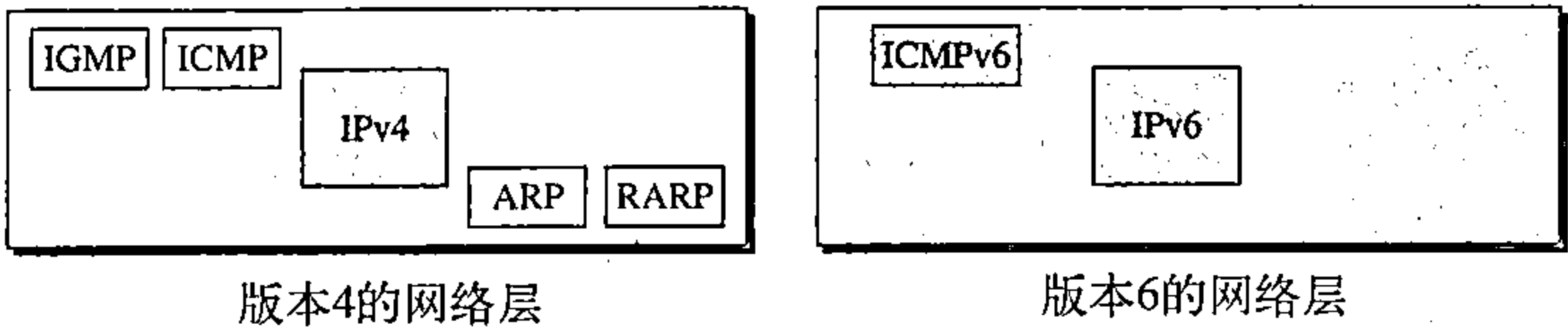


图21.23 版本4和版本6的网络层比较

版本4中的ARP和ICMP协议合并到ICMPv6。RARP协议从这个协议族中取消了，因为它很少使用。而且BOOTP具有与RARP相同的功能。

正如ICMPv4那样，我们将ICMP报文分成两大类。但是，每一类都比前面版本有更多的报文类型。

21.4.1 差错报告

正如我们在讨论版本4时所看到的那样，ICMP的一个主要职责就是对差错进行报告。有5种差错类型：目的端不可达、分组太大、时间超时、参数问题和重定向。ICMPv6组成差错分组，再封装成IP数据报，然后传递给产生差错的数据报的源端。表21.3对ICMPv4和ICMPv6的差错报告报文（error-reporting message）进行了比较。在版本6中源端抑制报文取消了，因为优先级和流标号字段允许路由器控制拥塞，并将最不重要报文丢弃。在这个版本中，不需要通知发送方将速率放慢。增加了分组太大报文，因为在IPv6中分段是发送方的责任。如果发送方没有对分组长度做出正确的判断，路由器就没有任何选择，而只能丢弃分组，并发送差错报告报文给发送方。

表21.3 ICMPv4和ICMPv6的差错报告的比较

报文类型	版本4	版本6
目的端不可达	是	是
源端抑制	是	否
分组太大	否	是
时间超时	是	是
参数问题	是	是
重定向	否	是

目的端不可达

目的端不可达报文 (destination-unreachable message) 的概念和ICMPv4完全相同。

分组太大

这是增加到版本6的一个新的报文类型。如果路由器接收到一个数据报，它大于数据报能够通过网络的最大传输单元 (MTU)，那么就要发生两件事。第一，路由器要丢弃该数据报；第二，要发送差错分组，即分组太大报文 (packet-too-big message) 给源端。

时间超时

这个报文与版本4相似。

参数问题

这个报文与版本4相应的报文相似。

重定向

重定向报文 (redirection message) 的目的与版本4的相同。

21.4.2 查询

除差错报告外，ICMP还能诊断某些网络问题。这是通过查询报文 (query message) 来完成的。已定义了4组报文：回送请求和回答、路由器询问和通知、邻站询问和通知以及组成员关系。表21.4显示了版本4与版本6的查询报文的比较。ICMPv6取消了两组查询报文：时间戳请求和回答以及地址掩码请求和回答。取消时间戳请求和回答是因为它们在其他协议中，如TCP已实现了，同时也因为在过去没有使用过它们。在IPv6中取消地址掩码请求和回答报文是因为地址的子网部分允许用户使用多达 $2^{32}-1$ 个子网。因此，定义在IPv4中的子网掩码在这里是不需要的。

表21.4 ICMPv4和IMCPv6中查询报文的比较

报文类型	版本4	版本6
回送请求和回答	是	是
时间戳请求和回答	是	否
地址掩码请求和回答	是	否
路由器询问和通知	是	是
邻站询问和回答	ARP	是
组成员关系	IGMP	是

回送请求和回答

回送请求和回答报文的格式和思路与版本4一样。

路由器询问和通告

路由器询问和通告报文的格式和思路与版本4一样。

邻站询问和通告

如前所述，版本4的网络层包含一个独立的协议称为ARP。在版本6中，这个协议取消了，而它的任务并入到ICMPv6中。思路完全相同，但报文格式改变了。

组成员关系

如前所述，版本4的网络层包含一个独立的协议称为IGMP。在版本6中，这个协议取消了，而它的任务并入到ICMPv6中。其目的完全相同。

推荐读物

作为深入讨论与本章有关的主题，推荐下列读物和网站，在本书末列出方括号[...]中的参考资料。

读物

ARP和RARP在[For06]的第7章和[Ste94]的第4、5章中有讨论。ICMP在[For06]的第9章和[Ste04]的第6章中有讨论。IGMP在[For06]的第10章和[Ste94]的第13章中有讨论。BOOTP和DHCP可参见[For06]的第16章和[Ste94]的第16章。ICMPv6的讨论见[For06]的第27章。

网站

- www.ietf.org/rfc.html 有关请求评论 (RCF) 信息。

请求评论

ARP和RARP的讨论可在下列请求评论中找到：

826, 903, 925, 1027, 1293, 1329, 1433, 1868, 1931, 2390

ICMP的讨论可在下列请求评论中找到：

777, 792, 1016, 1018, 1256, 1788, 2521

IGMP的讨论可在下列请求评论中找到：

966, 988, 1054, 1112, 1301, 1458, 1469, 1768, 2236, 2357, 2365, 2502, 2588

BOOTP和DHCP的讨论可在下列请求评论中找到：

951, 1048, 1084, 1395, 1497, 1531, 1532, 1533, 1534, 1541, 1542, 2131, 2132

本章小结

- 将分组传递给主机或路由器需要两级地址：逻辑地址和物理地址。物理地址在物理级标识一个主机或路由器。
- 逻辑地址到物理地址的映射可以是静态的或动态的。
- 静态映射要使用逻辑地址与物理地址的对照表，维护该表需要很大开销。
- 地址解析协议（ARP）是一种动态映射方法，在给出逻辑地址时可找出其物理地址。
- 在代理ARP中，一个路由器代表一组主机。当ARP请求寻找该组中任何一个主机时，该路由器就发送它自己的物理地址。这就产生了一个子网化的效应。
- 反向地址解析协议（RARP）是一种动态映射，它将一个给定物理地址和一个逻辑地址联系起来。
- ICMP发送4对查询报文：回送请求和回送回答、时间戳请求和回答、地址掩码请求和回答以及路由器询问和回答。
- 使用ICMP报文的头部和数据字段计算ICMP的校验和。
- 分组因特网搜寻器（*ping*）是一个应用程序，它使用ICMP的服务来测试主机的可达性。
- 多播就是将相同的报文同时发送给一个以上的接收方。
- 因特网组管理协议（IGMP）帮助多播路由器创建和更新多播地址表。
- 3种类型的IGMP的报文是查询报文、成员关系报告和离开报告。
- 延迟响应策略是防止局域网上不必要的通信。
- IGMP报文封装在IP数据报中。
- 包括以太网在内的大多数局域网支持物理多播寻址。
- 不支持物理多播寻址的广域网可使用称为隧道技术的过程来发送多播分组。
- BOOTP和动态主机配置协议（DHCP）都是客户机/服务器应用程序，用于将重要的网络信息传递给无盘计算机或第一次引导计算机。
- BOOTP请求被封装成UDP用户数据报。

- BOOTP是静态配置协议，它使用一个表将IP地址映射到物理地址。
- 中继代理是一个路由器，它帮助将本地的BOOTP请求发送到远程服务器。
- DHCP是动态配置协议，它有两个数据库：一个与BOOTP相似，另一个是IP地址池，用于临时的分配。
- DHCP服务器将IP地址的租用发送给客户使用一个特定的时间。
- DHCPv6和版本4一样，报告差错、处理组成员关系、更新特定的路由器和主机的表以及检查主机的生存能力。

习题

复习题

1. ARP分组的长度是固定的吗？试加以解释。
2. 当协议是IPv4而硬件是以太网时，一个ARP分组的长度是多少？
3. 在问题2中，携带ARP分组的以太网的帧长度是多少？
4. 以太网的广播地址是什么？
5. 为什么对一个失效的ICMPv4差错报文要限制再产生一个ICMPv4报文？
6. 在差错报告IMCPv4报文中包括IPv4的头部和数据报数据前8个字节的目的是什么？
7. 试给出一个例子，说明一个主机可能会永远收不到一个重定向报文。
8. ICMPv4分组的最小长度是多少？ICMPv4分组的最大长度是多少？
9. 携带ICMPv4分组的IPv4分组的最小长度是多少？最大长度是多少？
10. 我们如何能够确定是否IPv4分组携带了ICMPv4分组？
11. 携带IPv4分组的以太网帧的最小长度是多少？最大长度是多少？
12. 为什么没有必要将ICMPv4报文传送到它自己所在的网络之外？

练习题

13. 一个路由器的IPv4地址是125.45.23.12，而所在的以太网物理地址是：23：45：AB：4F：67：CD，接收到一个分组，其目的IPv4地址是125.11.78.10。试给出此路由器发出的ARP请求分组中的各项，假定子网化。
14. 试给出响应练习13而发送ARP分组中的各项目。
15. 将练习13的结果封装成数据链路层的帧，试填充所有字段。
16. 将练习14的结果封装成数据链路层的帧，试填充所有字段。
17. 主机A向主机B发送数据报。主机B从未接收到该数据报，而主机A也未收到出问题的通知。试给出可能发生的情况的两种不同的解释。
18. 试计算以下ICMP分组的校验和。
类型：回送请求 标识符：123 序号：25 报文：Hello
19. 一个路由器接收到一个IPv4分组，其源IP地址是130.45.3.3，而目的IP地址是201.23.4.6，该路由器不能在其路由表中找到其目的地址，试问发送出去的ICMP报文各字段应该是什么？
20. TCP接收到一个目的端口为234的报文分段，但TCP经过检查无法打开这个目的端口，试问发出去的ICMP报文各字段应该是什么？
21. 一个组多播地址是231.24.60.9。对于局域网在使用TCP/IP时，其48位的以太网地址是什么？
22. 如果一个路由器的组表中有20个项目，它应当周期性地发送20个不同的查询还是只发送一个？试解释之。

23. 如果一个主机愿意在5个组中继续其成员关系，它应该发送5个不同的成员关系报告报文还是只发送一个？
24. 一个以太网上的路由器接收到多播IP分组，其组标识符为226.17.18.4。当主机检查其多播组表时，它找到这个地址。试说明路由器如何将这个IP分组封装成以太网帧，并将它发送给各接收端。进一步给出以太网帧的所有项目。此路由器出去的IP地址是185.23.5.6，而物理地址是4A: 22: 45: 12: E1: E2，这个路由器需要ARP服务吗？
25. 一个IPv4地址为114.45.7.9的主机接收到一个IGMP查询，当它检查表组时发现没有项目，该主机应采取什么动作？它应该发送什么报文？
26. 一个IPv4地址为222.5.7.19的主机接收到一个IGMP查询，当它检查表组时发现有两个项目：227.4.3.7和229.45.6.23。该主机应该采取什么动作？它应该发送报文吗？如果是的话，什么类型报文和发送几个？
27. 在以太网中的IPv4协议能够支持多少个多播地址？IPv4协议能够支持多少个多播地址？当将多播IPv4地址转换为以太网多播地址时，地址空间损失多少？
28. 试将下列的IPv4多播地址转换成以太网多播地址，它们中有哪些指明了相同的以太网地址？
 - a. 224.18.72.8
 - b. 235.18.72.8
 - c. 237.18.6.88
 - d. 224.88.12.8

探究题

29. 用ping程序测试你自己的计算机（环回）。
30. 用ping程序测试美国国内的一个主机。
31. 用ping程序测试美国国外的一个主机。
32. 用tracert（或tracert）程序测试从你的计算机到学院或大学中的一个主机的路由。
33. 用tracert程序寻找你的服务器是否支持多播寻址。
34. DHCP使用诸如DHCPREQUEST、DHCPDECLINE、DHCPACK、DHCPNAK以及DHCPRELEASE的报文，试说明这些报文的目的是。

第22章 传递、转发和路由选择

这一章讨论IP分组到它们最终目的端的传递、转发和路由选择。所谓传递 (Delivery) 是指在网络层控制下用底层的网络对一个分组进行处理的方法。所谓转发 (Forwarding) 是指将一个分组传递到下一个站点的方法。所谓路由选择 (Routing) 是指在转发过程中创建路由表的方法。

路由选择协议 (routing protocol) 用于不断地更新在转发和路由选择中要查找的路由表。在本章中, 我们也讨论常用的单播和多播路由 (multicast routing) 协议。

22.1 传递

网络层负责用底层物理网络处理分组, 我们定义这种处理为分组的传递。

直接传递和间接传递

分组传递到最终目的端可使用两种不同的方法: 直接传递和间接传递, 如图22.1所示。

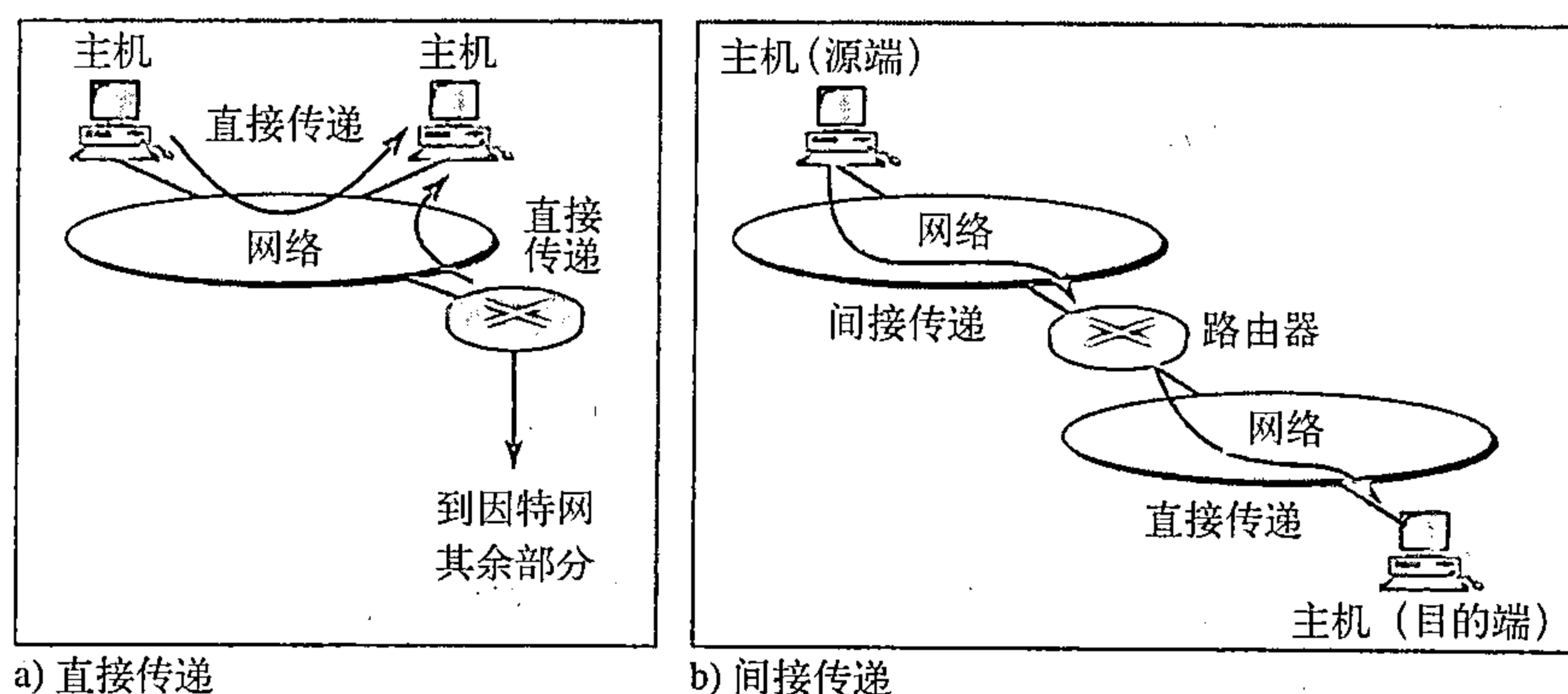


图22.1 直接传递和间接传递

直接传递

直接传递 (direct delivery) 就是分组的最终目的端的主机是与发送方都连接在同一个物理网络上。当分组的源端和目的端都在同一个物理网络上, 或传递是在最后一个路由器与目的主机之间进行时, 就出现直接传递。

发送方很容易确定传递是否是直接传递, 它只要提取出分组目的端的网络地址 (用掩码), 并与它所连接的网络地址进行比较。如果相同, 则传递就是直接的。

间接传递

如果目的主机与发送方不在同一个网络上, 分组就是间接传递。在间接传递 (indirect delivery) 时, 分组从一个路由器传送到另一个路由器, 直到它到达与最终目的端连接在同一个物理网络上的路由器为止。注意: 一个传递永远包含一个直接传递和0个或多个间接传递。还要注意: 最后的传递总是直接传递。

22.2 转发

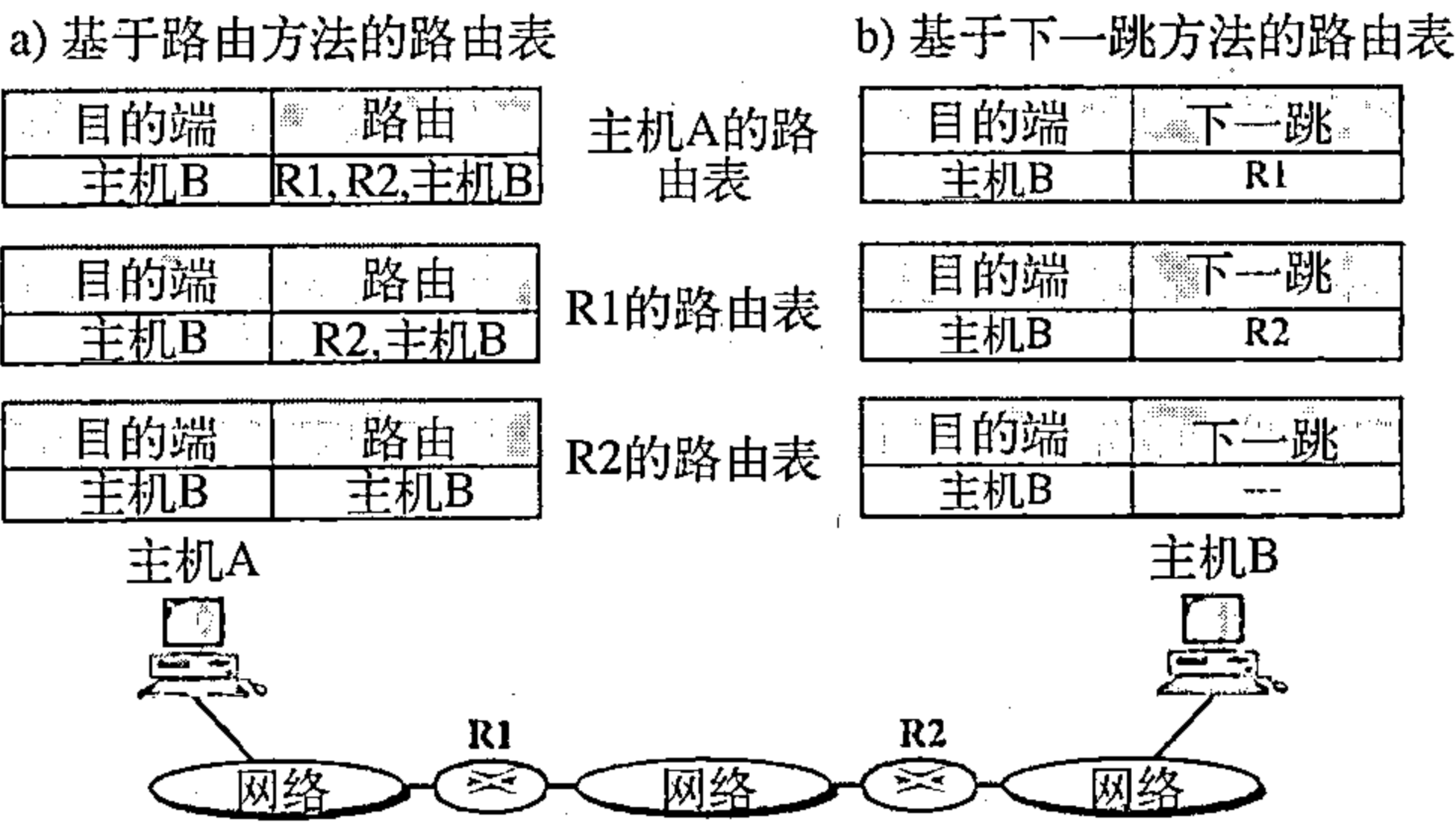
转发是指将分组路由到它的目的端。转发要求主机或路由器有一个路由表。当主机有分组要发送时，或路由器已收到一个分组要进行转发时，就要查找路由表以便求得到达最终目的端的路由。但是，这种简单方法对于今天的互联网（如因特网）已是不可能的，因为路由表中的项目数已使得路由表的查找效率很低。

22.2.1 转发技术

有些转发技术可以使路由表的大小成为可管理的，同时也可以处理一些问题（如安全性问题）。此处，我们简要讨论这些方法。

下一跳方法与路由方法

第一种减少路由表中内容的技术，称为下一跳方法（next-hop method）。在这种方法中，在路由表中只保留下一跳地址，而不保留完整的路由信息。在路由表中保留完整路由信息的技术，称为路由方法（route method）。路由表各项目必须彼此协调的，图22.2表示使用这种技术可以简化路由表。



默认方法

另一种简化路由表的技术是默认方法 (default method)。在图22.4中, 主机A连接到具有两个路由器的网络上。路由器R1用来将分组转发到连接网络N2的主机。但是, 对因特网的其余部分, 则使用路由器R2。因此, 可不必将整个因特网中的所有网络都列出, 主机A可以仅使用一个称为默认的项目 (通常定义网络地址为0.0.0.0)。

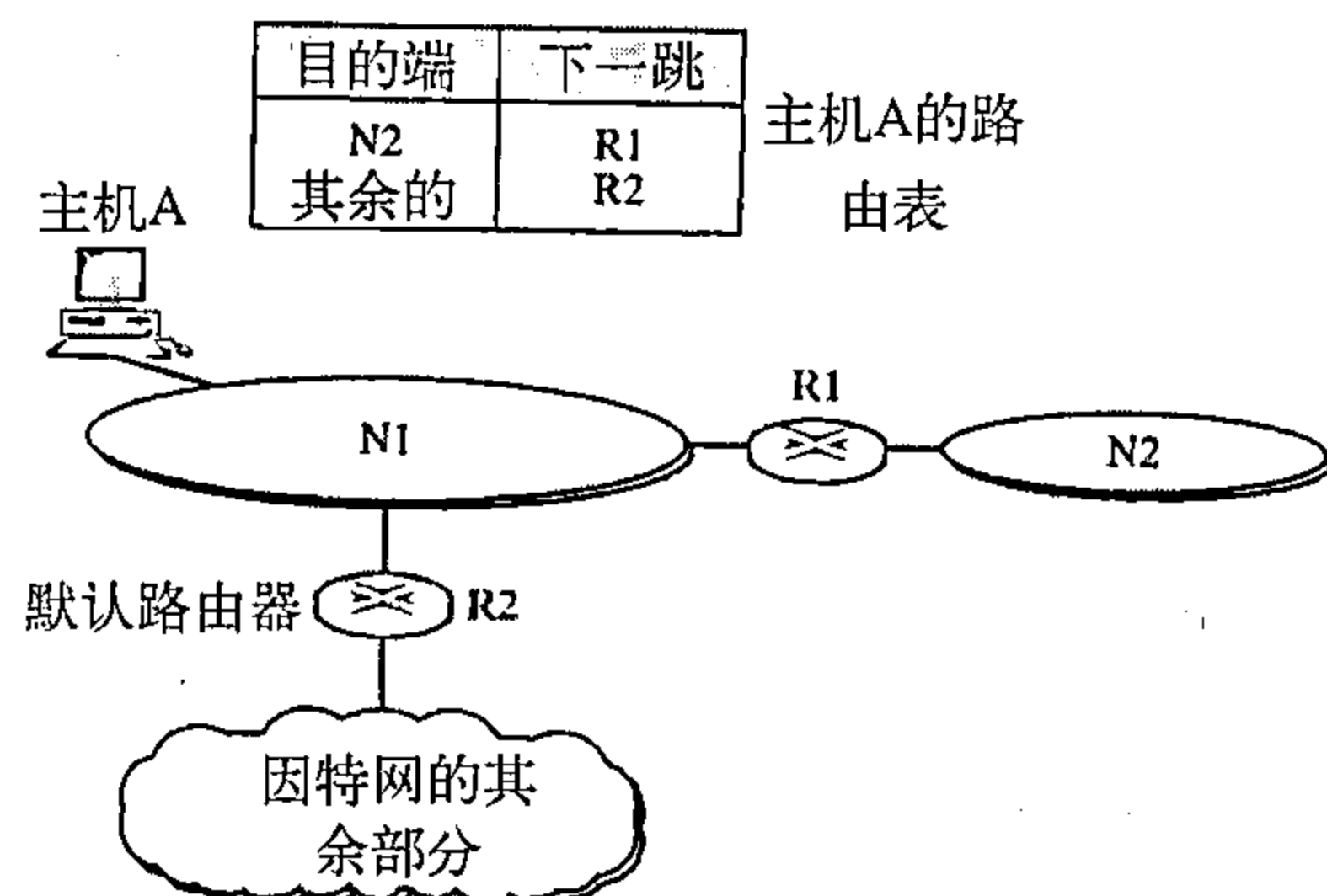


图22.4 默认方法

22.2.2 转发过程

让我们讨论转发过程。假定主机和路由器使用无类寻址, 这是因为分类寻址可以被作为无类地址的一种特殊情况处理。在无类寻址中, 路由表对涉及到的每一个地址块都需要有一行信息。路由表需要根据网络地址 (地址块的第一个地址) 进行查询, 但很遗憾, 在分组中只有目的地址而没有网络地址。为了解决这个问题, 在路由表中需要包含掩码 (/n)。因此, 对相应的地址块, 我们需要有包含该掩码的附加列。图22.5表示了无类寻址的一个简单转发模块。

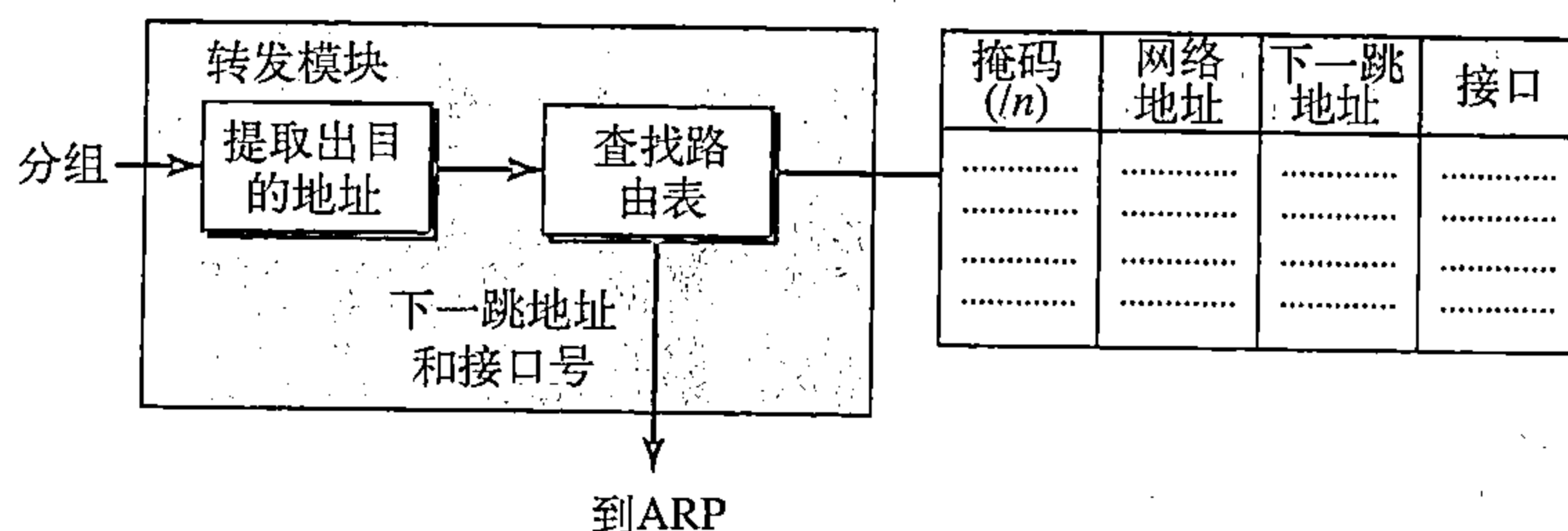


图22.5 无类地址简化的转发模块

注意：在路由表中至少是4列，通常有更多的列。

在无类寻址中，一个路由表至少要有4列。

例22.1

利用图22.6的网络配置，做出R1的路由表。

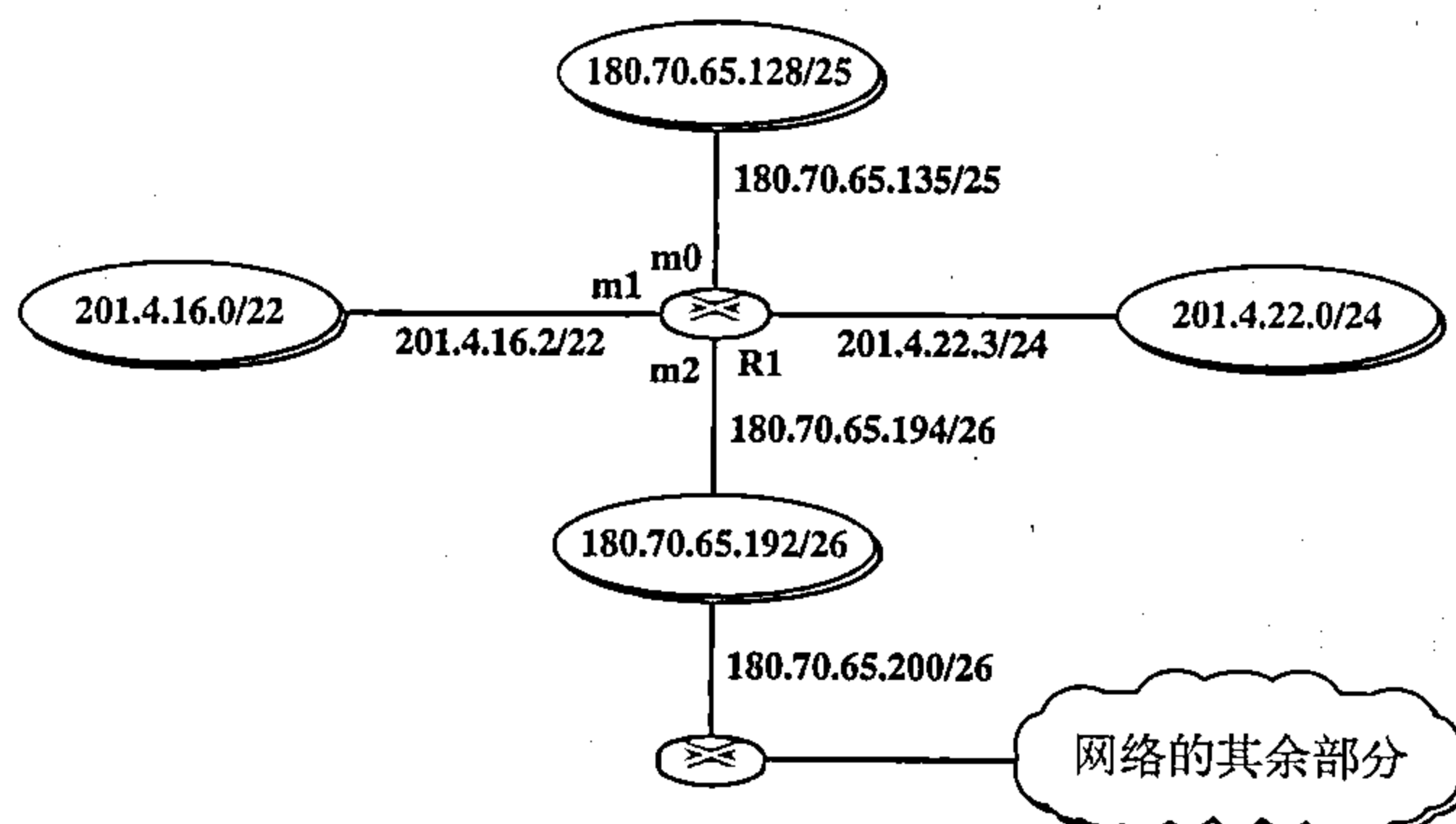


图22.6 例22.1的配置

解:

表22.1是相应的路由表。

表22.1 图22.6中路由器R1的路由表

掩 码	网络地址	下一跳	接 口
/26	180.70.65.192	——	m2
/25	180.70.65.128	——	m0
/24	201.4.22.0	——	m3
/22	201.4.16.0		m1
其余	其余	180.70.65.200	m2

例22.2

如果图22.6中的一个目的地址为180.70.65.140的分组到达路由器R1, 说明其转发过程。

解:

路由器执行下列步骤:

1. 第一个掩码 (/26) 作用于这个目的地址, 其结果是180.70.65.128, 它与对应的网络地址不匹配;
2. 第二个掩码 (/25) 作用于这个目的地址, 其结果是180.70.65.128, 它与对应的网络地址匹配。将下一跳地址 (nex-hop address) (在这种情况下, 它是分组的目的地址) 和接口号 m0传送到ARP做进一步处理。

例22.3

如果图22.6中的一个目的地址为210.4.22.35的分组到达路由器R1, 说明其转发过程。

解:

路由器执行下列步骤:

1. 第一个掩码 (/26) 作用于这个目的地址, 其结果是210.4.22.0, 它与对应的网络地址不匹配 (第一行);
2. 第二个掩码 (/25) 作用于这个目的地址, 其结果是210.4.22.0, 它与对应的网络地址不匹配 (第二行);
3. 第三个掩码 (/24) 作用于这个目的地址, 其结果是201.4.22.0, 它与对应的网络地址匹配。分组的目的地址和接口号 m3传送到ARP。

例22.4

如果图22.6中的一个目的地址为18.24.32.78的分组到达路由器R1, 说明其转发过程。

解:

这时所有掩码作用于这个目的地址都与网络地址不匹配。当它到达表的尾部时, 模块将下一跳地址180.70.65.200和接口号 m2传送到ARP。这就是需要通过路由器转发到因特网某处的输出包。

地址聚合

当我们使用无类寻址时, 很可能路由表的项目数量将会增多。这是因为无类寻址的意图是将整个地址空间划分成可管理的地址块。而路由表增大将增加查找时间。为了缓解这个问题, 我们设计了地址聚合 (address aggregation) 思想。在图22.7中有两个路由器, 路由器R1连接到4个组织机构的网络, 每个组织机构的网络都使用64个地址。而路由器R2在远离R1的某个地方。路由器R1的路由表较长, 这是因为分组必须要正确地路由到适当的组织机构网络。

而路由器R2的路由表很短。对于路由器R2来说，不管到哪一个组织机构，任一目的地址为140.24.7.0到140.24.7.255的分组都是从接口m0发送，因为4个组织机构的地址块被聚合为一个较大的地址块，所以这个称为地址聚合。如果每个组织机构的地址没有聚合成一块地址，那么路由器R2将要有较长的路由表。

注意：尽管地址聚合的思想与子网化的思想类似，但这里没有一个共同的站点；每个组织机构的网络都是独立的。另外，我们还可有若干级的聚合。

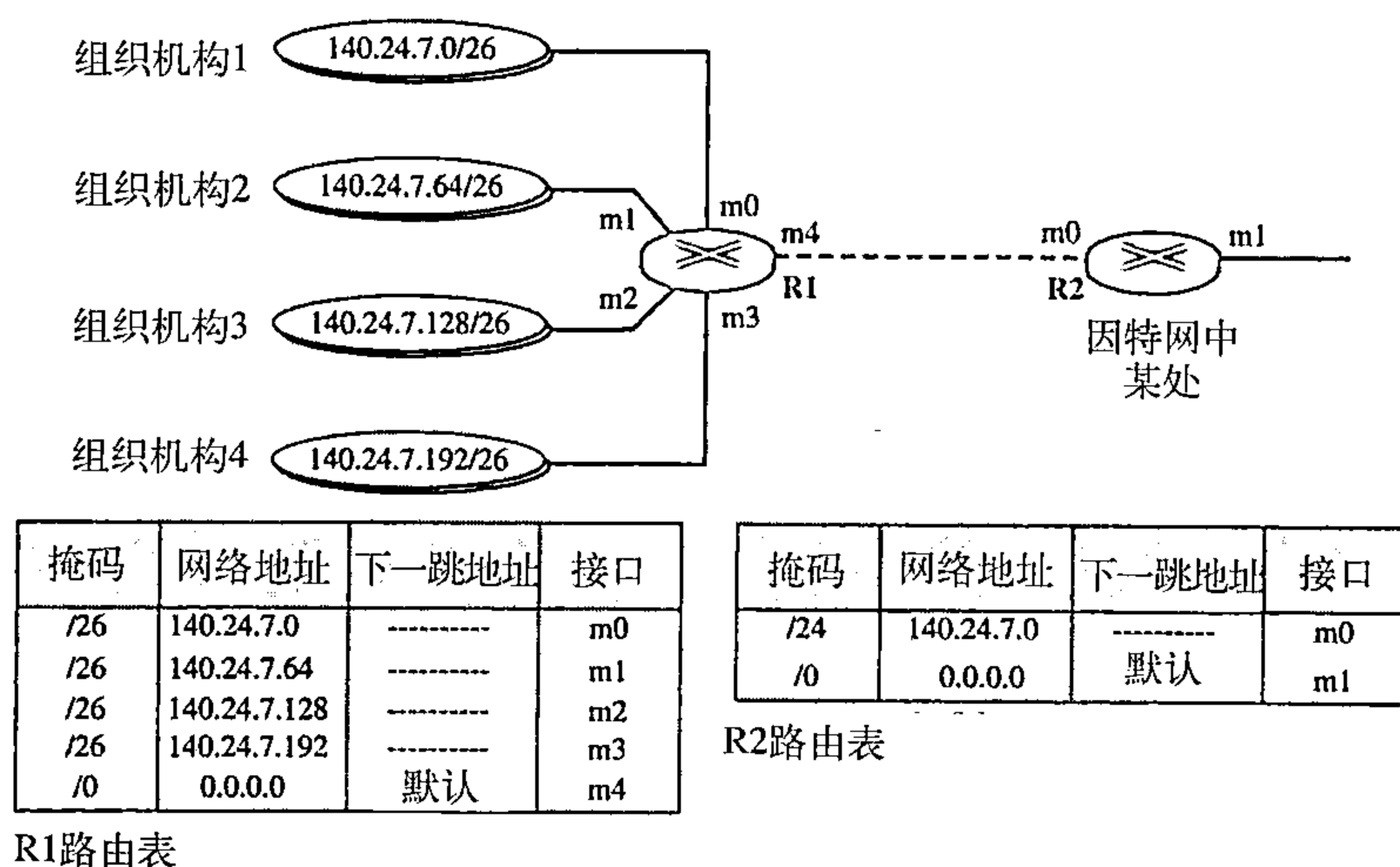


图22.7 地址聚合

最长掩码匹配

如果在图22.7中，其中一个组织机构在地理位置上与其他三个不邻近，将会发生什么？例如，如果组织机构4由于某种原因没有连接到路由器R1，我们能否还能用地址聚合的思想以及还能对组织机构4指派地址块140.24.7.192/26吗？

由于无类寻址的路由选择使用另一个最长掩码匹配 (longest mask matching) 原则，回答是行的。这个原则指明在路由表中掩码存放是按最长的到最短的次序。换言之，如果有三个掩码/27、/26和/24，则/27必定是第一个项目，而/24必定是最后项目。如果用这一原则，我们看看如果组织机构4与另外三个组织机构分离，情形会是怎样，如图22.8所示。

假定一个分组到达目的地址是140.24.200的组织机构4，将路由器R2中第一个掩码作用于它，得到网络地址140.24.7.192。该分组被正确通过接口m1而到达组织机构4。但是，如果路由表不按最长的前缀存储，作用掩码/24将给出分组到路由器R1的不正确路由选择。

分层路由选择

为了解决巨大的路由表问题，我们可在路由表中建立层次结构。在第1章中，我们提到过今天的因特网具有层次结构，就是说因特网被划分为国际的和国内的ISP，国内的ISP又被划分为地区ISP，而地区ISP再进一步划分为本地ISP。如果路由表也像因特网那样具有层次结构，那么可降低路由表的长度。

让我们考虑本地ISP情况。本地ISP可被分配一个单地址块，而该块是具有某一前缀长度的大块。本地ISP将这个块划分为不同大小的一些小块。如果分配给本地ISP的地址块以a.b.c.d/n开始，那么这个ISP可建立以e.f.g.h/m开始的一些块，其中m大于n，而对每个客户，m是可变的。

这如何降低路由表的长度呢？因特网的其余部分不必知道这样的划分。对因特网的其余

部分来说,本地ISP的所有客户都被定义为a.b.c.d/n。在这个大块中的每一个分组的地址都路由到本地ISP。对所有这些客户来说,在世界上的每个路由器中都只有一项,它们都属于同一组。当然,在本地ISP内部,路由器必须识别出子块并路由到目的客户。如果某一客户是一较大的组织机构,那么它也可以通过子网化和划分它的子块为更小的子块(子块的子块)建立另一个层次。在无类路由选择中,只要我们遵循无类寻址规则,层次的级是没有限制的。

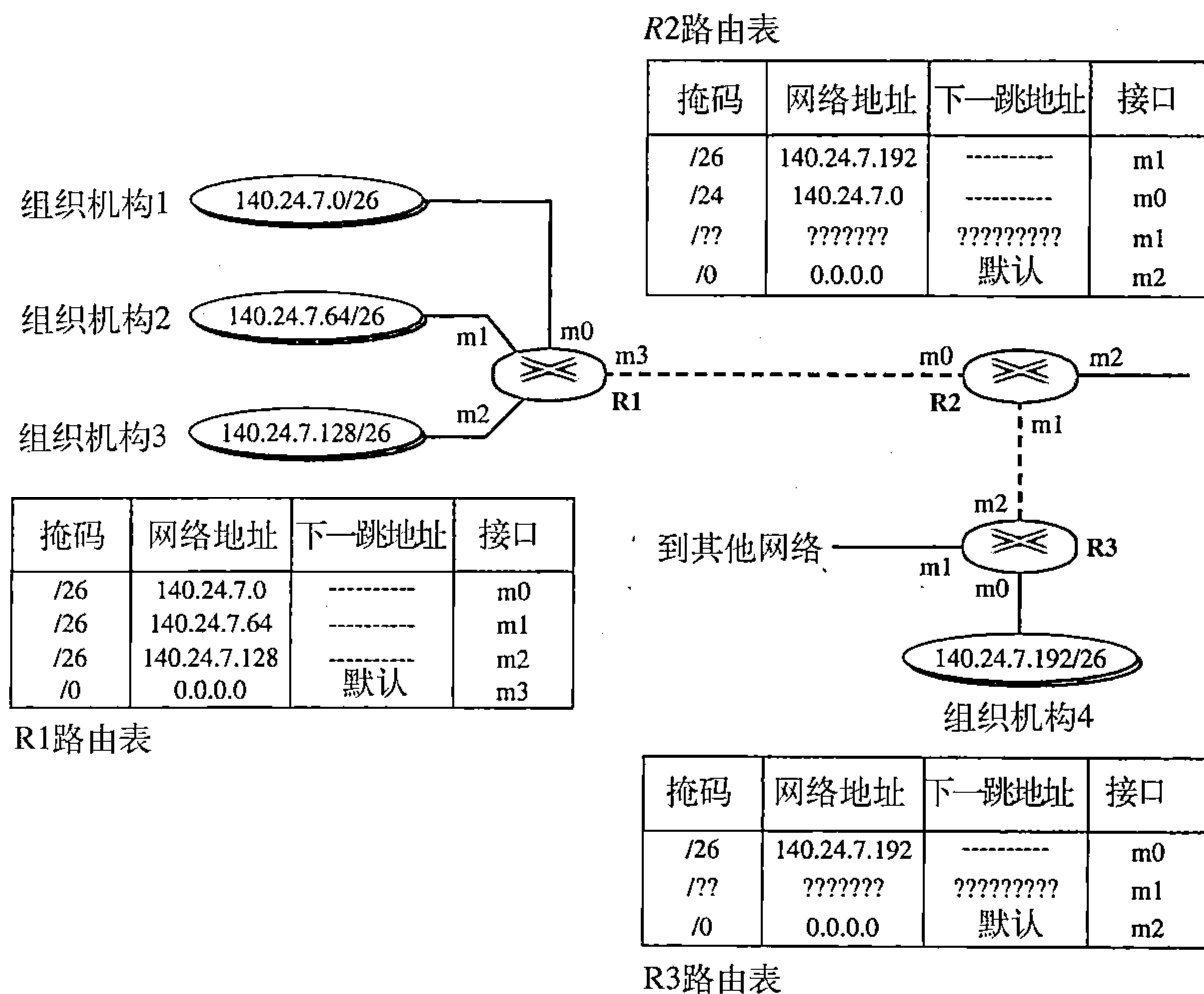


图22.8 最长掩码匹配

例22.5

作为分层路由选择 (hierarchical routing), 让我们考虑图22.9。区域ISP被授予以地址120.14.64.0开始的16 384个地址。区域ISP决定将这个块划分成4个子块, 每块4 096个地址。其中的3个子块分别指派给3个本地ISP, 而第2子块保留作将来使用。注意: 由于原始块的掩码是/18, 因此每块的掩码是/20。

第一个本地ISP将已指派给它的子块划分成8个较小的子块, 每一个指派给一个小的ISP。每个小的ISP对128个家庭 (H001到H128) 提供服务, 每个家庭使用4个地址。注意: 现在由于进一步划分成8个子块, 因此每个小ISP的掩码是/23。因为一个家庭只有4个地址 (2^{32-30} 等于4), 所以每个家庭的掩码是/30。

第二个本地ISP将它的块划分成4个子块, 并将地址指派给4个大型的组织机构 (LOrg01到LOeg04)。注意: 每个大型组织机构有1 024个地址。

第三个本地ISP将它的块划分成16个子块, 并将每个子块指派给一个小型的组织机构 (SOrg01到SOrg16), 每一个小型的组织机构有256个地址, 掩码是/24。

在这种配置中, 存在层次结构。在因特网上的所有路由器将带有目的地址从120.13.64.0到120.14.127.255的分组发送到区域ISP。

区域ISP向本地ISP1发送目的地址为120.14.64.0到120.14.64.255的分组。本地ISP1向H001发送目的地址为120.14.64.0到120.14.64.3的分组。

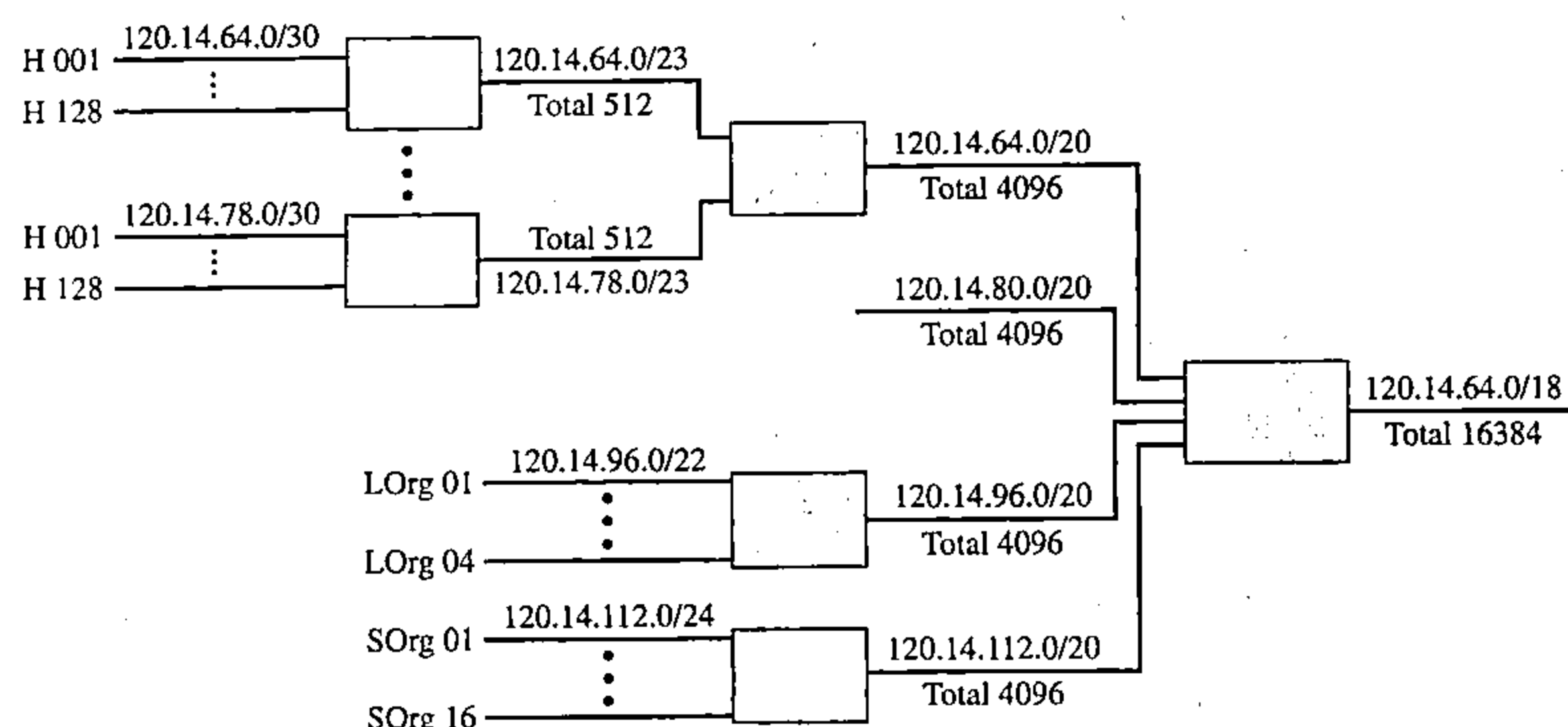


图22.9 ISP的层次结构路由选择

地理路由选择

为了更进一步地减少路由表的大小，我们需要将分层路由选择扩充为扩充包括地理的路由。我们必须将整个地址空间划分为数量不多的几个大块，我们指派一个大块给北美，一个大块给欧洲，一个给亚洲，一个给非洲等。欧洲以外ISP的路由器在它们的路由表中将只有到欧洲的一个项目。在北美以外的ISP路由器在它们的路由表只有到北美的一个项目等。

22.2.3 路由表

现在让我们讨论路由表，主机或路由器要转发IP分组就要有一个路由表，并给每一个目的端设置一个项目。路由表可以静态的，也可以是动态的。

静态路由表

静态路由表（static routing table）包含有人工输入的信息。网络管理人员将每一个目的地址的路由输入到路由表中。当路由表生成后，因特网中的变化无法自动在路由表中进行自动更新。路由表必须由网络管理员手工改变。

静态路由表用在不会经常改动的小型互联网中，或用于故障查找的试验互联网中。对于像因特网这样的大型互联网，静态路由表是极差的方法。

动态路由表

动态路由表（dynamic routing table）使用一个动态路由选择协议，如RIP，OSPF或BGP，因而可以周期性地更新。当因特网中发生变化时，例如当某个路由器关闭或某条链路中断，动态路由选择协议就自动更新所有路由器（最后也将在所有主机中）的路由表。

为了有效地传递IP分组，一个大的互联网如因特网需要动态地更新其路由表。我们将在本章的后面详细讨论三种动态路由选择协议。

路由表格式

正如前面所述的那样，对于无类寻址的路由表的最小列数是4，但是今天的路由表甚至有更多的列数。我们应该明白列数与厂商有关，并不是所有列在不同的路由器中都有。

目前路由器的路由表中的一些常有字段，如图22.10所示。

掩码	网络地址	下一跳地址	接口	标记	引用计数	使用
.....						

图22.10 路由表中常用的字段

- 掩码。这个字段定义应用于该表项的掩码。

- 网络地址。这个字段定义分组必须传递的最终的网络地址。在特定主机路由选择的情况下，该字段定义目的主机的地址。
- 下一跳地址。这个字段定义分组必须传递到下一跳路由器的地址。
- 接口。这个字段显示接口的名称。
- 标记。这个字段定义多达5个标记。所谓标记就是一个通/断开关，它表示或者存在或者不存在。这5个标记是：U（工作）、G（网关）、H（特定主机）、D（由于重定向而增加的）和M（由于重定向而修改的）。

a. U（工作）标记U指出路由器正在工作。如果没有这个标记就表示路由器已不工作，这时分组无法转发因而将被丢弃。

b. G（网关）标记G指出目的端是另一个网络。分组必须传递到下一个跳路由器以便传递（间接传递）。如果没有这个标记，就表示目的端在这个网络中（直接传递）。

c. H（特定主机）标记H指出在地址字段的项目是一个特定主机地址。如果没有这个标志，就表示这个地址只是目的端的网络地址。

d. D（由于重定向而增加的）标记D指出由于ICMP报文中的重定向报文而在主机路由表中增加了到这个目的端的路由选择信息。我们已在第21章讨论过重定向与ICMP协议。

e. M（由于重定向而修改的）标记M指出由于ICMP报文中的重定向报文而修改了到这个目的端的路由选择信息。我们已在第21章讨论过重定向与ICMP协议。

- 引用计数。这个字段给出在任何时候使用本路由的用户个数。例如，如果有5个人在同样的时间从本路由器连接到这个相同的主机，则这一列的值是5。
- 使用。这个字段指出经过本路由器发送到相应的目的端的分组数。

实用程序

有几个用于查找路由信息和路由表的内容的实用程序，我们讨论两个：netstat和ifconfig。

例22.6

在UNIX或者LINUX操作系统下，可用于查找路由信息和路由表的内容一个命令是netstat。下面显示了一个默认服务器的内容列表。我们在命令中使用了两个选项r和n，其中选项r表示我们对路由表感兴趣，而选项n是查找地址的数字形式。注意：这是一个主机的路由表，而不是路由器的。

```
$ netstat -rn
Kernel IP routing table
Destination      Gateway          Mask            Flags           Iface
153.18.16.0       0.0.0.0          255.255.240.0   U               eth0
127.0.0.0         0.0.0.0          255.0.0.0       U               lo
0.0.0.0           153.18.31.254   0.0.0.0         UG              eth0
```

注意：表中列的次序与我们表示的是不同的。此处目的端列定义网络地址。UNIX中的网关（gateway）与路由器同义，该列定义下一跳地址。值0.0.0.0表示传递是直接的。最后一行中的G表示了通过一个路由器（默认路由器）到达目的端。Iface列定义接口，主机只有一个实际的接口eth0，它是指连接到以太网的接口0。第二个接口lo，实际上是虚拟回送接口，它包含主机用回送地址127.0.0.0接收分组。

通过在给定接口（eth0）上使用ifconfig命令可以查找到有关服务器IP地址和物理地址更多的信息。

```
$ ifconfig eth0
eth0 Link encap:Ethernet HWaddr 00:B0:D0:DF:09:5D
inet addr:153.18.17.11 Bcast:153.18.31.255 Mask:255.255.240.0
...
```

从上述信息中，我们可推断出服务器的配置，如图22.11所示。

注意：ifconfig命令给出IP地址和接口的物理（硬件）地址。

22.3 单播路由选择协议

路由表可以是静态的也可以是动态的。静态路由表是由人工输入项目，而动态路由表在互联网中某处有变化时就自动地更新。当今，互联网需要

动态路由表。只要在互联网中有些变化，路由表就应当尽快地更新。例如，当一个路由器不通了，路由表就必须更新，而当发现有一条较好的路由时，路由表也需要更新。

由于需要有动态路由表，因此产生了多种路由选择协议。路由选择协议是一些规则和过程的组合，使得在互联网中的各路由器能够彼此互相通知这些变化。路由选择协议使得旧金山的路由器能够知道在得克萨斯的网络故障。路由选择协议还包括一些过程，用来合并从其他路由器接收到的信息。

22.3.1 优化原则

路由器从网络接收到分组，并将其转发到另一个网络。一个路由器通常连接到多个网络，当它接收到分组时，它应当将分组转发到哪一个网络呢？这个决定是基于最优化原则而做出哪一个可用路径是最佳路径？优化这一名词的定义是什么？

一种方法是给通过网络指定代价，我们称这个代价为度量（metric）。然而，给每一个网络指定的度量取决于协议的类型。有些简单协议，如路由信息选择协议（RIP），同等对待每一个网络，即通过每一个网络的代价都是一样，其跳数为1。因此，如果分组经过10个网络才到达目的端，则总的代价是10跳。

其他协议，如开放最短路径优先（OSPF），允许网络管理员基于所需服务类型指定通过网络的代价，即通过一个网络的路由可以有不同的代价。例如，如果最大吞吐量是所期望的服务类型，则一条卫星链路就比一条光纤线路具有更小的度量；另一方面，如果最小延迟是所期望的服务类型，则光纤线路比卫星链路具有更小的度量。OSPF允许每一个路由器有几个基于服务类型的路由表。

还有一些协议以完全不同的方式定义度量。在边界网关协议（BGP）中，准则就是可由网络管理员设置的策略。策略定义应当选择什么路径。

22.3.2 域内部和域间路由选择

今天的互联网是非常大的，以致于仅使用一个路由选择协议就无法处理更新所有路由器路由表的任务。为此，需要将互联网划分为多个自治系统。自治系统（autonomous system）

是一个单一的管理机构管辖下的一组网络和路由器。在自治系统内部的路由选择称为域内路由选择（interdomain routing）。在自治系统之间的路由选择称为域间路由选择（interdomain routing）。每一个自治系统可以选择一种或多种域内路由选择协议处理自治系统内部的路由选择。但是，处理自治系统之间的路由选择，通常只能使用一种域间路由选择协议（见图22.12）。

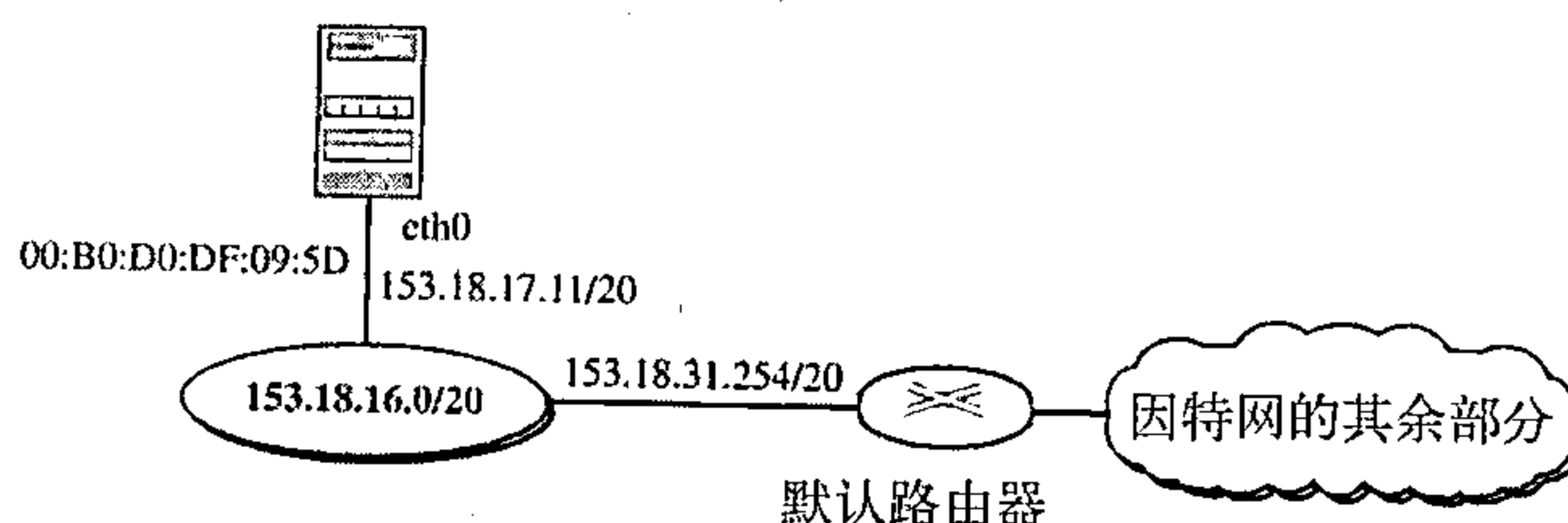


图22.11 例22.6的服务器的配置

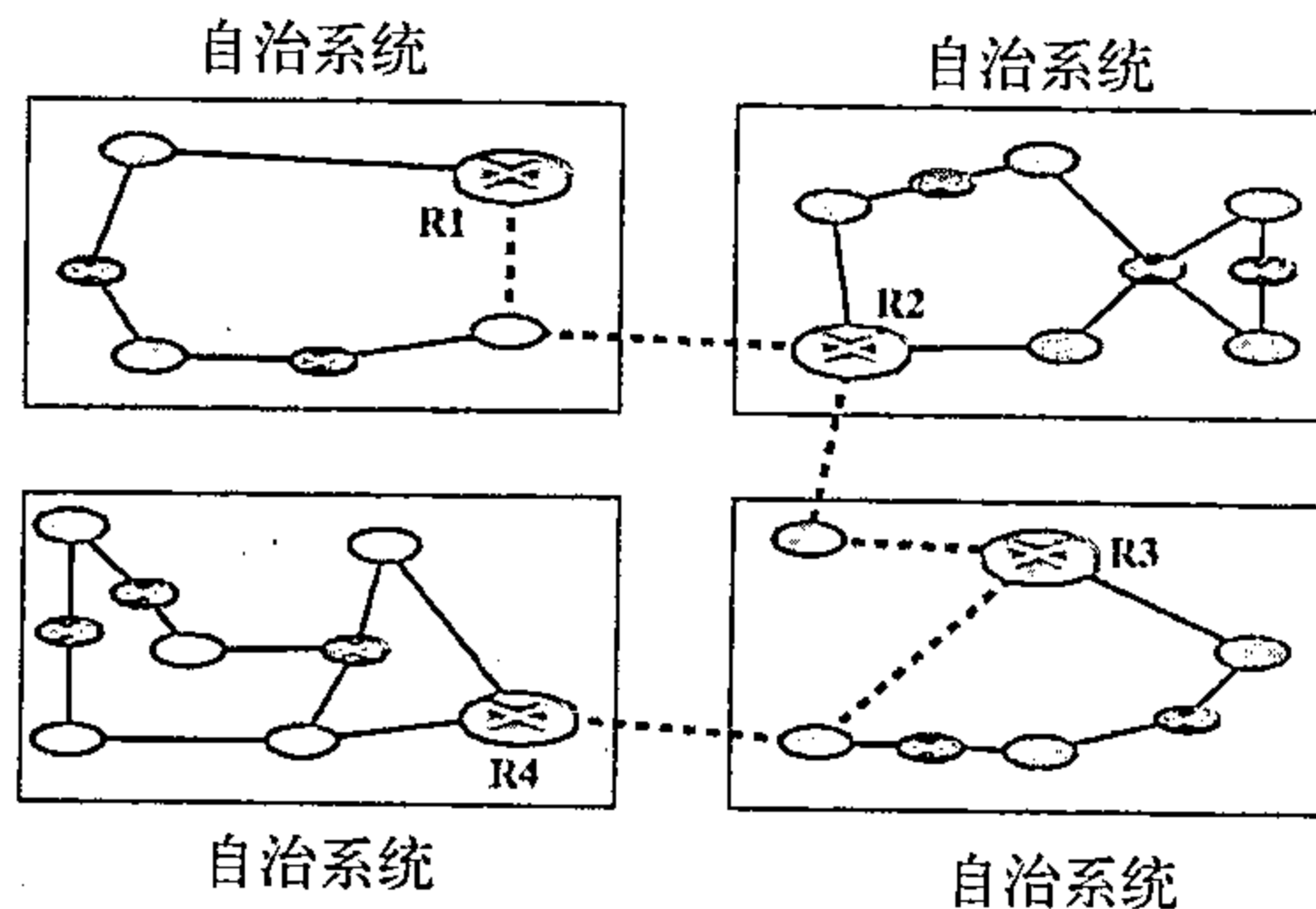


图22.12 自治系统

现有几种域内和域间路由协议在使用。本章只讨论几个最流行的协议。两个域内路由协议：RIP和OSPF，以及一个域间路由协议BGP（见图22.13）。

路由选择信息协议（Routing Information Protocol, RIP）是一种距离向量协议，开放最短路径优先（Open Shortest Path First, OSPF）是一种链路状态协议，边界网关协议（Border Gate Protocol, BGP）是一种路径向量协议。

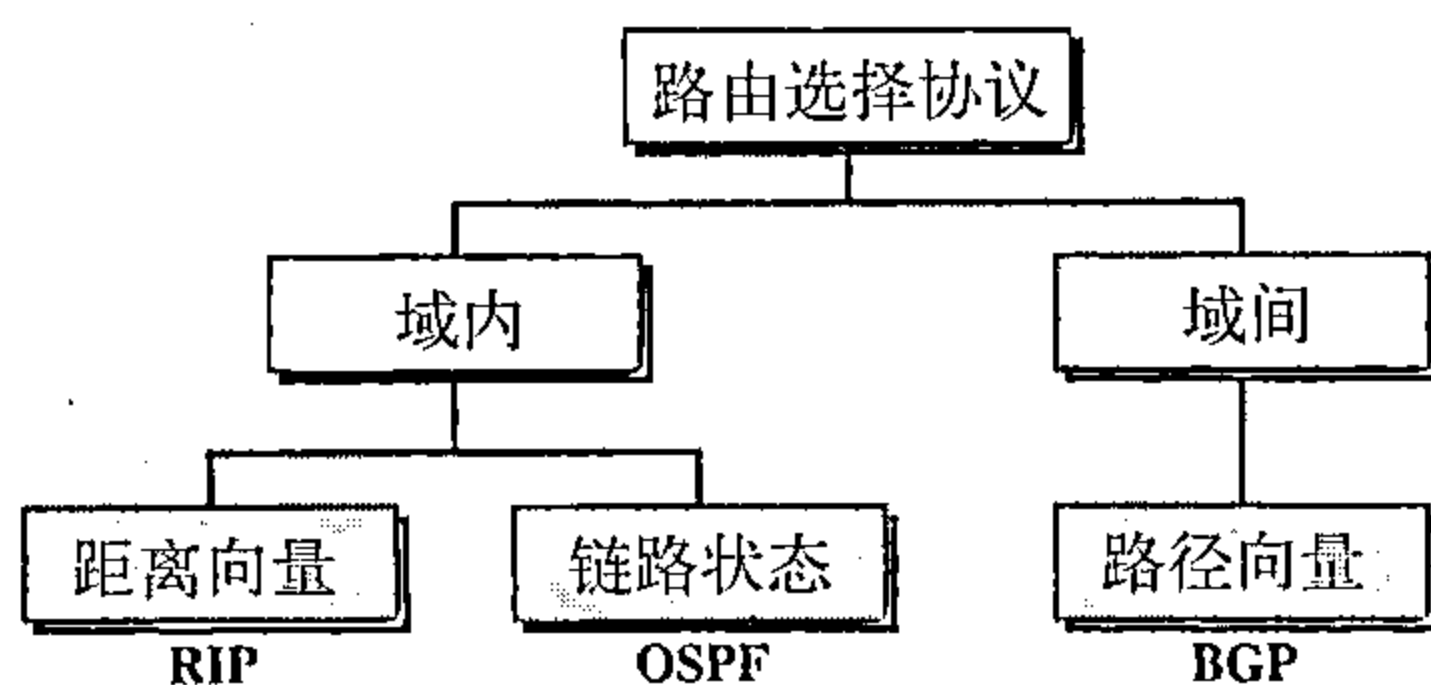


图22.13 流行的路由选择协议

22.3.3 距离向量路由选择

在距离向量路由选择中，任何两个节点之间路由最低代价是最小距离的路径。正如其名字的含义，每个节点都保留一张到其他每个节点的最小向量距离（表）。每个节点也用这张表中所表示的路由中的下一个节点（下一跳路由选择）指导分组流向目的节点。

我们可将节点看做区域中的城市，而将线看做连接城市的道路。一张表可表示城市之间路程最小距离的图。

图22.14表示具有相应表的5个节点的系统。

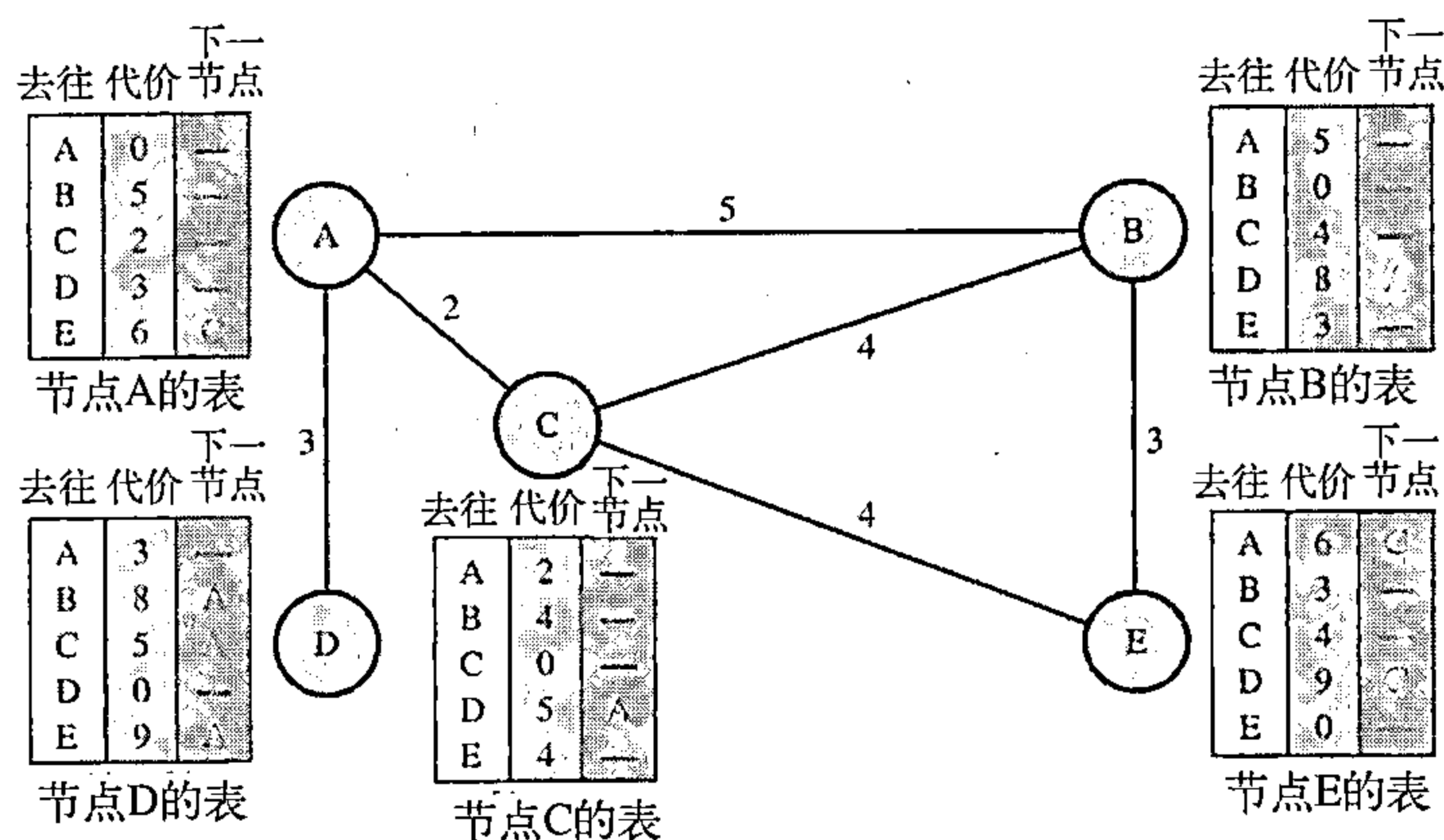


图22.14 距离向量路由选择表

节点A的表说明从该节点如何到达其他节点。例如，到达节点E的代价是6，其路由是经过C。

初始化

图22.14中的表都是稳定的，每个节点都知道如何到达其余任一个节点及其代价。但是，在开始时，并不是这样的。每一个节点仅知道与它直接连接的邻站（immediate neighbor）的距离。到目前为止，我们假定每一个节点能向邻站发送报文并得到与这些邻站的距离。图22.15表示了每个节点的初始表，其中不是邻站的距离标为无穷大（不可达）。

共享

距离向量路由选择的总的思想是与邻站共享路由信息。尽管节点A不知道节点E，但节点C知道节点E。如果节点A共享节点C的路由表，那么节点A也知道如何到达节点E。另外，节点C不知道如何到达节点D，但节点A知道。如果节点C共享节点A的路由表，那么节点C也知道如何到达节点D。换言之，相邻站两个节点A和C如果彼此帮助，那么就可以改进它们的路由表。

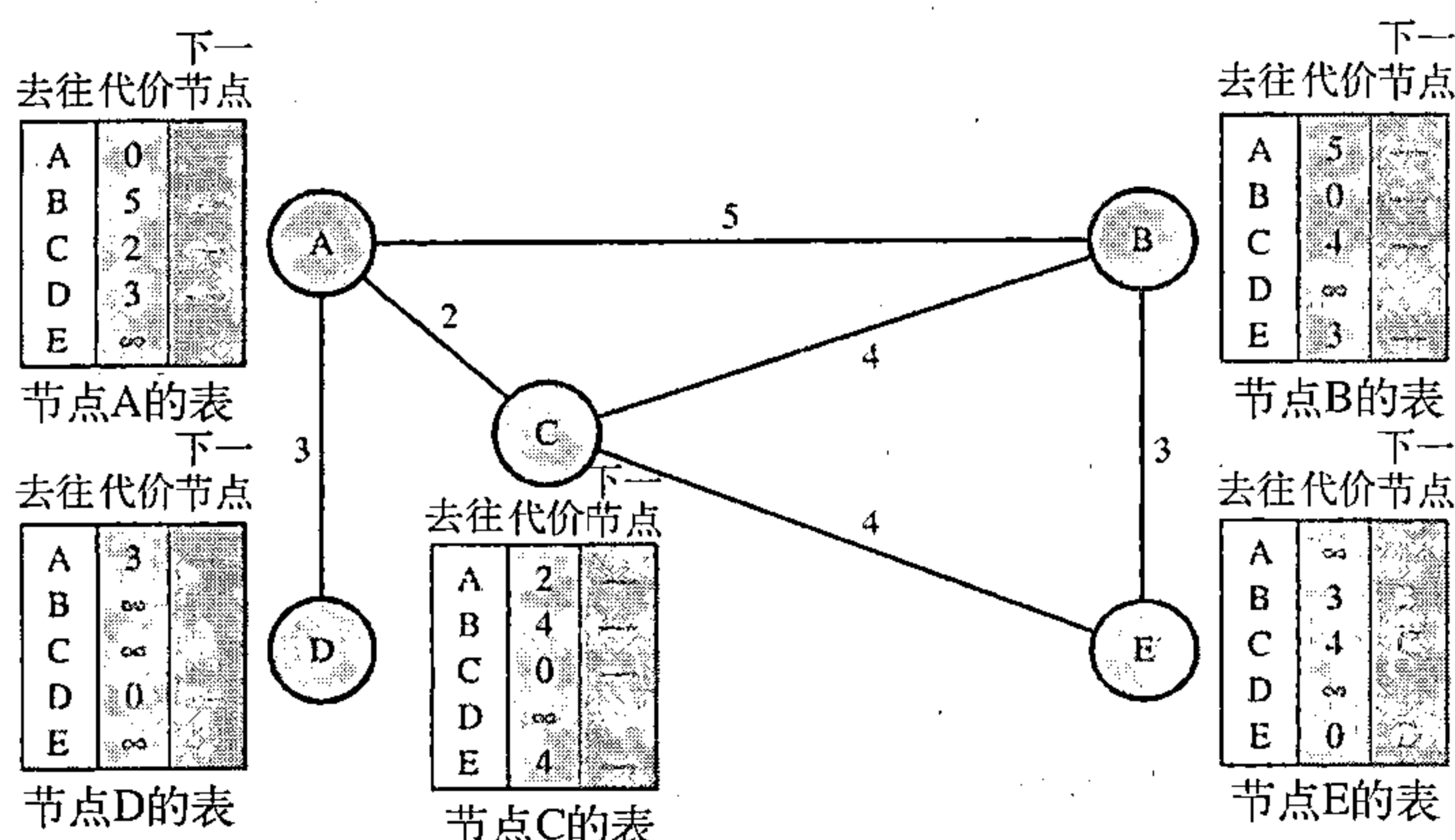


图22.15 距离向量路由选择的初始表

只还有一个问题，与每个邻站需共享表中多少项？一个节点并不知道邻站的表，解决的最好方法是每个节点向邻站发送它的完整表，让邻站决定哪些项目是有用的，哪些项目应丢弃的。但是表中第三列（下一节点）对邻站是没有用处的，因此，当邻站接收到一个表时，第三列用发送方的节点名替代。如果有一行可以用，则下一节点是表的发送节点。所以一个节点仅向任一邻站发送它的表中前二列。换言之，这里的共享是指仅共享前二列。

在距离向量路由选择中，每个节点与它的邻站周期性地或有变化时共享它的路由表。

更新算法

当一个节点从它的邻站接收到二列的表时，它需要更新它的路由表。更新用三个步骤：

1. 接收节点在表的第二列中的每一个值加上它与发送节点之间代价的值，这是显然的。因为节点C声称它到目的距离是 x 英里，而节点A和C的距离是 y 英里，则A与目的地的距离是 $x+y$ 英里。

2. 如果接收节点使用来自任一行的信息，接收节点需要把发送节点名加入作为第三列，也就是发送节点作为路由的下一个节点。

3. 接收节点将修改过的接收到的表与它的旧表的相应行进行逐行比较。

a. 如果下一个节点项目不同，则接收节点选取具有最小代价的行；如果最小代价相同，则保持旧的。

b. 如果下一个节点项目相同，则接收节点选取新行。例如，假定节点C在前面已经告知到节点X的距离是3。现在假如节点C到节点X没有路径，即节点C现在告知这条路径距离为无穷大。节点A不能不理睬这值，即使它的旧项目较小。旧的路径不再存在，新的路径距离为无限大。

图22.16表示了节点A在接收到来自节点C的部分表后，如何更新它的路由表。

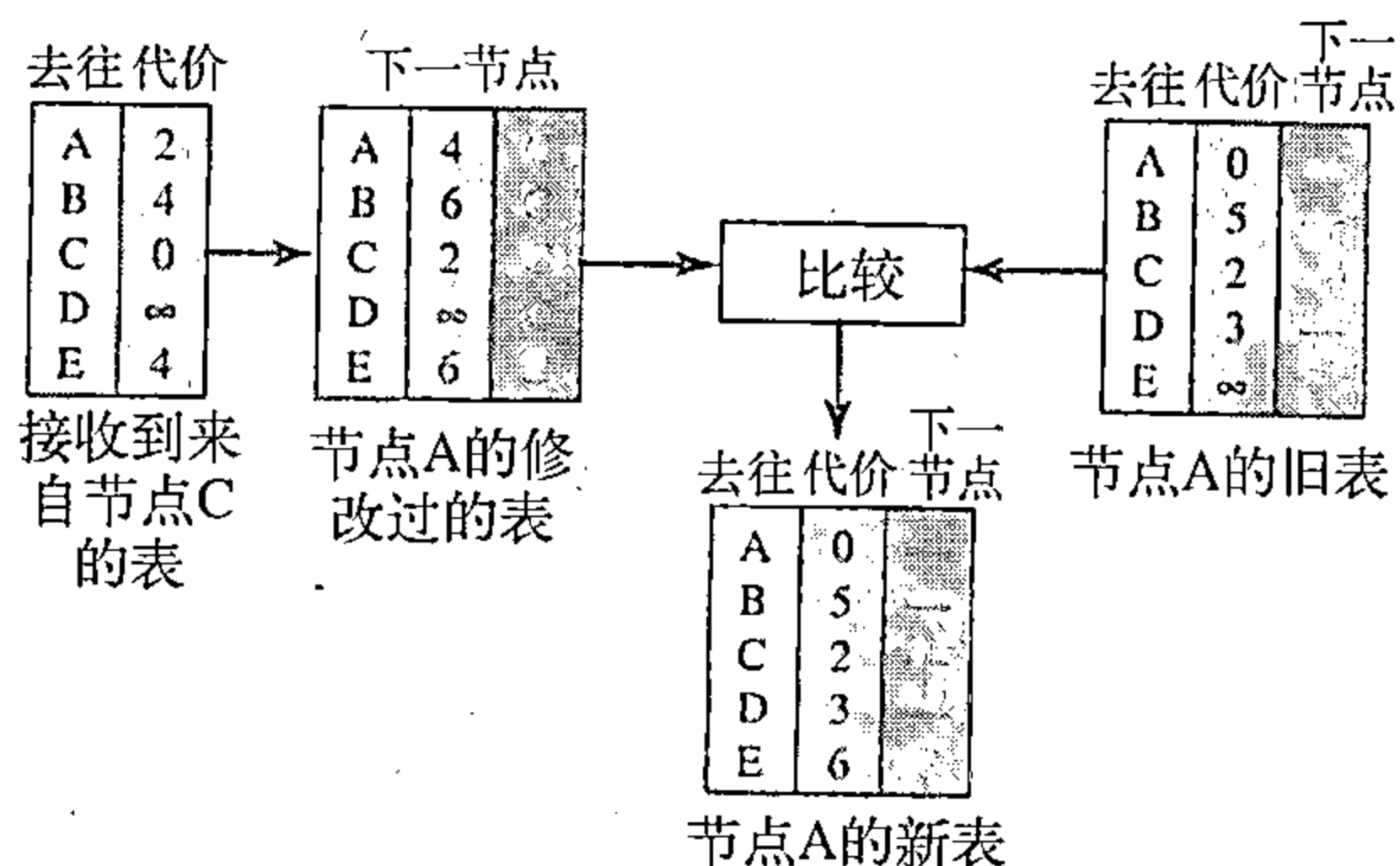


图22.16 距离向量更新

这里需要强调几点。第一，任何数加无穷大仍为无穷大。第二，修改过的表说明了如何从节点A经过节点C到达节点A。如果节点A需要通过节点C到达它自己，则必须去往节点C然后返回，其距离为4。第三，修改节点A的唯一好处是最后的项目，原先节点A不知道如何到达节点E（距离是无穷大），而现在知道通过

节点C代价是6。每一节点可用来自其他节点的表更新它的表。在短时间内，如果网络本身没有变化，如链路故障等，那么每一个节点达到一个稳定状态，它的表的内容保持不变。

何时共享

现在问题是一个节点何时向所有邻站节点发送部分的路由表（仅二列）？周期性地和当表有改变的时，就发送该表中的两列。

周期更新 通常，每隔30秒，节点发送它的路由表一次，定期更新。其更新时间依赖于所用的距离向量路由选择的协议。

触发更新 节点在它的路由表有变化时，向它的邻站节点发送它的二列路由表，这称为触发更新（triggered update）。变化是由下列的原因引起：

1. 节点接收到邻站的表，引起它自己表的更新；
2. 节点检测到邻站链路有故障。

两个节点循环不稳定性

距离向量路由选择的一个问题就是不稳定性，就是说使用这种协议的网络可能成为不稳定的。要理解这个问题，让我们考察图22.17的情况。

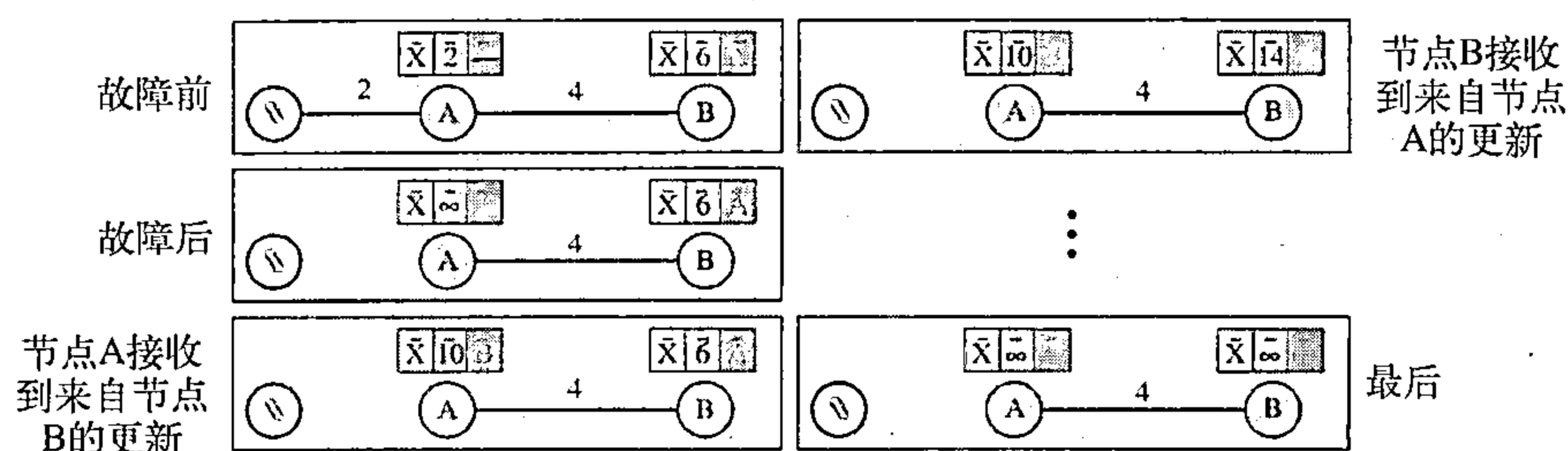


图22.17 两个节点不稳定性

图22.17表示一个有三个节点的系统。我们仅表示了与讨论需要有关的路由表部分。开始时，两个节点A与B都知道如何到达节点X。但突然A与X断开，节点A改变它的路由表。如果A立即发送它的表给B，一切都正常。但是，在B接收A的路由表之前，B已发送它的路由表，该系统变成不稳定。节点A接收B发来的路由表，它假定B已找到了到达X的路径，立即更新它的路由表。基于触发更新策略，A向B发送它的新更新的路由表。现在B认为A的周围环境已改变并更新它的路由表，于是到达X的代价逐渐地增加直到无穷大。在这时刻，A与B两人都知道X不可达。但是，在这个过程中，系统是不稳定的。节点A认为通过B有路径到达X，而节点B认为通过A有路径到达X。如果A接收到目的地为X的一个分组，它向B发送，然后返回到A。同样的，如果B接收到目的地为X的一个分组，它向A发送，然后返回到B。这样分组在A与B之间来回往返产生循环问题。对于这类不稳定性已经提出几个解决办法。

定义无穷大 第一个明显的解决办法是将一个较小的数值定义为无穷大，例如100。对于前述的情况，系统将在小于20次更新下是不稳定的。事实上，大多数距离向量路由协议的实现都定义两个相邻的节点距离为1，并认为16为无穷大。这就是说距离向量路由协议不能用于大的系统，网络的规模在每个方向不能超过15次跳。

分割范围 另一个解决办法是分割范围（split horizon）。在这种策略中，不是通过每一个接口发送整个表，而是每一个节点通过每一个接口仅发送它的表的一部分。按照它的表，如果节点B认为通过A到达X是最佳路径，则节点B不需要向A通知这一部分信息，因为这一信息来自A（A已知道），把来自节点A的信息修改后，发送回节点A可能会产生混淆。在前面图22.17情况下，节点B在向节点A发送它的表之前，就删除它的路由表中最后一行，节点A仍保

持到X的距离为无穷大。稍后，当节点A发送它的路由表到节点B时，节点B也修改它的路由表。在第一次更新后，系统成为稳定，两个节点A和B都知道X是不可达的。

分割范围与毒性逆转 使用分割范围策略有一个缺点，距离向量路由协议经常使用定时器。如果没有关于一条路径的新信息，那么节点就在它的表中删除该路径。在前面情况下，当节点B删除它向节点A的通知中到X的路径时，节点A不能推断这是由于分割范围策略（信息源是A），还是因为节点B最近没有接收到关于X的消息。分割范围策略可与毒性逆转（poison reverse）策略结合起来。节点B依旧可通知X的值，但如果信息源是节点A，则它用无穷大表示距离作为警告“不要使用这个值，我知道的这个路径来自你。”

三个节点不稳定性

二个节点不稳定性可用分割范围策略与毒性逆转策略结合起来避免。但是，如果不稳定性存在于三个节点间，则稳定性不能保证。如图22.18所示。

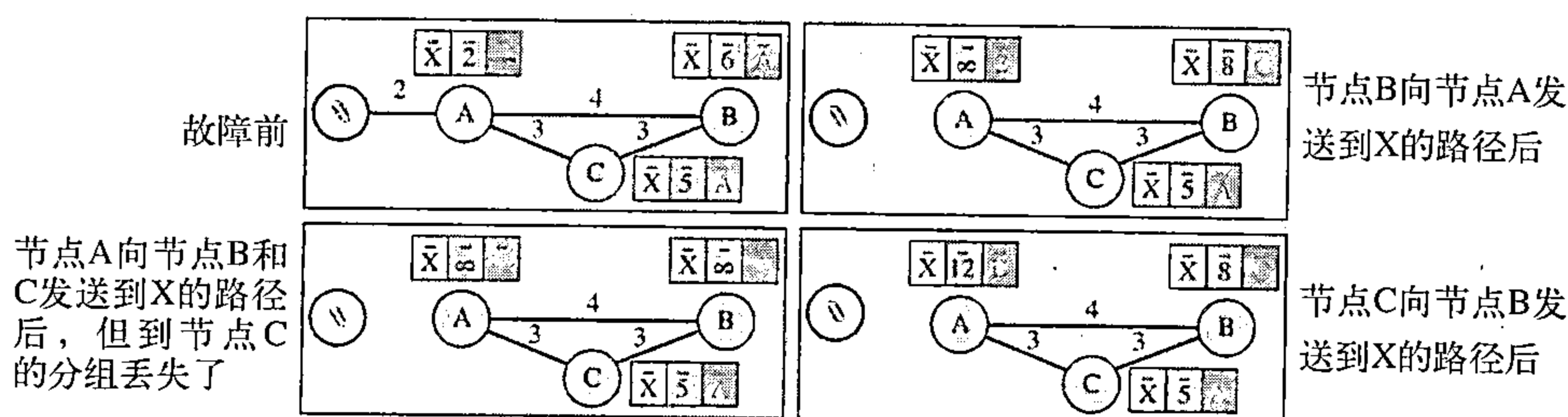


图22.18 三个节点不稳定性

假设发现节点X不可达后，节点A发送一个分组给节点B和C通知它们路径的情况，节点B立即更新它的表，可是到C的分组在网络中丢失，因此永远到达不了C。节点C根本不知道，仍认为有一条通过A到达X的距离为5的路径。稍后，节点C向B发送它的路由表，其中包含有到X路径。节点B此时完全糊涂了。它接收到从C到X的路径信息并按照算法更新它的路由表，其中从C到达X的代价为8。这个信息来自C而不是A，稍后节点B向节点A通知这条路径。节点A糊涂了并更新它的路由表，指出A通过B可达到X，其代价为12。当然，这个循环在继续。现在节点A对C而不对B通知到达X的路径增加了代价，然后节点C用增加的代价向节点B通知该路径。节点B向节点A重复相同的操作等。当每个节点的代价到达无穷大时，循环停止。

RIP

路由选择信息协议（Routing Information Protocol）是一个在自治系统内部使用的域内路由选择协议。它是基于距离向量路由选择的一个非常简单的协议。RIP基于下列考虑直接地实现距离向量路由选择：

1. 在一个自治系统中，包括了路由器和网络（链路）。路由器有路由表，而网络没有路由表；
2. 路由表中的目的端这一列是网络，这表示它的第一列定义了目的网络地址；
3. RIP所用的度量很简单，距离定义为到达目的端的链路（网络）个数。因此，RIP的度量称为跳数（hop count）；
4. 16就定义为无穷大，就是说在使用RIP的任何自治系统中，任何路径不能大于15跳；
5. 下一个节点这一列定义为被发送分组所要到达的目的路由器的地址。

图22.19表示了一个具有7个网络和4个路由器的自治系统，也表示了每个路由器的路由表。让我们考察R1的路由表，该表有7个项目，表示了在这个自治系统中如何到达每个网络。路由器R1直接连接到网络130.10.0.0和130.11.0.0，这就是说对于这两个网络没有下一跳项目，向

左边远方3个网络之一发送一个分组，路由器R1需要R2传递该分组。对这3个网络的下一跳项目是IP地址为130.10.0.1的R2接口。向右边远方2个网络发送一个分组，路由器R1需要IP地址为130.11.0.1的路由器R4。其他表可相似地解释。

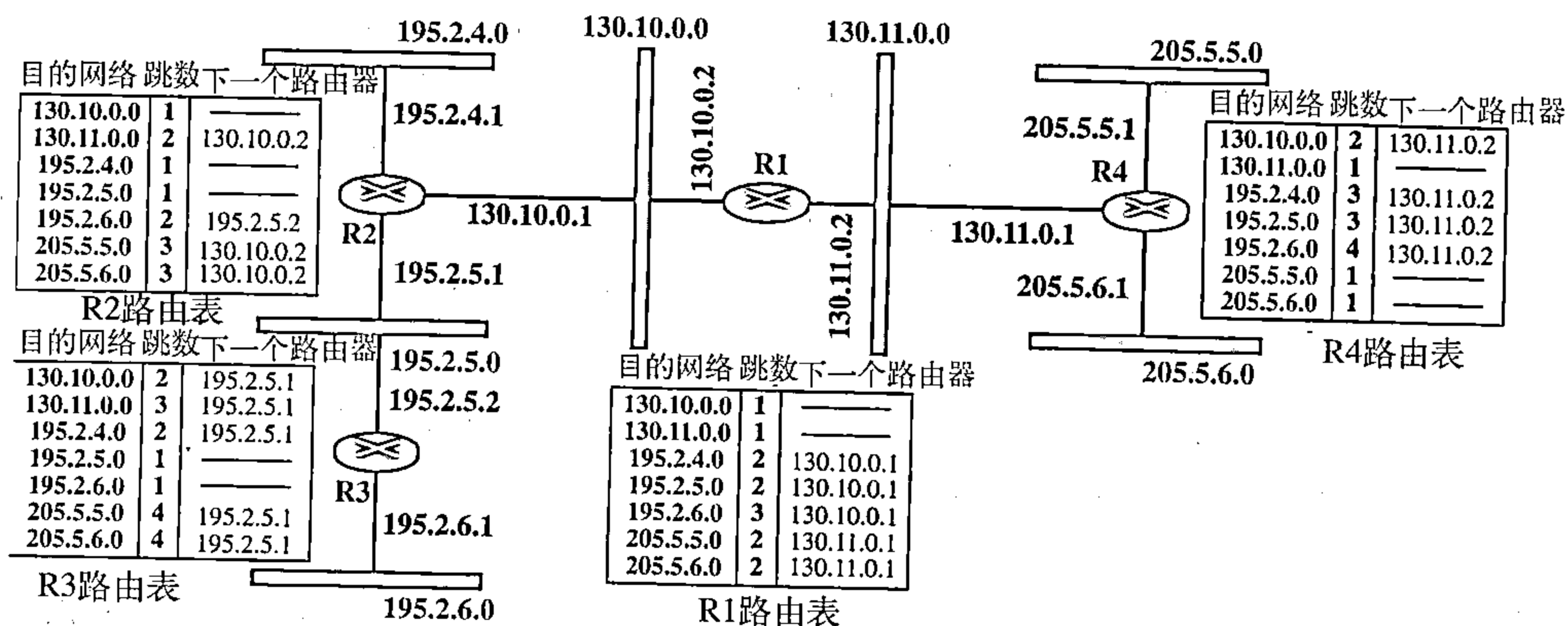


图22.19 使用RIP的区域例子

22.3.4 链路状态路由选择

链路状态路由选择与距离向量路由选择的基本原理不同。在链路状态路由选择中，区域中的每一个节点拥有该区域的全部拓扑结构（所有节点和链路的列表，它们如何连接包含类型、代价也就是度量和链路的接通或断开的情形）。根据这个拓扑结构，一个节点可用Dijkstra的算法（Dijkstra's algorithm）建立一个路由表。图22.20表示了这一概念。

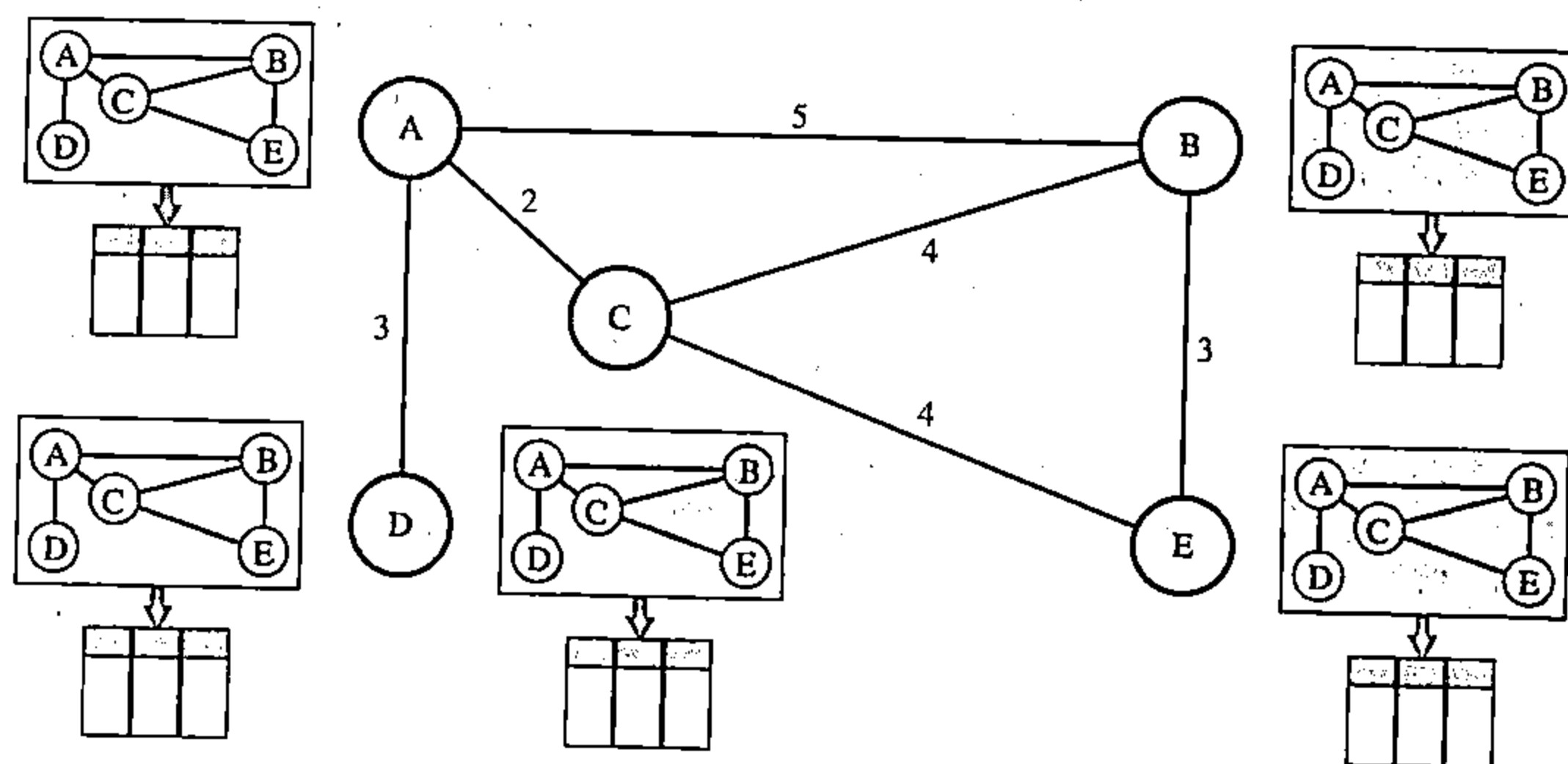


图22.20 链路状态路由选择的概念

该图显示了具有5个节点的简单区域，每个节点使用相同的拓扑建立各自的路由表，但是由于计算是基于该拓扑的不同解释上，所以每个节点的路由表是唯一的。这类似于一个城市的地图，每个人都可有同样的地图，但到达他特殊目的地却有不同路径。

拓扑必须动态地表示每一节点最近的状态和链路。如果网络中的任一点有改变（例如，某一链路断开连接），则必须更新每个节点的拓扑。

如何使公共拓扑成为动态的以及如何存储在每个节点中？没有一个节点在开始时或在网络上的某处发生变化后就能知道该拓扑。链路状态路由选择是基于下面的假设，虽然不知道拓扑的全局知识，但每个节点知道自己这一部分的链路状态（如类型、状态和代价）。换言之，整个拓扑可由每一个节点的部分知识复合而成。图22.21的区域与图22.20的区域相同，但它包含了每个节点部分的知识。

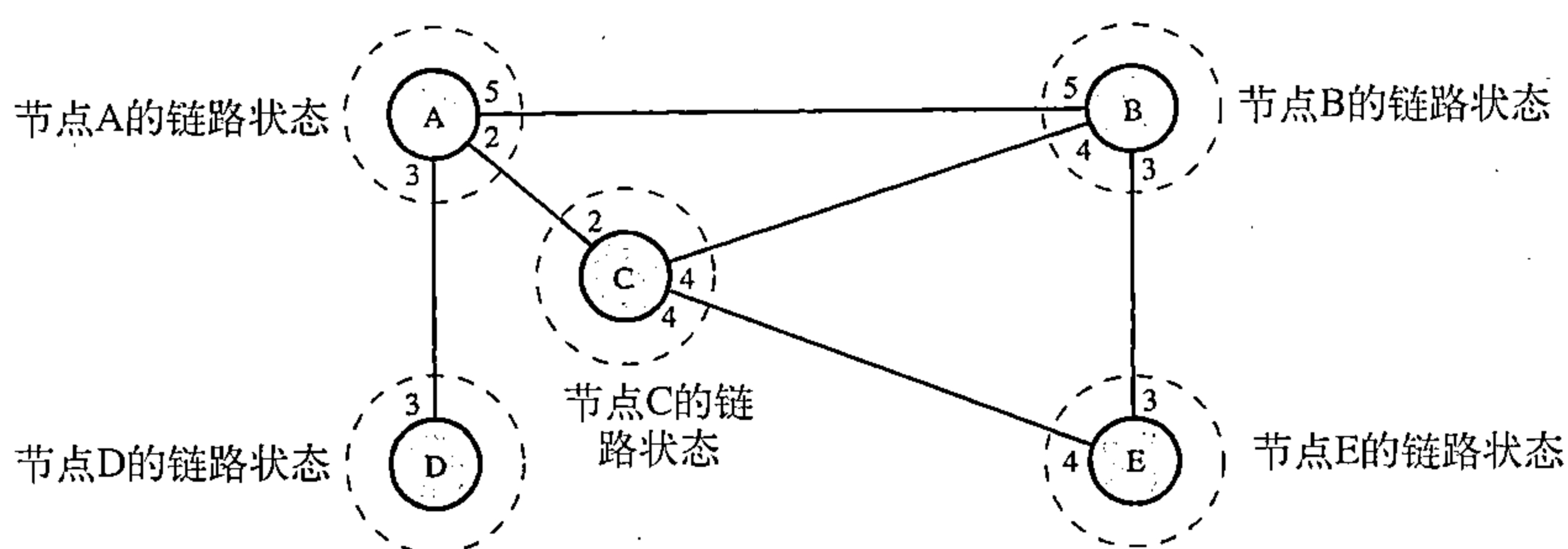


图22.21 链路状态知识

节点A知道它与节点B连接的度量为5，与节点C连接的度量为2，与节点D连接的度量为3。节点C知道它与节点A连接的度量为2，与节点B连接的度量为4，与节点E连接的度量为4。节点D知道它与节点A连接的度量为3，等等。虽然知识有重叠部分，但这个重叠部分保证了公共拓扑的建立（对于每一个节点的整个区域图）。

建立路由表

在链路状态路由选择（link state routing）中，为了保证每个节点对于其余每一节点都具有最小成本的路由表，需要做4件事：

1. 按每个节点建立称为链路状态分组（LSP）的链路状态；
2. 用一种有效而可靠的方法向其他每个路由器扩散LSP，这称为洪泛（flooding）；
3. 为每个节点构成一个最短路径树；
4. 基于最短路径树计算路由表。

链路状态分组（LSP）的生成 链路状态分组可携带大量信息。但是，我们假定它携带最小的数据量：节点的标识、链路的清单、序列号和寿命。前两个是节点标识和链路清单，它们是生成拓扑所需要的。第3个是序列号，它用于洪泛和区别新与旧的LSP。第4个是寿命，它防止旧的LSP在区域中长期保留。在两种情况下产生LSP：

1. 当区域的拓扑有变化时，触发LSP扩散是快速通知区域中任一节点更新它的拓扑主要方法；
2. 基于周期性产生。在这种情况下，其周期比距离向量路由选择的周期更长些。事实上，对于这种类型的LSP扩散没有实际的需要。它是保证旧的信息从区域中除去。因为周期的扩散以实现为基础，定时器设定的范围通常是 60 分或 2小时。一个比较长的周期保证洪泛在网络上不会产生太多的通信量。

LSP洪泛法 一个节点已准备好LSP之后，它必须向其余所有的节点扩散，不仅只是邻站。这个过程称为洪泛法，而洪泛方法按下列几步进行：

1. 创建节点的LSP，并从每个接口发送LSP的副本；
2. 接收到LSP的每个节点与已有的副本比较。如果新到达的LSP是比较旧的（用列序号检验），则丢弃该LSP；如果是比较新的，则该节点执行下面的工作：
 - a. 它丢弃旧的，保留新的；
 - b. 节点通过它的每个接口，但除掉接收分组的接口，再发送分组的副本。这将保证了洪泛在区域的某处停止（此时仅有一个接口）。

最短路径树的构成：Dijkstra算法 接收到所有的LSP后，每个节点就有了完整的拓扑副本，但是对到其余每个节点找出最短距离还是没有完成，需要有最短路径树（shortest path tree）。

树是由节点与链路组成的图，其中有一个节点称为根，所有其他节点都可从根开始通过

唯一的一条路径到达。最短路径树是根与其他节点之间路径最短的一个树。对每个节点，我们需要有该节点为根的最短路径树。

Dijkstra算法 (Dijkstra algorithm) 是从图创建一个最短路径树。算法将节点划分成两个集合：试验的和永久的。算法寻找一个现行节点的一些邻站节点，将它们作为试验的，检查它们。如果它们能通过一些准则，则将它们变为永久的。我们可以用图22.22中的流程图形式化地定义该算法。

让我们用该算法应用于节点A，如图22.23所示。在每步中寻找最短路径。我们需要计算从根到每个节点的累计代价，已显示在节点旁边。

下面表示这些步骤，在每步结束，我们表示了永久（已决定的）节点和试验（未决定的）节点以及累计代价。

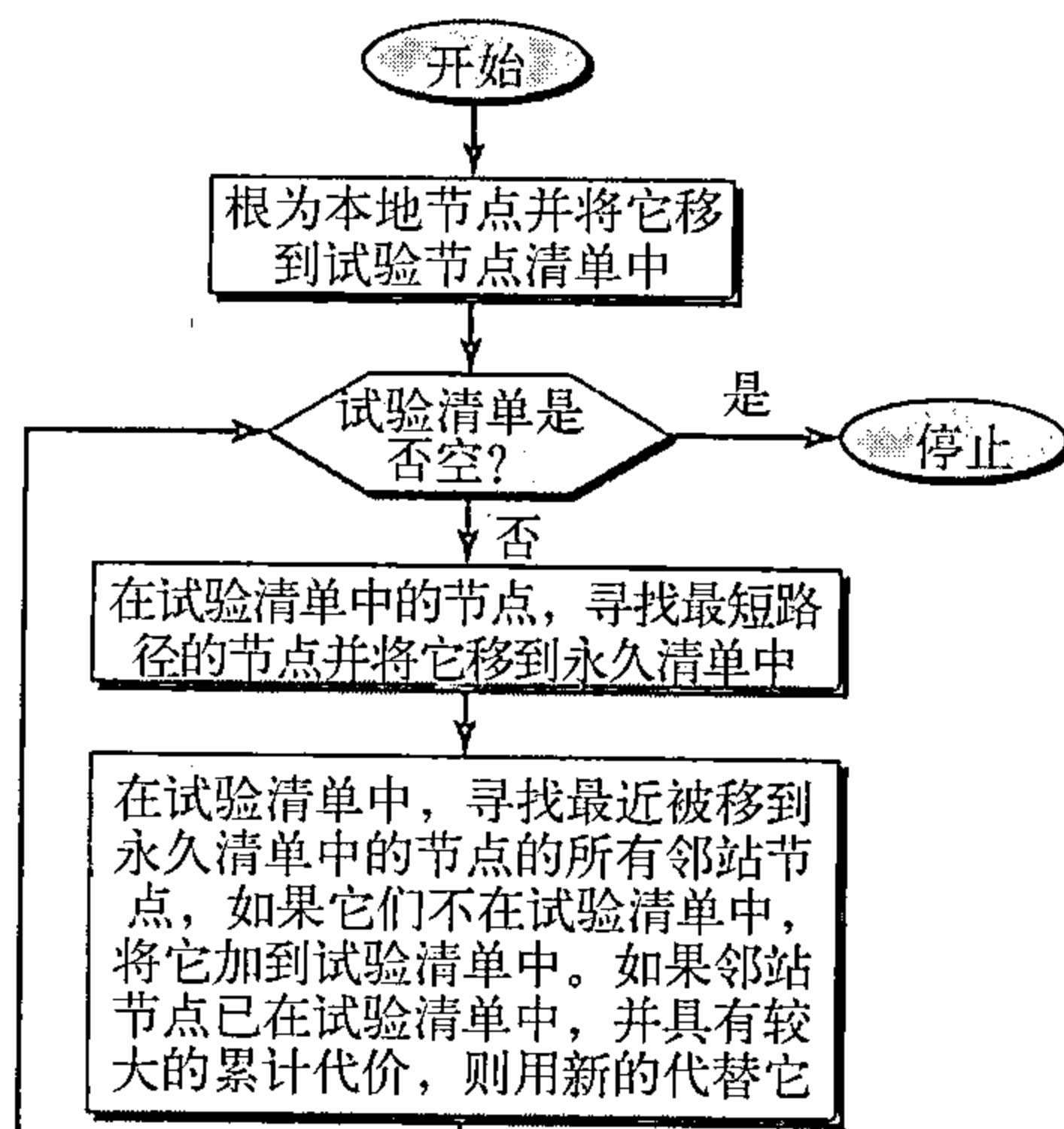


图22.22 Dijkstra算法

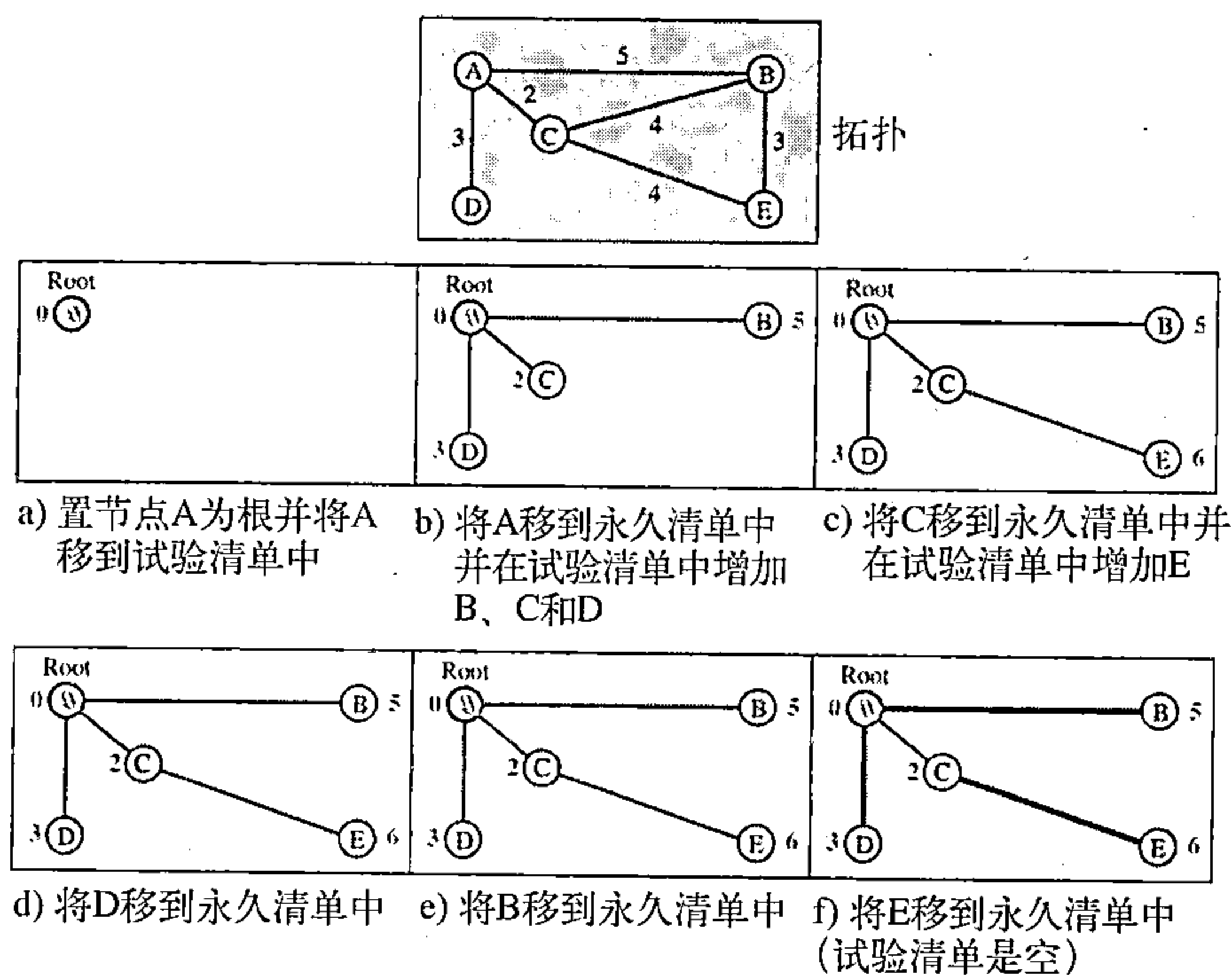


图22.23 最短路径树构成的范例

1. 我们设置节点A为树的根，并将它移到试验清单中。两个清单是：

永久清单：空

试验清单：A (0)

2. 节点A与试验清单中的所有节点有最短的累计代价，我们将A移到永久清单中，并在试验清单中增加A的所有邻站。两个新的清单是：

永久清单：A (0)

试验清单：B (5)，C (2)，D (3)

3. 在试验清单中的所有节点，节点C有最短累计代价，我们将C移到永久清单中，节点C有3个邻站，但节点A已处理，那些没有处理的是B和E。但是B已在试验清单中而累计代价为5，节点A也可经过C到达节点B，其代价为6。因为5小于6，我们在试验清单仍保留代价为5的节点B而不被代替。两个新的清单是：

永久清单：A (0)，C (2)

试验清单：B (5)，D (3)，E (6)

4. 在试验清单中的所有节点，节点D有最短累计代价，我们将D移到永久清单中，节点D

没有未处理的邻站增加到试验清单中。两个新的清单是：

永久清单：A (0), C (2), D (3)

试验清单：B (5), E (6)

5. 在试验清单中的所有节点，节点B有最短累计代价，我们将B移到永久清单中。我们需要增加B的邻站的所有未处理的节点到试验清单中（这个节点正是E）。但是，E (6) 已在该清单中并具有最小的累计代价。作为B的邻站E，它的累计代价是8，所以我们仍保留节点E (6) 在试验清单中。两个新的清单是：

永久清单：A (0), B (5), C (2), D (3)

试验清单：E (6)

6. 在试验清单中的所有节点，节点E有最短累计代价，我们将E移到永久清单中，节点E没有邻站。现在试验清单为空，我们停止处理。最短路径树已构成，最后的清单是：

永久清单：A (0), B (5), C (2), D (3), E (6) 试验清单：空

从最短路径树计算路由表 每个节点用最短路径协议构造它的路由表，路由表说明了从根到每个节点的代价。表22.2表示了节点A的路由表。

将表22.2与图22.14中节点A的路由表相比较，对于节点A，距离向量路由选择与链路状态路由选择的最终结果相同。

OSPF

开放最短路径优先或OSPF协议 (Open Shortest Path First protocol) 是基于链路状态路由选择的一个域内路由选择协议。它的域也是一个自治系统。

区域 为了有效地和及时地处理路由选择，OSPF将自治系统划分为一些区域。一个区域 (area) 是包含在自治系统中的一些网络、主机和路由器的集合。自治系统又可划分为多个不同的区域，在一个区域里所有网络必须是互相连接的。

在一个区域内的路由器都使用洪泛法传送路由选择信息。在一个区域的边界，称为**区域边界路由器** (area border router) 的一些特殊的路由器，将有关本区域的信息概括起来发送给其他区域。在自治系统中有一个特殊区域称为主干；在自治系统中的所有区域必须连接到主干上。换言之，主干相当于主区域，而且其他区域相当于从区域。但是，这并不表示在各区域内的路由器不能相互连接。在主干中的路由器称为主干路由器 (backbone router)。注意：一个主干路由器也可以同时是一个区域的边界路由器。

如果由于某些问题，在主干和区域之间的连通性被破坏了，那么网络管理员就必须在路由器之间建立一条虚链路 (virtual link)，以保持作为主区域的主干的各功能的连续性。

每一个区域有一个区域标识。主干的区域标识是0。图22.24显示了一个自治系统及其其他的区域。

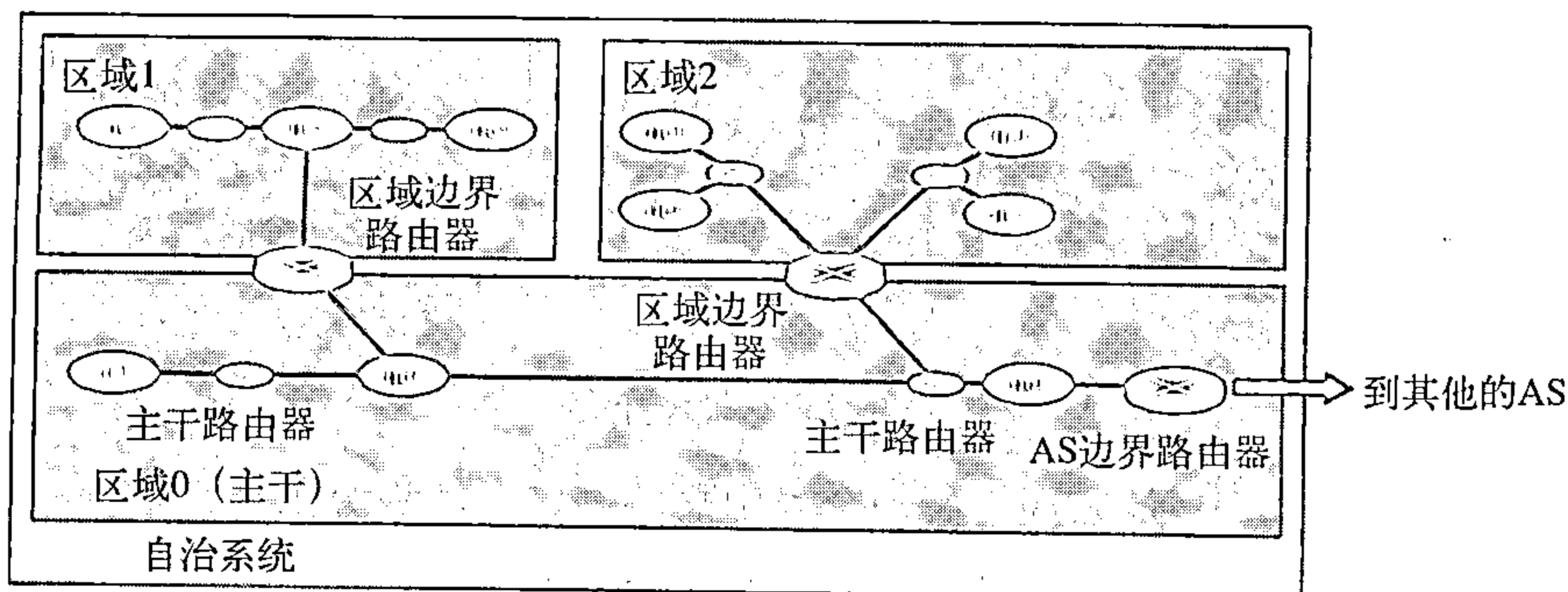


图22.24 自治系统中的区域

度量 OSPF协议允许网络管理员给每一条路由指定一个代价，称为度量 (metric)。度量可以基于服务类型 (最小延迟、最大吞吐量等)。事实上，一个路由器可以有多个路由表，而每一个路由表基于不同的服务类型。

链路类型 在OSPF术语中，一个连接称为链路。已定义了4种类型的链路：点对点链路、过渡链路、残桩链路和虚链路 (见图22.25)。

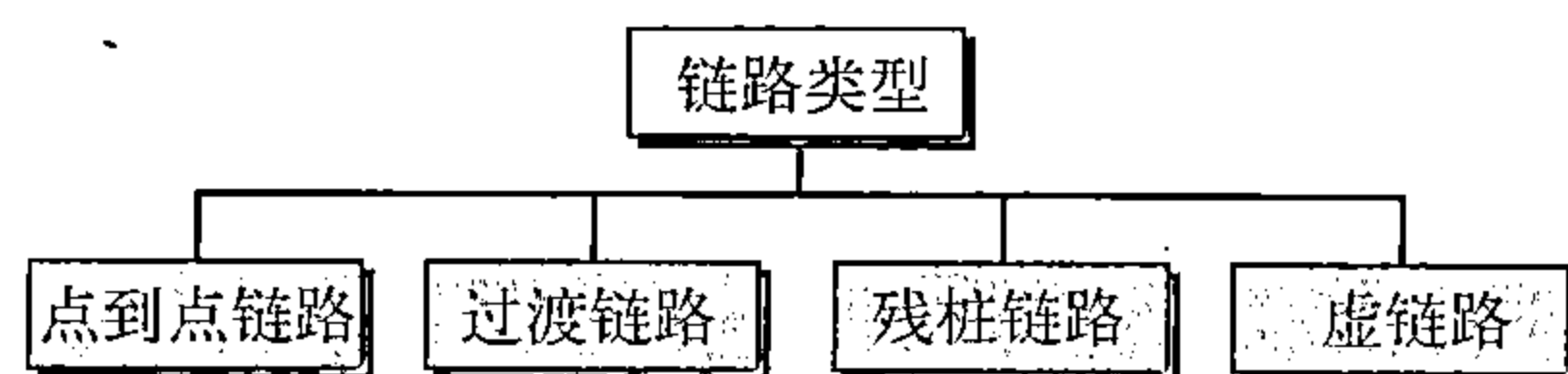


图22.25 链路的类型

点到点链路 (point-to-point link) 连接两个路由器，而中间没有任何其他的主机或路由器。换言之，该链路 (网络) 的目的就是为了连接两个路由器。这种链路类型的例子就是两个路由器用一条电话线 (或一条T线) 连接起来。没有必要给这种类型的链路指定一个网络地址。用图表示时，路由器用节点表示，而链路用一条连接两个节点的双向边来表示。度量表示在两端，各表示每一个方向的度量，通常它们是一样的。即每一个路由器只有一个邻站在链路的另一端 (见图22.26)。

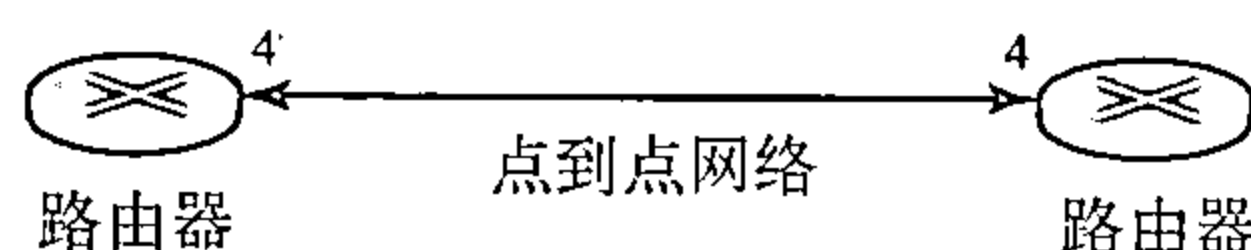


图22.26 点到点链路

过渡链路 (transient link) 是一种连接多个路由器的网络。数据可以从任何一个路由器进入网络，并从任何一个路由器离开网络。所有局域网和某些具有两个或更多的路由器的广域网都属于这种类型。在这种情况下，每一个路由器都有好几个邻站。例如，考虑图22.27a中的以太网。路由器B的邻站是路由器A、C、D和E。如果我们要表示这种情况下的邻站关系，就使用图22.27b。

这种链路既低效又不实际，低效是因为每一个路由器需要通知其他4个邻站的路由器，总共有20个通知；不实际是因为在每一对路由器之间没有一个单独的网络 (链路)，只有一个网络用做所有这5个路由器之间的交叉路口。

为了表示出每一个路由器是通过一个单独网络连接到其他的每一个路由器，我们将这个网络用一个节点来表示。但是，网络并不是一个机器，它不能像路由器那样工作。因此，在网络中某一个路由器就要负起这个责任。这个路由器有双重作用，它是一个真的路由器，又是一个指定路由器，用图22.27c的拓扑结构可以表示过渡网络的连接。

一方面现在每一个路由器只有一个邻站，即指定路由器 (网络)，另一方面这个指定路由器 (即网络) 有5个邻站。我们可看到，向邻站通知的数目由20减少到10。另外，链路用连接节点的双向边来表示。但是，虽然从每一个节点到指定路由器都有一个度量，但从指定路由器到其他任何节点都没有度量，原因就是指定路由器代表了网络，我们不能为它计算两次。当一个分组进入网络时，我们指定一个代价；当分组离开网络到其他路由器时，就没有代价。

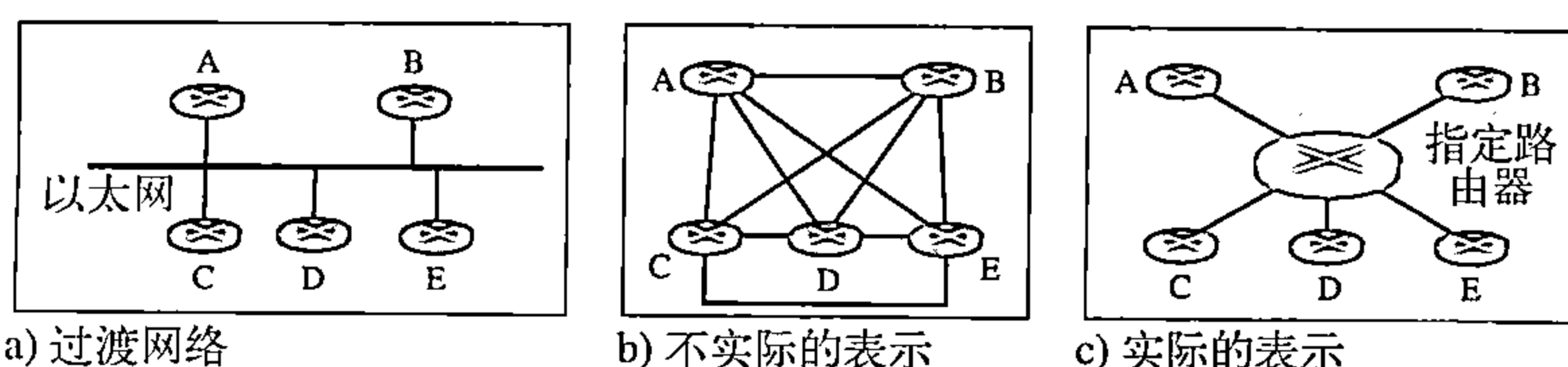


图22.27 过渡链路

残桩链路 (stub link) 是只连接到一个路由器的网络。数据分组通过这个单一的路由器进入网络，而离开网络也是通过这个路由器。这是过渡网络的一个特例。对于这种情况，可以将路由器表示为一个节点，而用指定路由器表示这个网络。但是，链路是单向的，即从路由

器到网络（见图22.28）。

当两个路由器之间的链路断开时，网络管理员就在它们之间使用一条更长的路径，它可能经过好几个路由器，创建一条虚链路（virtual link）。

图形表示 现在让我们考虑一个自治系统如何用图形表示。图22.29表示了具有7个网络和6个路由器的一个小型自治系统。有两个网络是点到点网络，我们使用记号如N1和N2表示过渡网络和残桩网络。没有必要给点到点网络指定一个标识符。该图也表示OSPF所见到的自治系统的图形。

我们用正方形节点表示路由器，用椭圆节点表示网络（用指定路由器表示的网络）。但是，OSPF将两者都看做节点。注意：我们共有3个残桩网络。

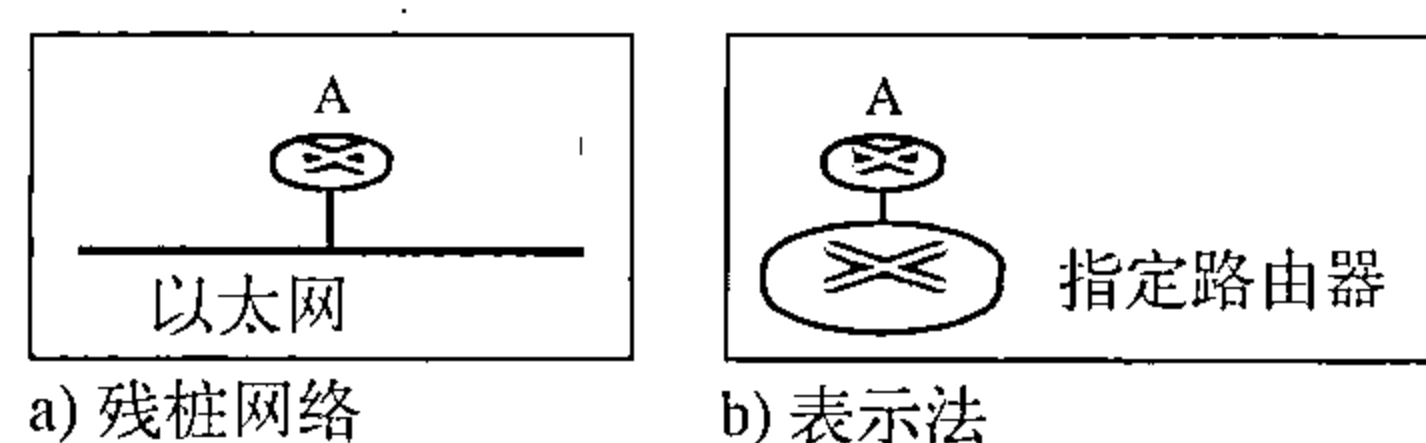
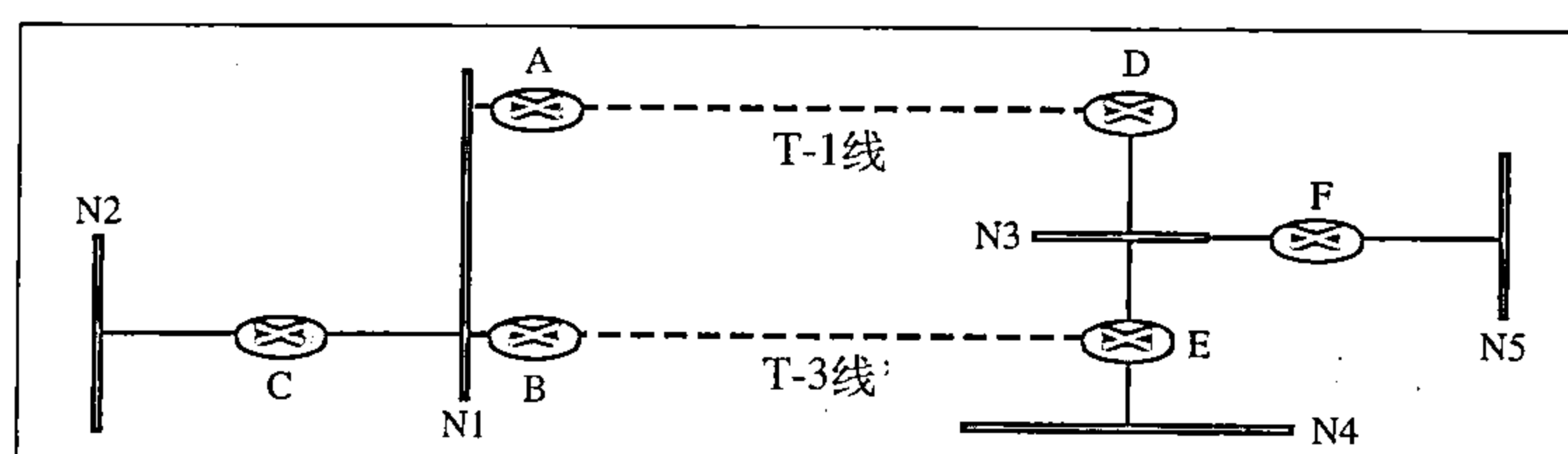
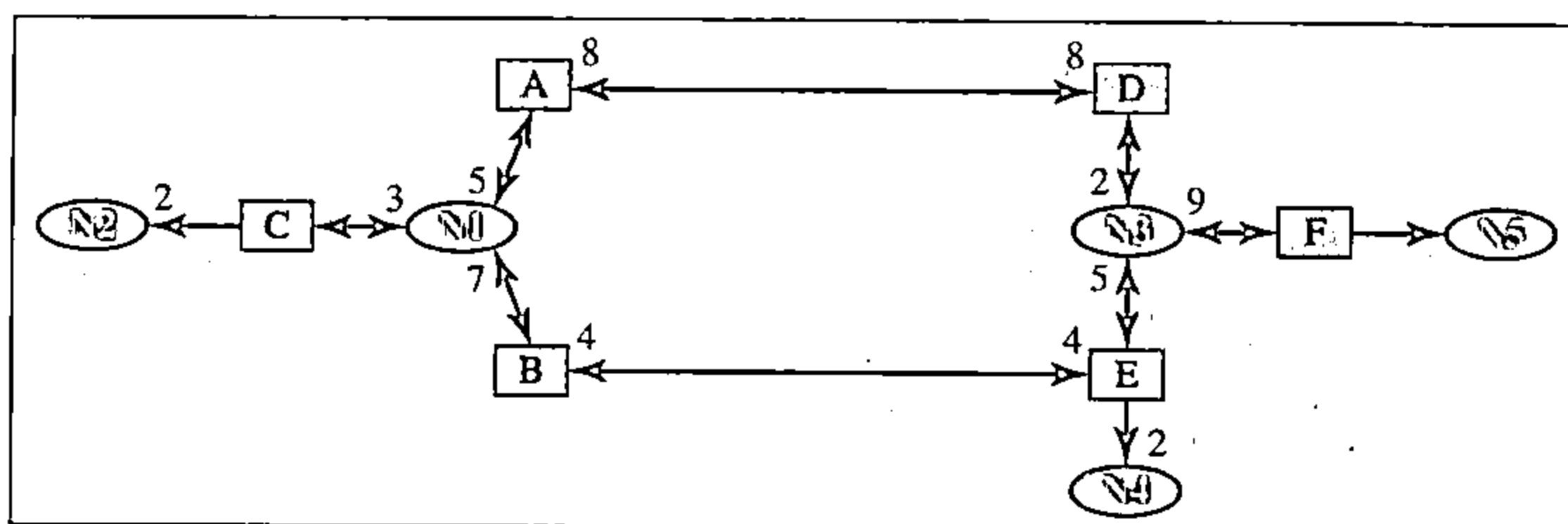


图22.28 残桩链路



a) 自治系统例子



b) 图形表示

图22.29 一个自治系统的例子及其OSPF图形表示

22.3.5 路径向量路由选择

距离向量路由选择和链路状态路由选择都是域内路由选择协议。它们用于一个自治系统内部，而不是自治系统之间。因为可量测性，这两个协议都不适合域间路由选择。当操作区域变大时，这两个路由选择协议变得很难处理。如果在操作区域中存在多个跳数，距离向量路由选择协议是不稳定的。而链路状态路由选择协议需要有巨大的资源来计算路由表，由于洪泛也会产生严重的通信拥塞。因此，需要第三个路由选择协议，我们称它为路径向量路由选择（path vector routing）。

已经证明路径向量路由选择对区域之间的路由选择是有用的，它的原理与距离向量路由选择相似。在路径向量路由选择中，我们假定每个自治系统有一个节点的行为代表了整个自治系统（也可以多个，但对于我们概念性的讨论一个已足够），该节点称为代言节点（speaker node）。

一个自治系统的代言节点生成一个路由表并通知相邻自治系统中的代言节点。除了每个自治系统仅有一个代言节点可以彼此通信外，它与距离向量路由选择的思路相同。但通知的内容是不同的，一个代言节点通知是在它的自治系统或其他自治系统中的路径，而不是节点跳数的度量。

初始化

在开始时, 每个代言节点仅能知道它的自治系统内部节点的可达性。图22.30表示了4个自治系统组成的互联网络中每个代言节点的初始路由表。

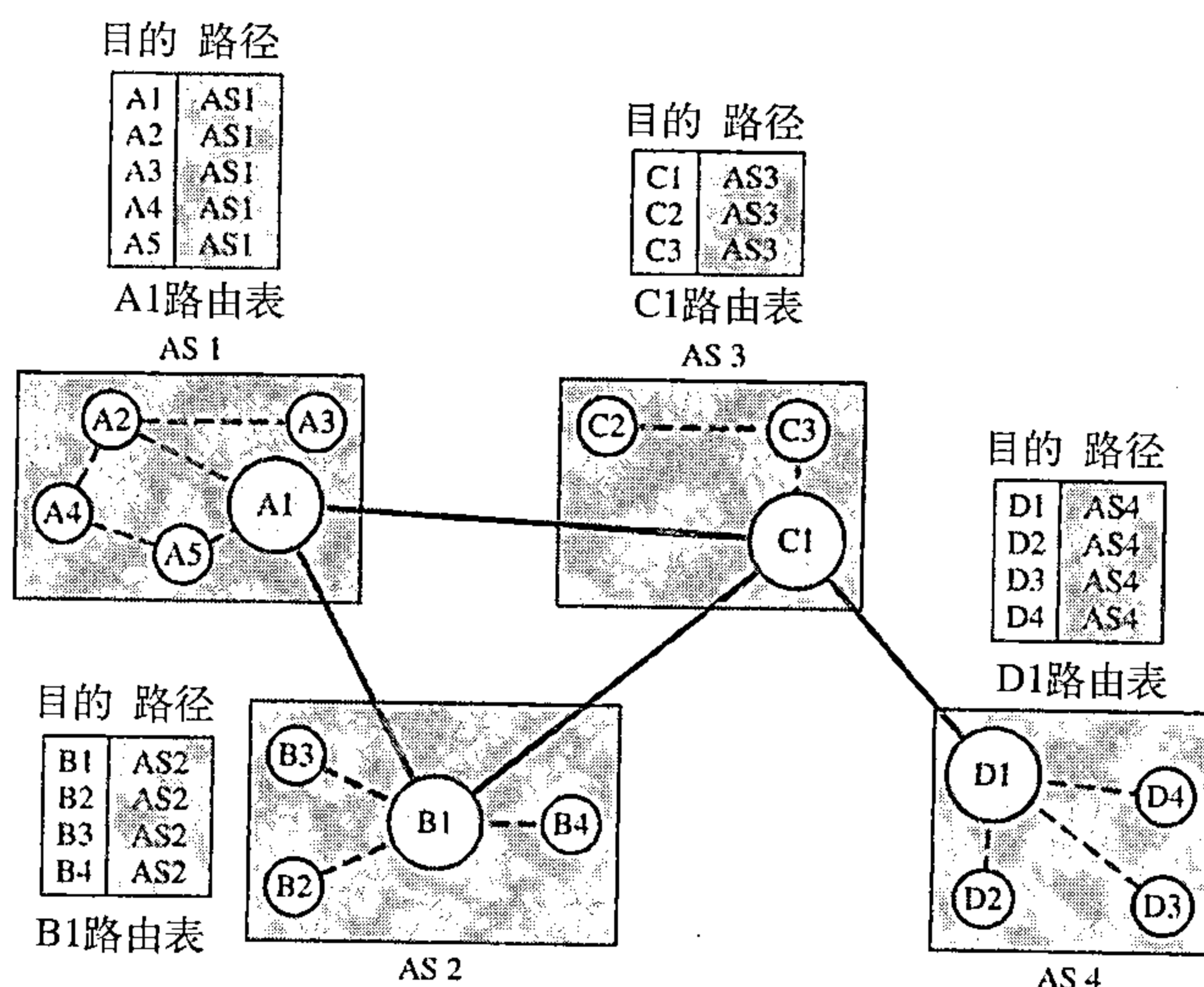


图22.30 路径向量路由选择的初始路由表

节点A1是AS1的代言节点, 节点B1是AS2的代言节点, 节点C1是AS3的代言节点, 节点D1是AS4的代言节点。节点A1生成的初始路由表说明了A1到A5是在本地的AS1中, 通过它可以到达。节点B1生成初始路由表通知B1到B4在本地的AS2中, 它可以经过B1到达等。

共享 正如距离向量路由选择那样, 在路径向量路由选择中, 自治系统的一个代言节点与它邻站的代言节点共享它的路由表。在图22.30中, 节点A1与B1及C1共享它的路由表, 节点C1与节点D1及B1共享它的路由表, 节点B1与节点C1及节点A1共享它的路由表, 以及节点D1与C1共享D它的路由表。

更新 当一个代言节点从一个邻站接收到一个两列的路由表时, 它更新它自己的路由表, 更新内容包括增加不在表中的节点以及其自治系统与发送方的自治系统之间的路径。此后, 每一个代言节点都有一个如何到达其他自治系统各个节点的路由表。图22.31表示系统稳定后, 每一个代言节点的路由表。

按照图22.31, 如果路由器R1接收一个到节点A3的分组, 它知道这条路径是在AS1中 (分组在本自治系统内)。但如果它接收一个到D1的分组, 它知道这分组应该从AS1到AS2然后再到AS3。路由表完全表示了该路径。另外, 如果AS4中的节点D1接收一个到节点A2的分组, 它知道应该经过AS4, AS3和AS1。

目的	路径	目的	路径	目的	路径	目的	路径
A1	AS1	A1	AS2-AS1	A1	AS3-AS1	A1	AS4-AS3-AS1
...	...	...	...	...	...	...	...
A5	AS1	A5	AS2-AS1	A5	AS3-AS1	A5	AS4-AS3-AS1
B1	AS1-AS2	B1	AS2	B1	AS3-AS2	B1	AS4-AS3-AS2
...	...	...	...	...	...	...	...
B4	AS1-AS2	B4	AS2	B4	AS3-AS2	B4	AS4-AS3-AS2
C1	AS1-AS3	C1	AS2-AS3	C1	AS3	C1	AS4-AS3
...	...	...	...	...	...	...	...
C3	AS1-AS3	C3	AS2-AS3	C3	AS3	C3	AS4-AS3
D1	AS1-AS2-AS4	D1	AS2-AS3-AS4	D1	AS3-AS4	D1	AS4
...	...	...	...	...	...	...	...
D4	AS1-AS2-AS4	D4	AS2-AS3-AS4	D4	AS3-AS4	D4	AS4

图22.31 三个独立系统的稳定表

- **预防回路。**在路径向量路由选择中可以避免距离向量路由选择的不稳定性和产生回路。当路由器接收到报文时，它要检查它所在的自治系统是否在到目的网络的路由表中。如果是的话，就会产生回路，这个报文就被忽略。
- **策略路由选择。**通过路径向量路由选择可以很易地实现策略路由选择。当路由器接收到报文时，它就检查其路径。如果在路径中所列出的某个自治系统不符合其策略，它就忽略这条路径和目的端。它不用这条路径更新其路由表，也不将这个报文发送给它的邻站节点。
- **优化路径。**什么是在路径向量选择中最佳的路径？我们在自治系统中寻找一条到达目端的最佳的路径。因为包含路径的每个自治系统中，它们有不同度量准则。某个自治系统内部使用RIP，它是以跳数为度量。而另一个自治系统使用OSPF，它是以最小延迟为度量。所以在这个路由中不可能明确地含有这些度量。最佳路径是符合组织机构标准的路径。在图22.30中，每个自治系统到目的端可能有多条路径。例如，AS4到AS1可以是AS4-AS3-AS2-AS1，或者AS4-AS3-AS1。对于这些路径，我们选取经过自治系统数目最小的一条。但也不总是这种情况，还有其他准则，如保密、安全和可靠性。

BGP

边界网关协议（BGP）是使用路径向量路由选择的域间路由选择协议。它是1989年问世，现已有4个版本。

自治系统类型 正如以前我们说过，因特网被划分成多层次的域，称为自治系统。例如，一个大型的企业有它自己的网络并有完整的控制系统，它是一个自治系统。为本地客户提供服务的本地ISP也是一个自治系统。自治系统可分三大类：残桩的、多接口的和转送的。

- **残桩自治系统。**它与其他自治系统只有一个连接。在该自治系统中，域间数据流量起始或终止于该自治系统。即其源IP地址或目的IP地址所指定的主机位于该自治系统中。但是，数据流量不能通过残桩自治系统。残桩自治系统或者是一个源端或者是一个接收器。好的例子是小型企业或小的本地ISP。
- **多接口自治系统。**它与其他自治系统有多个连接。但它仍旧还是数据流量的源端或接收器。它能接收来自多个自治系统的数据流量，也能发送数据流量到多个自治系统，但没有过渡的数据流量。它不允许来自一个自治系统的数据经它转发到另一个自治系统。一个多接口自治系统的例子是一个连接到多个区域或国家自治系统的大型企业，不允许转送数据。
- **转送自治系统。**它是一个多接口自治系统，但它允许过渡数据流量。过渡自治系统的例子是国家或国际ISP（因特网主干网）。

路径属性 在前面的例子中，我们讨论到目的网络的路径。这个路径可以用自治系统的列表来表示。但是，实际上是用属性的列表来表示的。每一个属性给出关于路径的一些信息。当使用它的策略时，属性列表可帮助接收信息的路由器做出更好的决定。

属性可分为两大类：熟知的和可选的。熟知属性（well-known attribute）是每一个BGP路由器必须知道的，而可选属性（optional attribute）则不需要被每一个路由器都知道。

熟知属性本身又可划分为两类：强制的和自选的。熟知强制属性是在一条路由的描述中必须出现的属性，熟知自选属性是每一个路由器必须知道的，但不一定需要包括在每一个更新报文中。一个熟知强制属性是ORIGIN，它定义路由选择信息的源端（RIP、OSPF等等）；另一个熟知强制属性是AS_PATH，它定义自治系统的列表，表中记录了通过哪个自治系统可以到达目的端。还有一个熟知强制属性是NEXT-HOP，它定义分组必须发送到的下一个路由器。

可选属性也可以划分为两类：传递的和非传递的。可选传递属性是没有实现这个属性的路由器必须传递给下一个路由器的属性，可选非传递属性是如果接收路由器还没有实现的这个属性，则必须丢弃的属性。

BGP会话 使用BGP的两个路由器之间的路由信息的交换产生一次会话。会话是两个BGP系统仅为交换路由选择信息而建立的一次连接。为了可靠性，BGP使用TCP作为其传输层协议。换言之，作为一个应用程序，BGP级的会话是TCP级的一条连接。但是，对于BGP和其他应用程序所做的TCP连接略有不同，对于BGP建立的TCP连接可持续一段较长的时间直到某一不寻常的事件发生。因此，BGP会话有时称为半永久连接。

外部与内部BGP 再明确些，有两种类型的BGP会话：外部BGP（E-BGP）会话和内部BGP（I-BGP）会话。E-BGP会话用于属于两个不同自治系统的两个代言节点之间交换信息，而I-BGP会话用于一个自治系统内部的两个路由器之间交换路由信息。图22.32表示了这一思想。

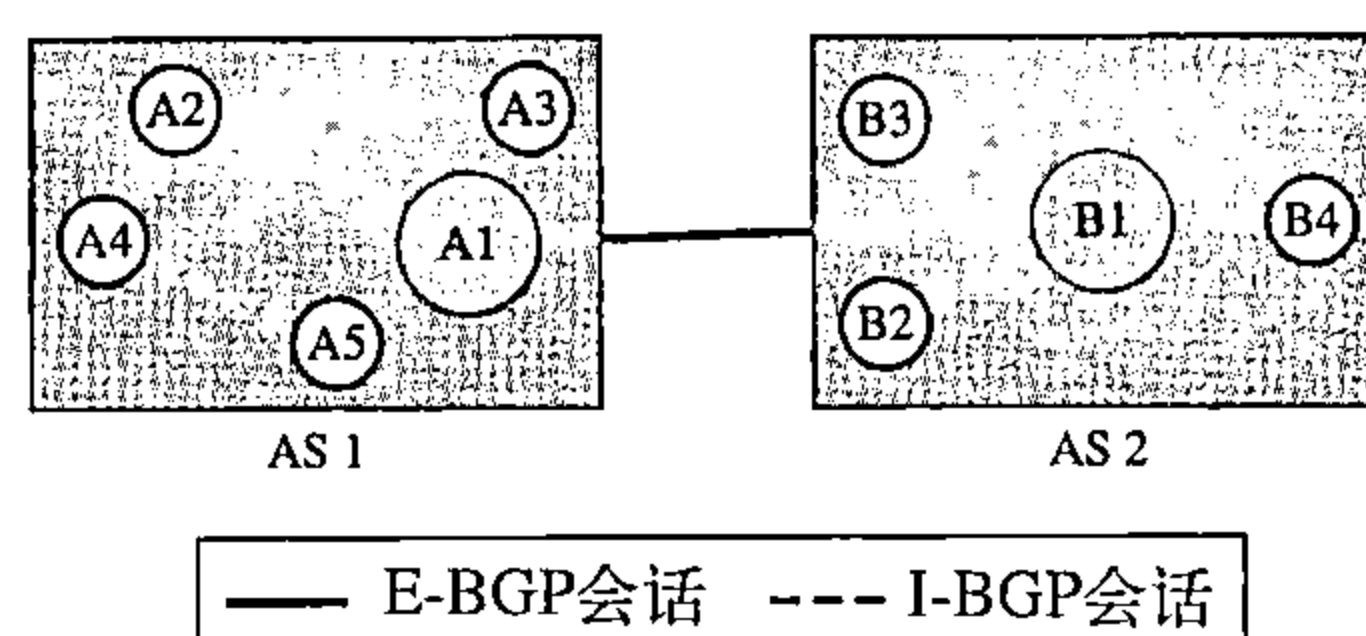


图22.32 内部和外部BGP会话

AS1与AS2之间建立的会话是E-BGP会话，两个代言路由器交换它们所知道的因特网中有关网络的信息。但是，这两个路由器需要从自治系统中的其他路由器收集信息，这是由I-BGP会话完成。

22.4 多播路由选择协议

在这一节，我们讨论多播与多播路由选择协议。首先定义术语多播（multicasting），并将它与单播和广播相比较，还简要介绍多播的应用。最后，我们转向讨论多播路由选择以及它的一般思想和相关目的。我们还会讨论当今因特网常用的多播路由选择协议。

22.4.1 单播、多播与广播

一个报文可以单播、多播或广播。当这些术语涉及因特网时，让我们清晰地说明这些术语。

单播

在单播通信中，只有一个源端和一个目的端。源端和目的端的关系是一对一的。在这种类型的通信中，IP数据报中的源地址和目的地址都是分配给主机（更确切地说是主机端口）的单播地址。如图22.33所示，一个单播分组从源端S1开始，经过路由器到达目的端D1。为了简化该图，可将网络视为路由器之间的一条链路。

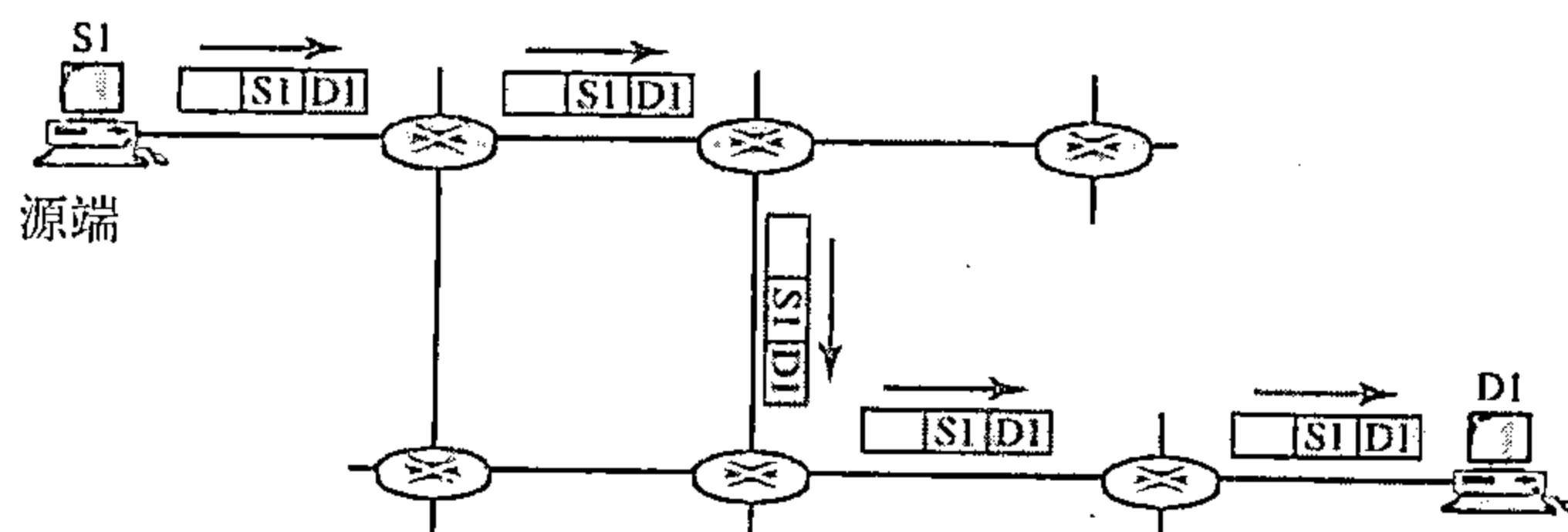


图22.33 单播

注意：在单播（unicast）中，当路由器接收到一个分组时，它仅从路由选择表所定义的端口中的一个（属于最优路径上的一个）将分组转发出去。如果路由器不能在路由选择表中找到目的地址，它就可能将分组丢弃。

在单播中，路由器将接收到的分组仅从其端口中的一个转发出去。

多播

在多播通信中，存在一个源端和一组目的端，其关系是一对多的。在这类通信中，源地址是一个单播地址，而目的地址是一个组地址。组地址定义了一个或多个目的地址，该组地址标识了该组的成员。图22.34表示了多播的思想。

一个多播分组从源端S1发出传向属于组G1的所有目的端。在多播传输中，当路由器接收到一个分组时，它可能通过它的多个端口将其转发出去。

在多播中，路由器可能通过它的多个端口将其所接收的分组转发出去。

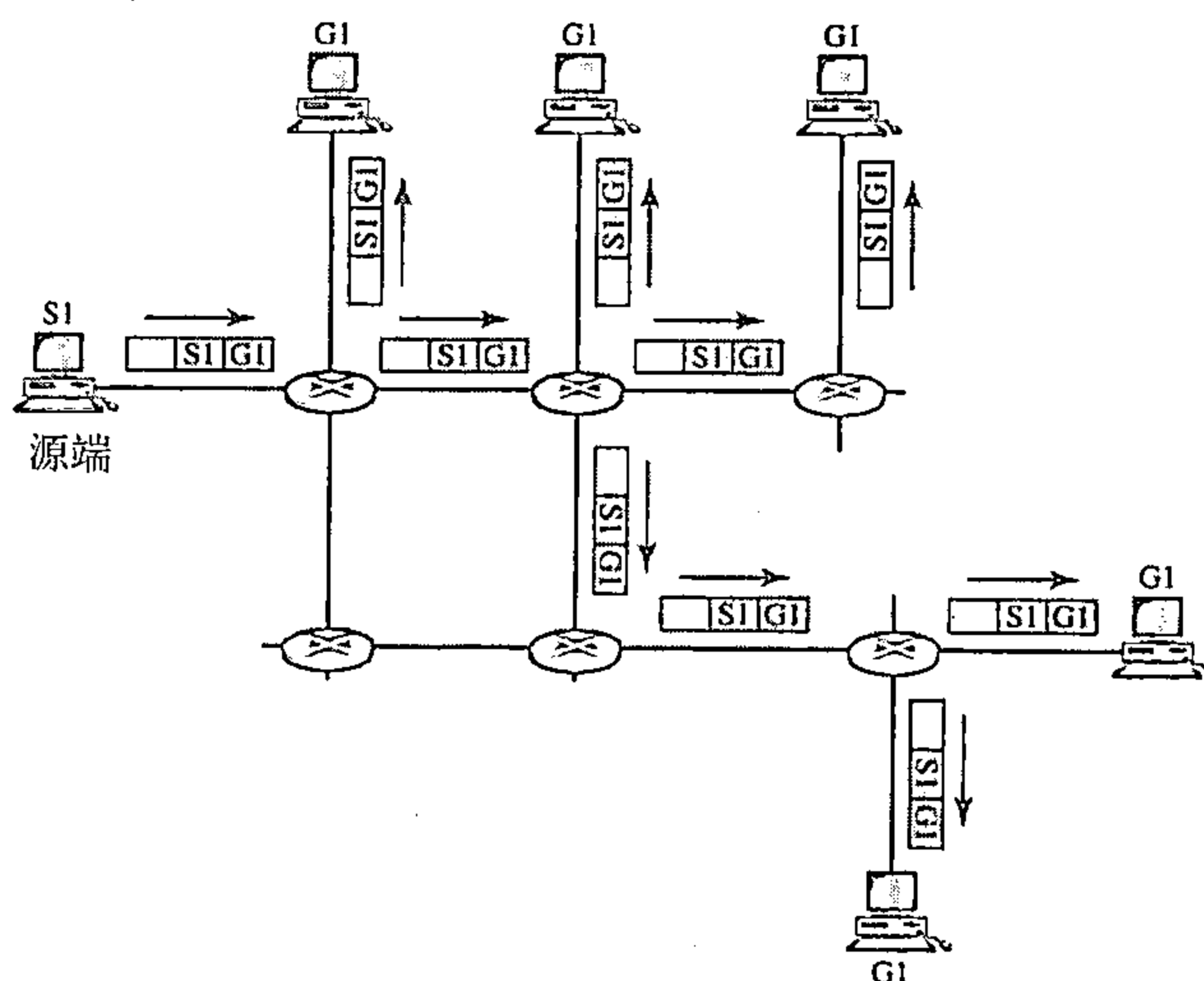


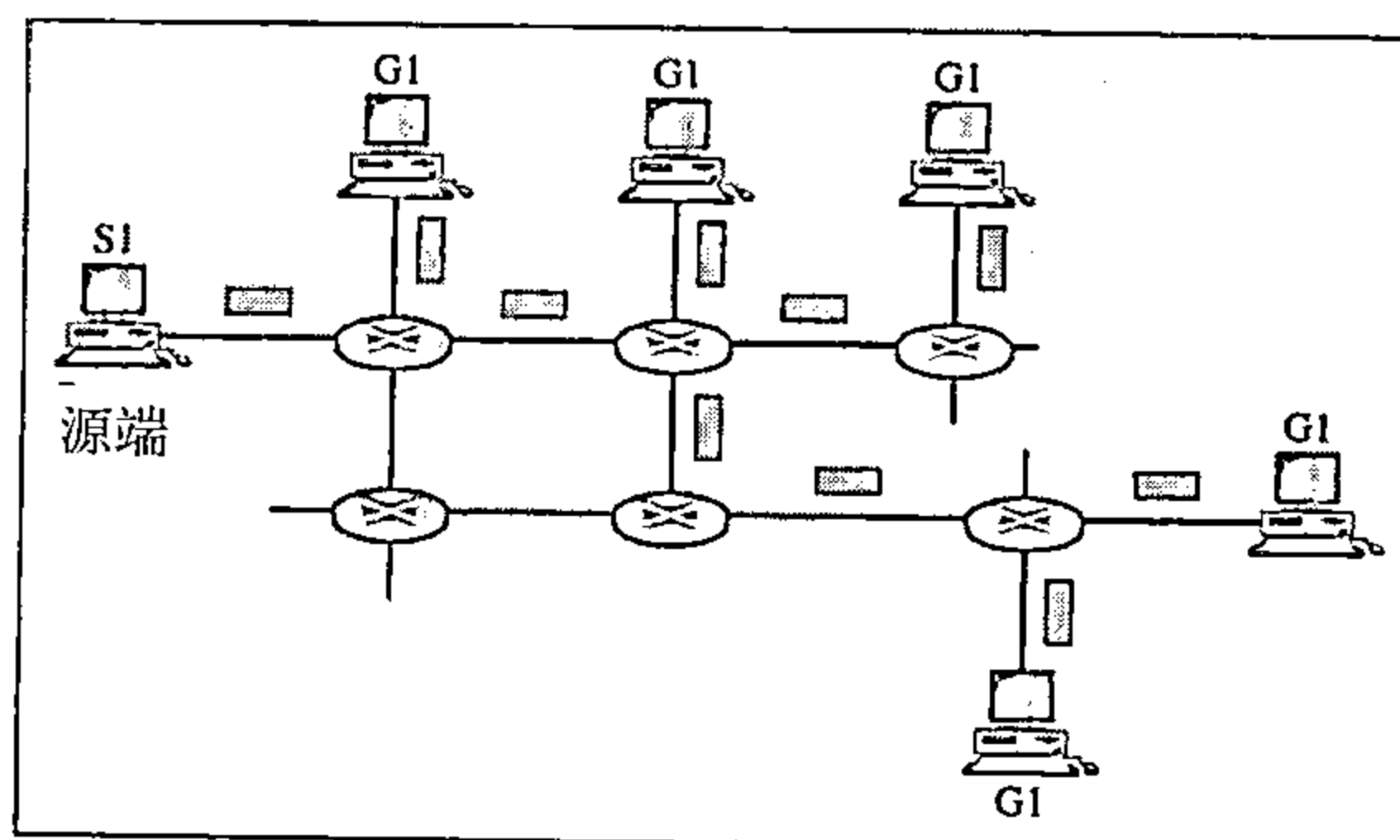
图22.34 多播

广播

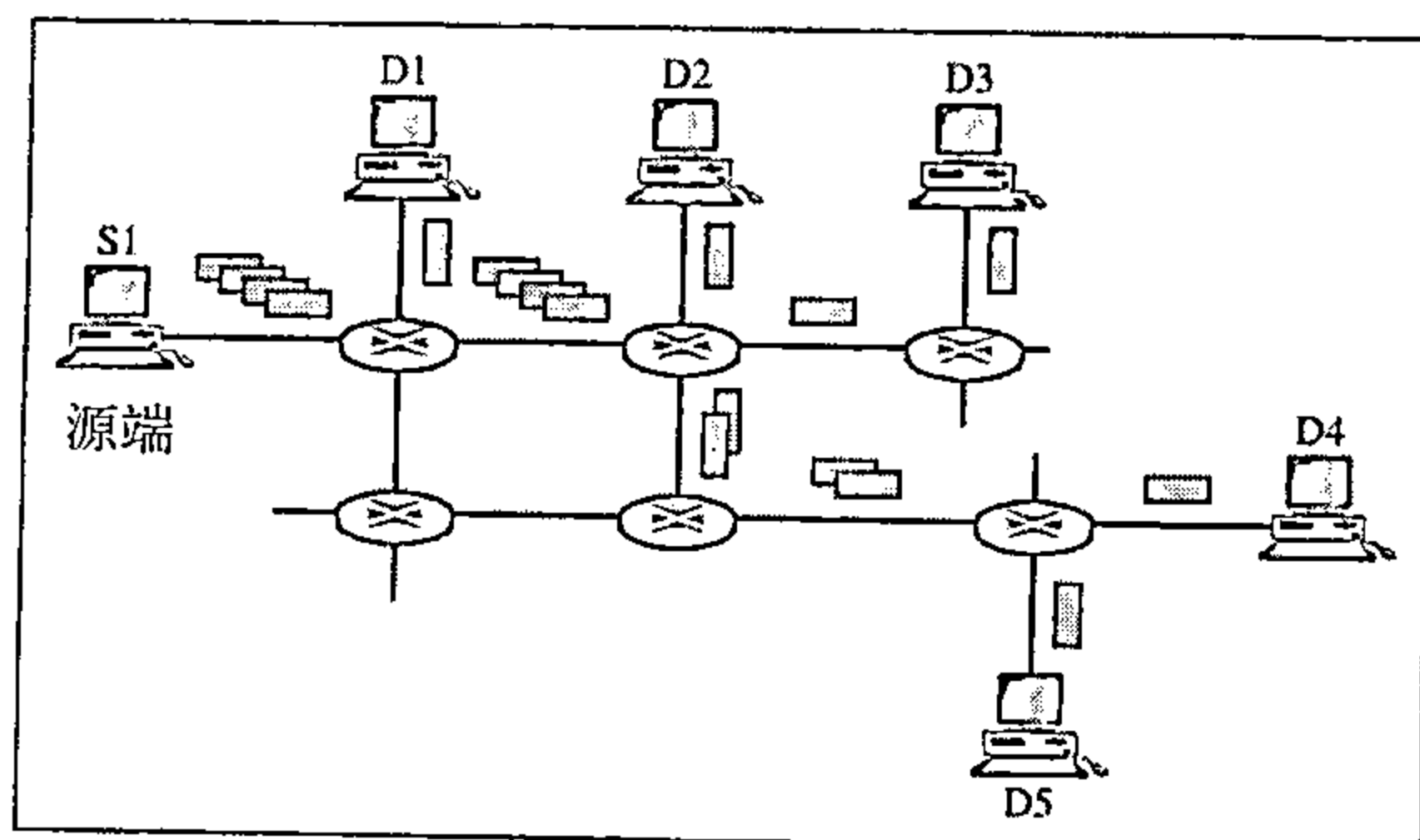
在广播通信中，源端与目的端的关系是一对所有的。源端只有一个，而目的端是其余所有的主机。因特网没有明确地支持广播 (broadcasting)，因为它会产生大量的通信量，并需要高带宽。设想一下，如果一个人想给与因特网相连的其他每个人都发送一条报文，那么在因特网上会产生多大的通信量。

多播与多个单播

本节结束之前，我们必须区别多播与多个单播，这两个概念用图22.35说明。



a) 多播



b) 多个单播

图22.35 多播与多个单播

多播是从源地址以一个单个分组发出，它被路由器复制。每个分组中的目的地址对所有的副本是相同的。注意：在任何两个路由器之间只有分组的一个副本。

在多个单播（multiple unicasting）中，从源端发出多个分组。例如，如果有5个目的地址，源端发送5个分组，每个分组具有不同的单播目的地址。注意：在两个路由器之间可能有多个副本在传送。当一个人发送一封电子邮件给一组人，这就是多个单播。电子邮件软件创建报文的多个复制件，其中每一个具有不同的目的地址并逐一发送它们。这不是多播，而是多个单播。

用单播仿真多播

人们可能会问当多播可用单播仿真时，为什么对多播还要用一个单独的机制。对于这个问题有两点明显的理由：

1. 多播比多个单播效率更高。在图22.35中，我们看到多播比多个单播要求带宽小，在多个单播中，某些链路必须处理多个副本；
2. 在多个单播中，由源端生成的一些分组，这些分组之间有相对的延迟。如果有1 000个目的地址，第一个分组与最后一个分组之间的延迟可能是不可接受的。如果在多播中，因为由源端仅生成一个分组，因此没有延迟。

用单播仿真多播效率不高并可能产生延迟，特别是对于大的组。

22.4.2 应用

当今多播有许多应用，如访问分布式数据库（distributed database）、信息发布、电话会议和远程学习。

访问分布式数据库

当前大多数数据库都是分布式的，即信息通常在生成时存储在多个地方。需要访问数据库的用户不知道信息的地址。用户的请求是向所有数据库的地方多播，而有该信息的地方响应。

信息发布

商业机构时常需要向它们的客户发送信息。如果信息的性质对每个客户都相同，则它可以多播。采用这种方式，一个商业机构可发送一个报文到多个客户。例如，可向购买某个特殊软件包的所有客户发送一个更新软件。

新闻传播

同样的，新闻可通过多播简单地传播。单独一个报文可发送给特殊感兴趣的那些人员。例如，中学篮球冠军巡回赛的统计结果可被送给多份报纸的运动编辑记者。

电话会议

电话会议（Teleconferencing）包含多播，所有出席会议的人都在同一时间接收到相同的信息。为此，可构成临时组或永久组。例如，每个星期一举行例行会议的工程组可编为永久组，而计划休假人员的组可编为临时组。

远程学习

多播使用中一个正在成长的领域是远程学习（distance learning）。某一教授讲的课可被一个特定组的学生接收到。这特别适用于那些不能到大学课堂听课的学生。

22.4.3 多播路由选择

在这一节，我们首先讨论所有多播协议的共同部分——最佳路由选择。然后，我们给出

多播路由选择协议的概况。

优化路由选择：最短路径树

最佳域内路由选择过程最后形成的结果是寻找最短路径树。树的根是源端，而叶子都是可能的目的端。从根到每个目的端的路径是最短路径。但是，在单播中树的数目和树的构成与多播路由选择是不同的。让我们分别加以讨论。

单播路由选择 在单播路由选择中，当路由器接收到一个要转发的分组时，它需要寻找到分组的目的端的最短路径。路由器在路由表中查询那个特定的目的地址，对应于该目的地址的下一跳是最短路径的起始点，路由器知道到达每个目的端的最短路径，这说明路由器有一个最佳地到达所有目的地址的最短路径树。换言之，路由表的每一行是一条最短路径；而整个路由选择表是一个最短路径树。在单播路由选择中，每个路由器需要有唯一的最短路径树转发分组，然而每个路由器有它自己的最短路径树。图22.36显示了这样的情形。

图22.36显示了路由器R1路由表和最短路径树。路由表中的每一行对应于从根到相应网络的一条路径。整个路由表代表了最短路径树。

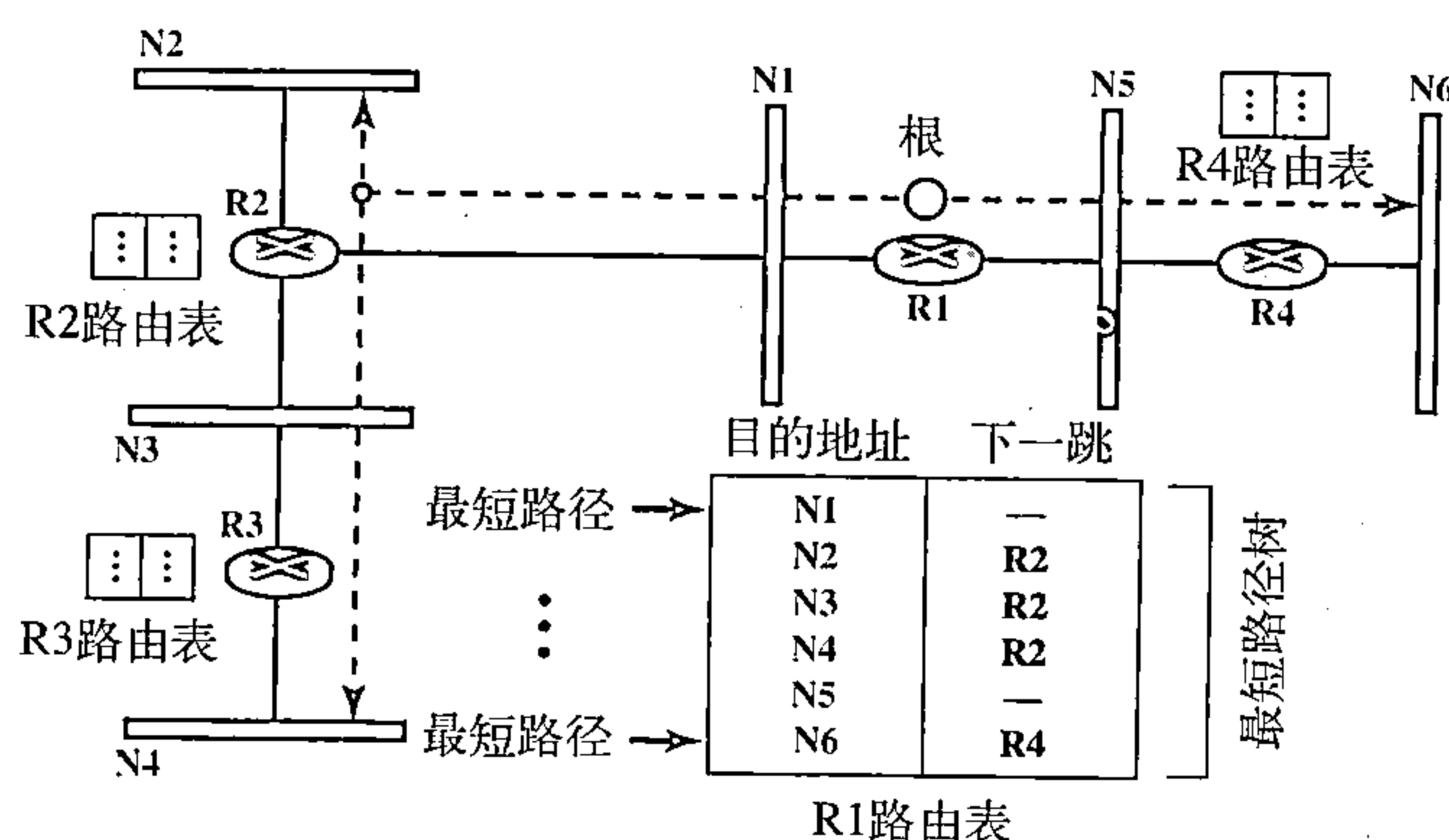


图22.36 单播路由选择中的最短路径树

在单播路由选择中，区域中的每一个路由器都有一张表，该表定义了到可能目的地址的一棵最短路径树。

多播路由选择 当路由器接收到一个多播分组时，情形与接收到一个单播分组时有所不同。一个多播分组可能有多个网络中的目的地址。向组中的成员转发单个分组要求一个最短路径树。如果有 n 个组，那么我们需要 n 个最短路径树。可以想像多播路由选择的复杂性。为了解决这个问题，已使用两种方法：基于源树和组共享树。

- **基于源树。**在基于源树（source-based tree）方法中，每个路由器要求每组都要有一个最短路径树。一个组的最短路径树定义了包含那个组的忠实成员的每个网络的下一跳。在图22.37中，我们假定在区域中只有5个组：G1、G2、G3、G4和G5。此时，G1在4个网络上有忠实成员，G2在3个网络上有忠实成员，G3在2个网络上有忠实成员，G4在2个网络上有忠实成员以及G5在2个网络上有忠实成员。我们已在每个网络上表示了忠实成员所在组的名。图22.37也表示了路由器R1的多播路由表。每个组有一个最短路径树，所以5个组有5个最短路径树。如果路由器R1接收到目的地址为G1的一个分组，那么它需要将分组的一个副本发送给与它有连接的网络，向路由器R2发送一个副本，向路由器R4发送一个副本，这样G1的所有成员都能接收到一个副本。在这种方法中，如果组的个数是 m ，那么每个路由器需要有 m 个最短路径树，每组一个。我们可以想像，如果有

成百或成千的组，路由表将是多么复杂。然而，我们将说明不同的协议将如何处理以减小其复杂性。

在基于源树方法中，对每个组每个路由器都需要有一个最短路径树。

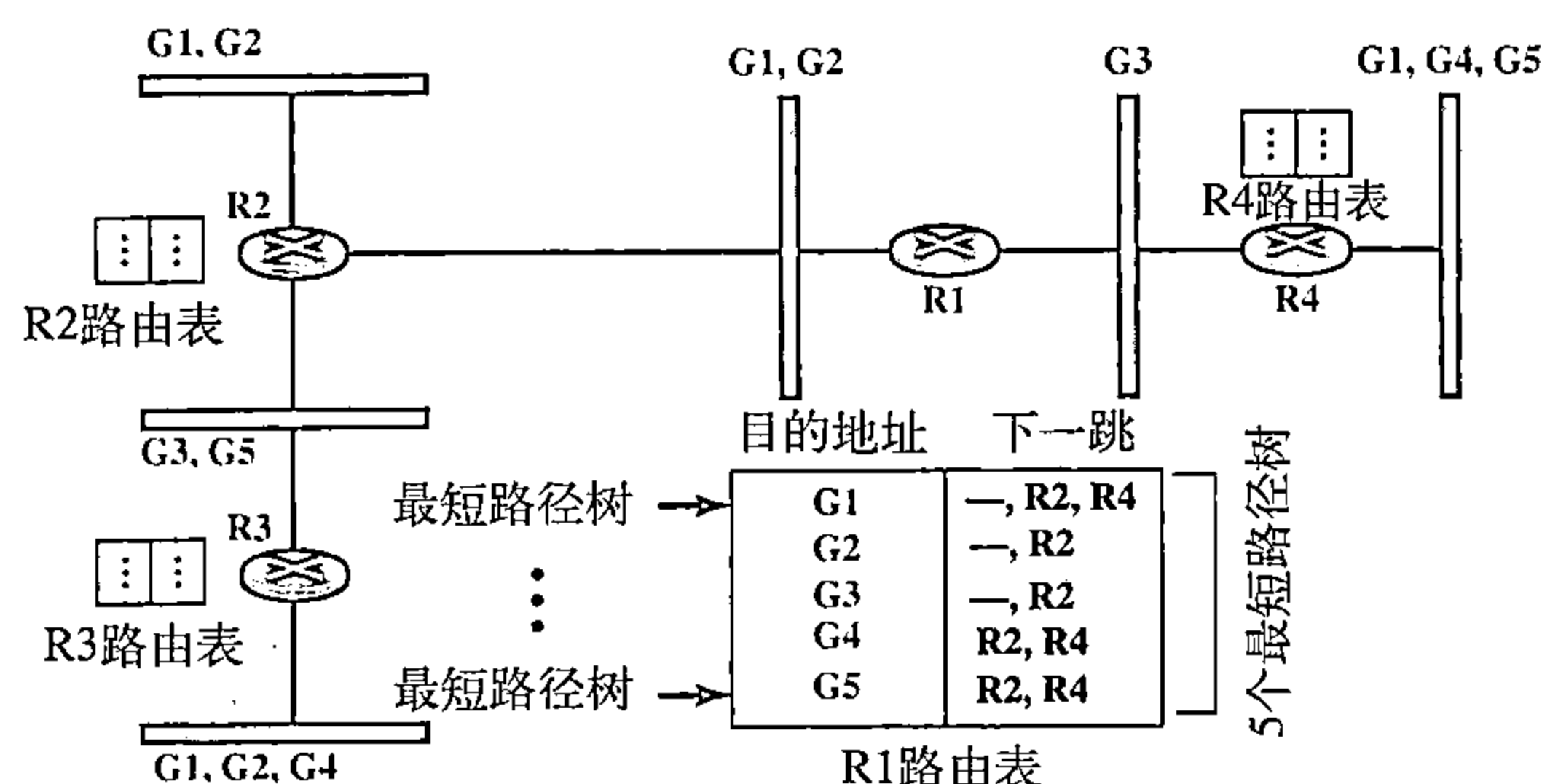


图22.37 基于源树方法

- 组共享树。在组共享树（group-shared tree）方法中。不是每一个路由器都具有 m 个最短路径树，而只是一个指定的路由器称为中央核心或会合路由器（rendezvous router）担任分配多播通信量的责任。中央核心路由器在它的路由表中有 m 条最短路径。区域中其余的路由器都没有。如果一个路由器接收到一个多播分组，它将该分组封装在单播分组中，并发送给核心路由器，核心路由器从它的封装体中重新恢复这个多播分组，并查询它的路由表路由该分组。图22.38表示这个思路。

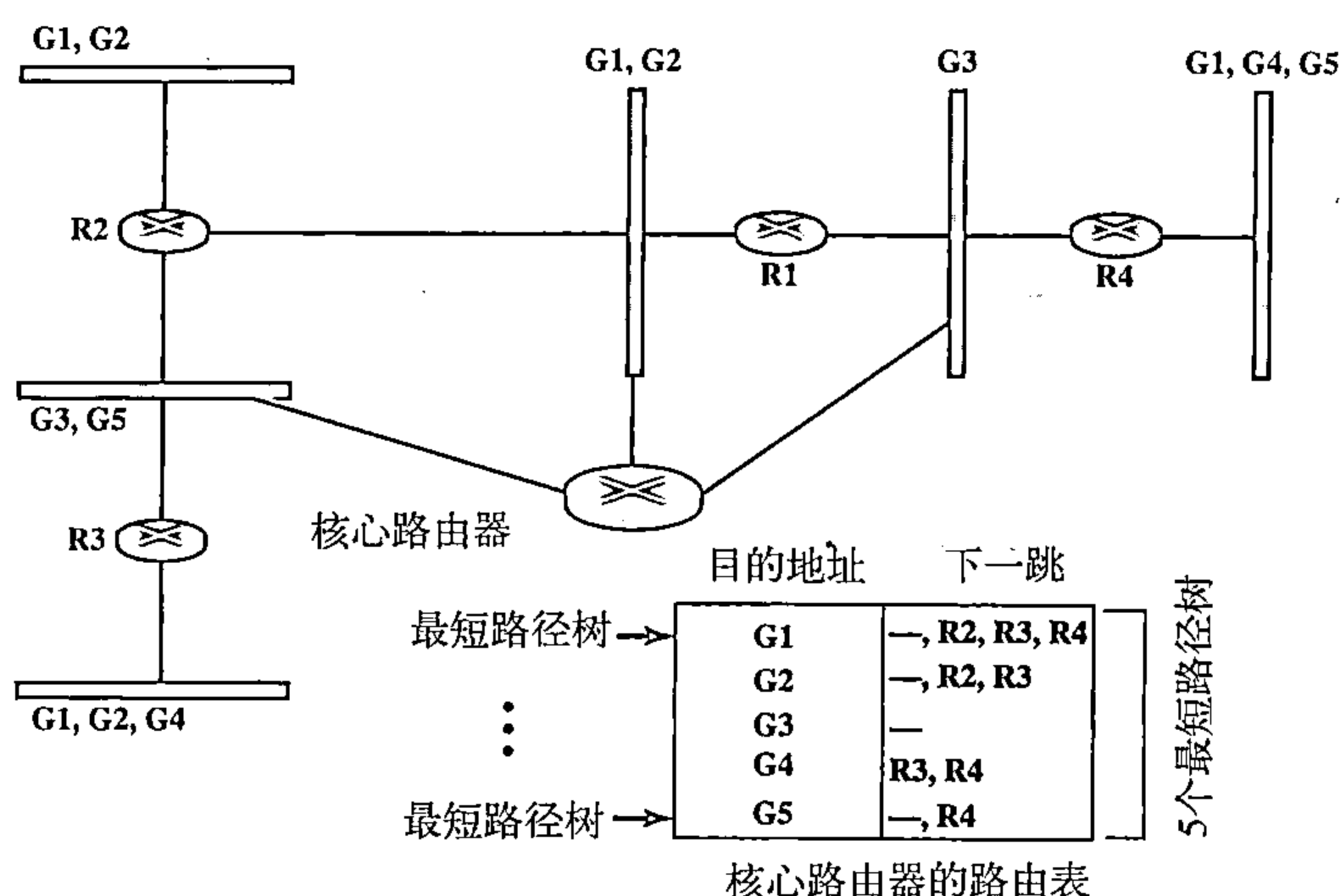


图22.38 组共享树

在组共享树方法中，只有一个核心路由器，它对多播所涉及的每一个组有一个最短路径树。

22.4.4 多播路由选择协议

在最近十几年间，已经出现一些多播路由选择协议。其中有些是单播路由选择协议的扩展，另一些全是新开发的。

在本章的余下部分就讨论这些多播路由选择协议。图22.39显示了这些协议的分类。

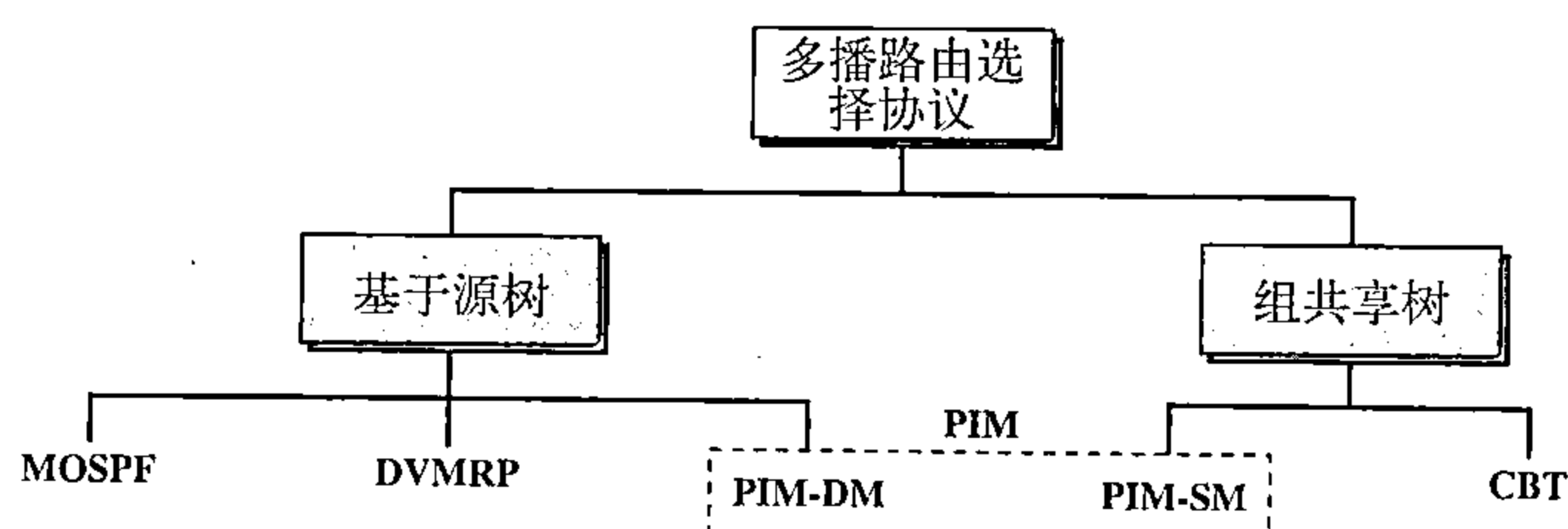


图22.39 常用的多播路由选择协议的分类

多播链路状态路由选择：MOSPF

这一节简要讨论多播链路状态路由选择及其在因特网中的实现，MOSPF。

多播链路状态路由选择 我们在22.3节中讨论了单播链路状态路由选择，我们说每个路由器用Dijkstra算法创建一个最短路径树，路由表是最短路径树的一种转换形式。多播链路状态路由选择是单播路由选择的直接扩充，它使用一个基于源树方法。尽管它完全包含单播路由选择，但多播路由选择的扩充部分是很简单的和直接的。

多播链路状态路由选择使用基于源树方法。

记住在单播路由选择中，每个节点需要通告它的链路状态。对于多播路由选择来说，每个节点要修正状态的解释，一个节点通知链路上有忠实成员的每个组。此时，状态的意思是“在链路上哪些组是活动的”。有关组的信息来自IGMP（见第21章）。每个运行IGMP的路由器请求链路上主机寻找成员状态。

当路由器接收到所有LSP时，它用Dijkstra算法创建 n 个最短路径树，从中创建 n 个拓扑结构（ n 是组的个数）。于是，每个路由器有一个路由表，该表代表与组一样多的最短路径树。这种协议要解决的唯一问题是创建和保存多个最短路径树所需的时间和空间。解决办法是仅当需要时才创建树。当路由器接收到具有多播目的地址的分组时，它运行Dijkstra算法计算那个组的最短路径树。其结果被高速缓存，以防万一到那个目的地址还有其他的分组出现。

MOSPF 多播开放最短路径优先（Multicast Open Shortest Path First, MOSPF）协议是对OSPF协议的一个扩充，它运用多播链路状态路由选择创建基于源树策略的树。该协议增加了一个新的链路状态更新分组，将一个主机的单播地址与组地址或该主机提供的地址关联起来。这个分组被称为组成员关系LSA。用这种方法，能够在树中只包括属于一个特定组的主机（用它们的单播地址）。换句话说，可以生成一个包含属于一个组所有主机的树，但在计算时使用主机的单播地址。为了提高效率，路由器按需求计算最短路径树（当它接收到第一个多播分组时）。此外，能将该树存储在高速缓存中，以备将来相同的源/组对使用。MOSPF是一种数据驱动（data-driven）协议。当MOSPF路由器第一次发现一个具有给定源和组地址的数据报时，该路由器就计算Dijkstra最短路径树。

多播距离向量：DVMRP

在这节中，我们简要讨论多播距离向量路由选择及其在因特网中的实现，DVMRP。

多播距离向量路由选择 单播距离向量路由选择是很简单的，扩充它支持多播路由选择是复杂的。多播路由选择不允许一个路由器向它的邻站发送它的路由表。它使用单播距离向量路由表的信息从头创建一张路由表。

多播距离向量路由选择使用基于源树，但路由器从不真正创建路由表。当路由器接收到多播分组时，它转发该分组好像它正在查询一张路由表一样。我们可以说最短路径树是短暂的，它使用后（转发分组后），该表就消失了。

为了达到这个目的，多播距离向量算法是基于4个决策的一个过程。每个策略建立在前一个策略之上，并改进前一个存在的缺点。我们依次说明并观察每个策略是如何改进的。

- 洪泛。洪泛是第一个策略。一个路由器接收到一个分组，除该接收接口外它甚至不查找目的组地址就向它的所有接口发送该分组。洪泛实现多播目的：具有活动成员的每个网络都接收到该分组，而没有活动成员的网络也接收到该分组。这是广播而不是多播。这就出现另一个问题：它会形成回路。一个分组离开路由器，它可能从另一个接口或同一接口重新返回，再次被转发。有些洪泛协议为了避免回路暂时保存分组副本一段时间，然后就丢弃。下一个策略是逆路径转发，纠正这个缺点。

洪泛策略广播分组，但在系统中形成回路。

- 逆路径转发 (RPF)。逆路径转发是洪泛策略的一个修改版。为了防止回路的形成，只转发一个副本而将其他副本都丢弃。在RPF中，路由器只转发那个已经通过从源端到路由器的最短路径进行传输的副本。为了找到这个副本，RPF使用了单播路由选择表。路由器接收到一个分组，并从该分组中提取出源地址（一个单播地址）。路由器查询它的单播路由表，就好像它要发送一个分组到这个源地址，路由表会告诉路由器下一跳地址。如果到达的多播分组正好来自于路由表中所定义的下一跳地址，那么该分组是沿着从源端到路由器的最短路径传输的，这是因为在单播距离向量路由选择协议中，最短路径的反方向的。如果A到B的路径是最短的路径，那么B到A的路径也是最短的。如果分组是沿着最短路径传输，则路由器转发它，否则丢弃它。

这种策略防止回路形成，因为从源到路由器总是沿着最短路径传输。如果一个分组离开后又返回，显然这不是最短路径。图22.40清楚地说明了这一点。

图22.40表示了部分区域和源端。计算路由器R1、R2和R3的最短路径，用粗线表示。当R1通过接口m1接收到来自源端的一个分组时，它查询它的路由表并找到从R1到源端的最短路径是通过接口m1，转发该分组。但是，如果通过接口m2到达的分组一个副本，则丢弃。因为m2不是R1到源端的最短路径。对R2和R3也一样。

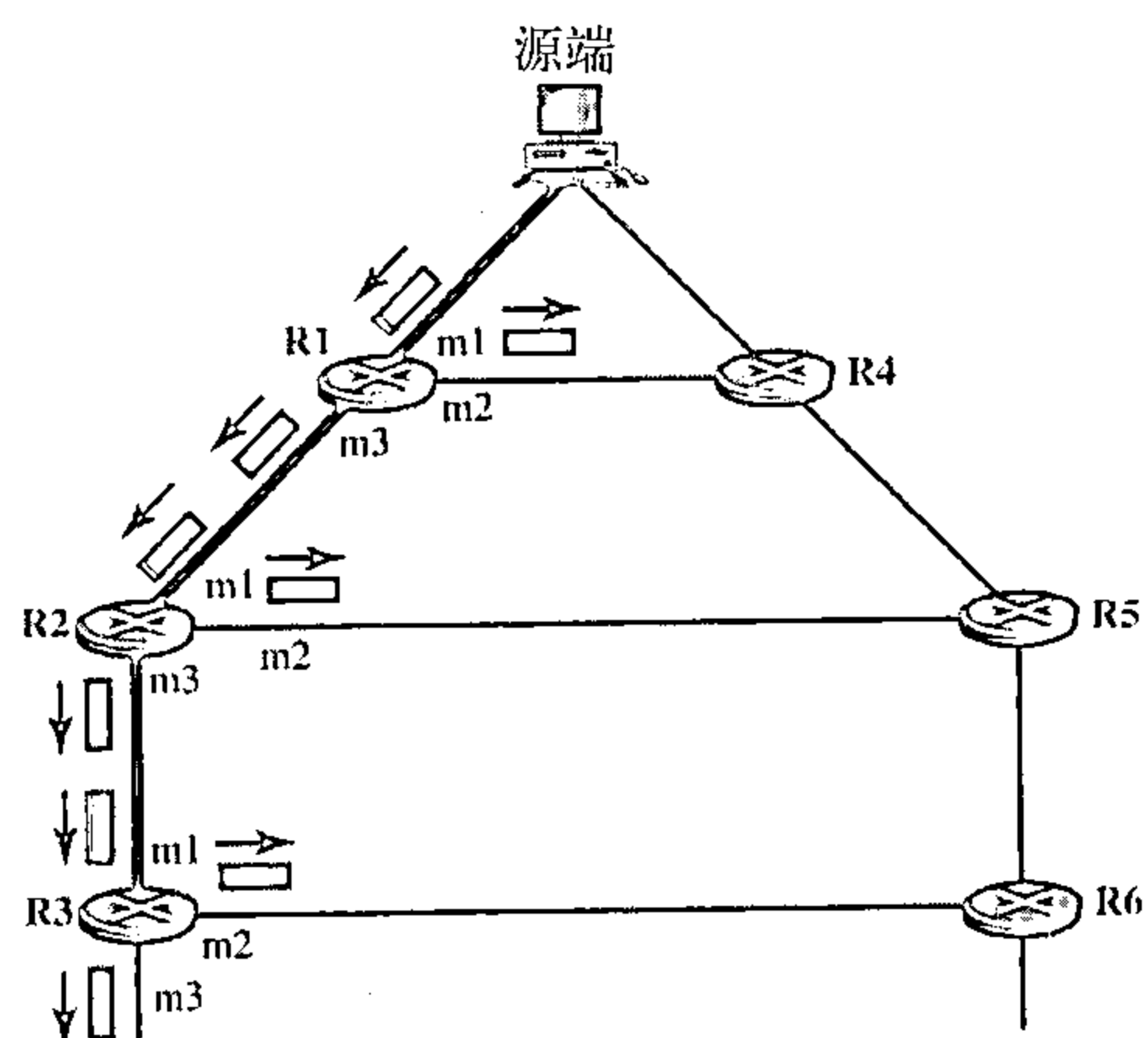


图22.40 逆路径转发 (RPF)

读者可能会问，如果一个分组副本到达R3的m1接口，经过R6、R5、R2传输，然后通过接口m1进入R3将会发生什么？这个接口的确是R3的接口，该分组的副本是否被转发？回答是在这种情形下什么也不会发生，因为当分组从R5到R2传输时，它将被R2丢弃，永远不会到达R3。源端的上行路由器总是丢弃那些不是通过最短路径传输的分组，因此避免了下行路由器的混淆。

RPF消除了洪泛过程的回路。

- 逆路径广播 (RPB)。RPF能确保每个网络接收一个多播分组的副本而不构成回路。然而，它不能保证每个网络只接收一个副本（一个网络可能接收两个或多个副本）。其原因是，RPF并不是基于目的地址（组地址）的，而转发是基于源地址的。为了直观地了解这个问题，让我们考察图22.41。

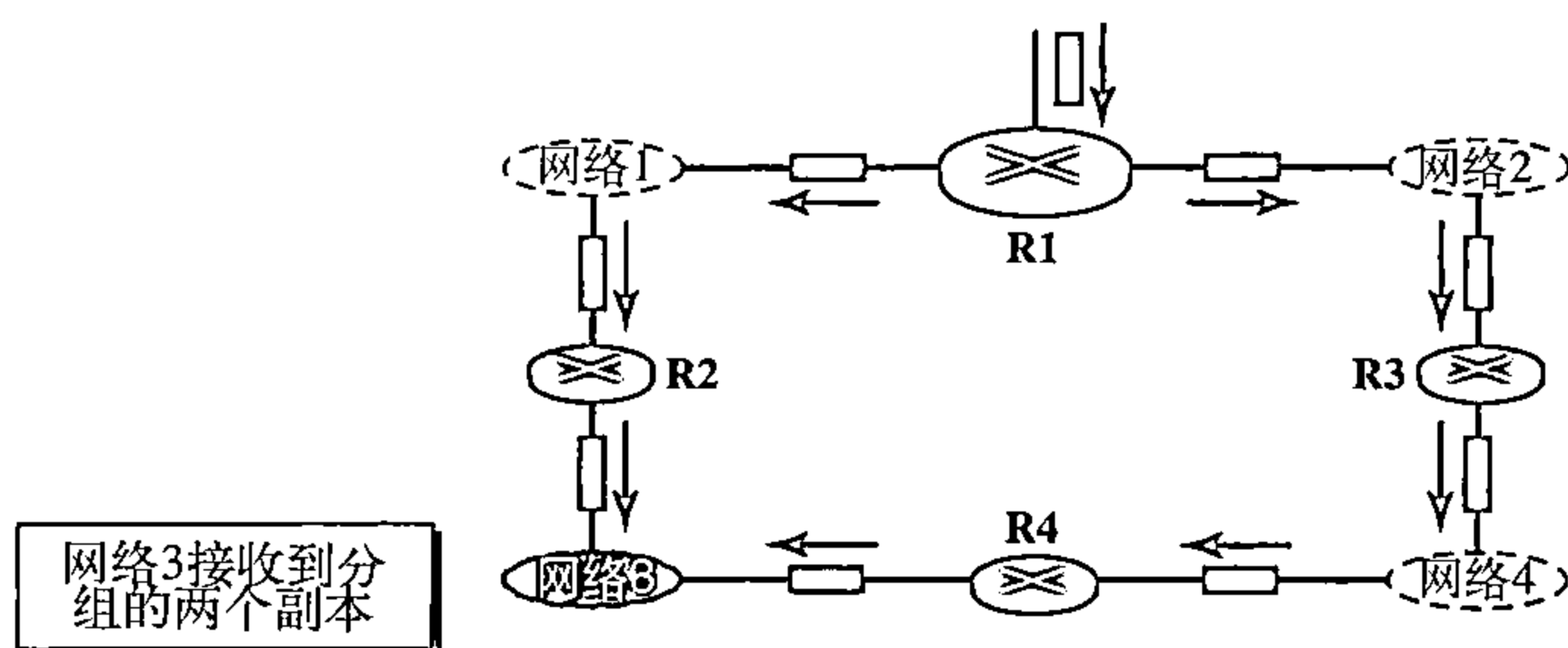


图22.41 RPF的问题

在图22.41中，即使每个路由器从每个接口只发送出分组的一个副本，网络3还是接收到分组的二个副本。有重复的原因是没有生成树，而是用图。网络3有二个父路由器：R2和R4。为了消除重复情况的发生，对每个网络必须只定义一个父路由器，而且必须施加一种限制，即：一个网络只能通过一个指定的父路由器（designated parent router），接收一个来自特定源端的多播分组。

现在，这种策略就很清楚了。对每个源端，路由器只将分组从指定的父路由器的端口发送出去。这种策略称为逆路径广播（RPB）。RPB不仅能确保分组到达每个网络，而且能保证每个网络只接收一个副本。图22.42说明了RPF与RPB之间的差异。

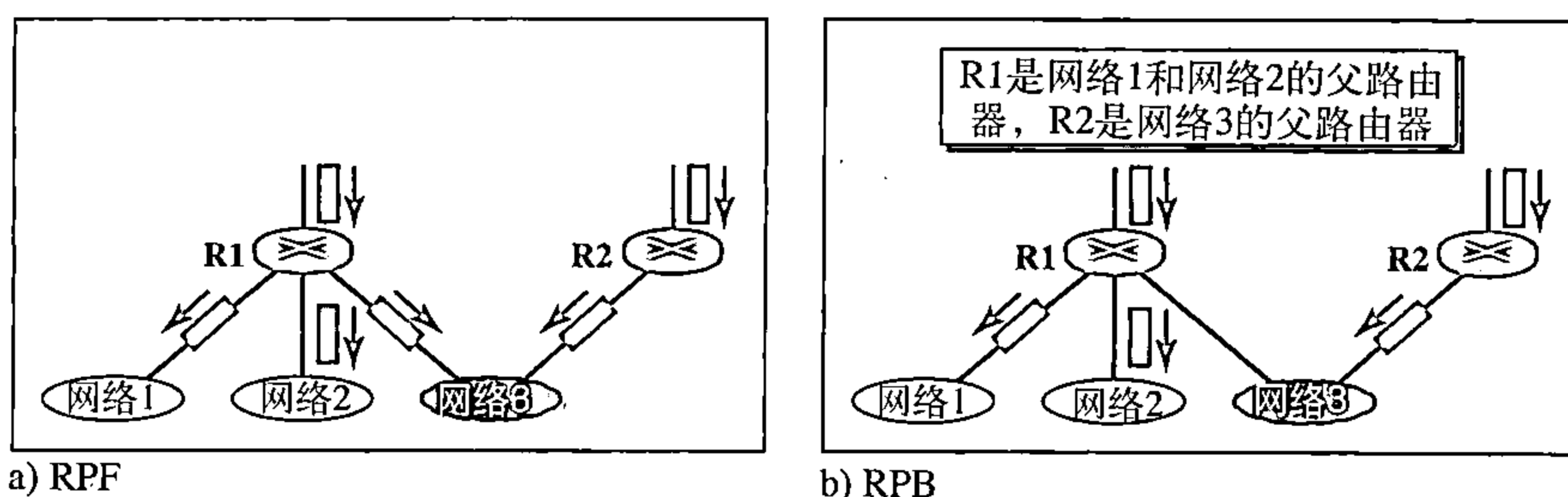


图22.42 RPF与RPB的比较

读者可能会问父路由器是如何确定的。用到源端具有最短路径的路由器作为指定的父路由器。因为路由器周期地向每个其他的路由器发送更新分组（在RIP），它们很容易确定在邻近路由器中哪一个路由器到源端有最短路径（在源端作为目的端解释时）。如果存在多个路由器相等时，选取具有最小IP地址的路由器。

RPB创建从源端到每一个目的端的一个最短路径广播树，它保证每个目的端接收到分组的一个且仅有一个副本。

- 逆路径多播（RPM）。也许读者已经注意到了，RPB并不能对分组进行多播，而只能对它进行广播。这样做是没有效率的。为了使其更有效率，对特定的组，多播分组只能到达那些具有活动成员的网络。该策略称为逆路径多播（reverse path multicasting, RPM）。为了把广播转变成多播，协议使用两种方法：修剪和嫁接。图22.43说明了修剪和嫁接的思想。

每个网络的指定的父路由器负责保存成员关系信息，这通过第21章所讨论的IGMP协议来完成。当连接网络的路由器发现没有参加多播分组时，修剪过程就开始了。路由器开始发送一个修剪报文（prune message）到上行路由器，这样它可拒绝相应的接口。即上行路由器可停止通过那个接口发送这个组的多播报文。现在，如果这个路由器接收到来自所有下行路由

器的修剪报文，它依次地发送一个修剪报文到上行路由器。

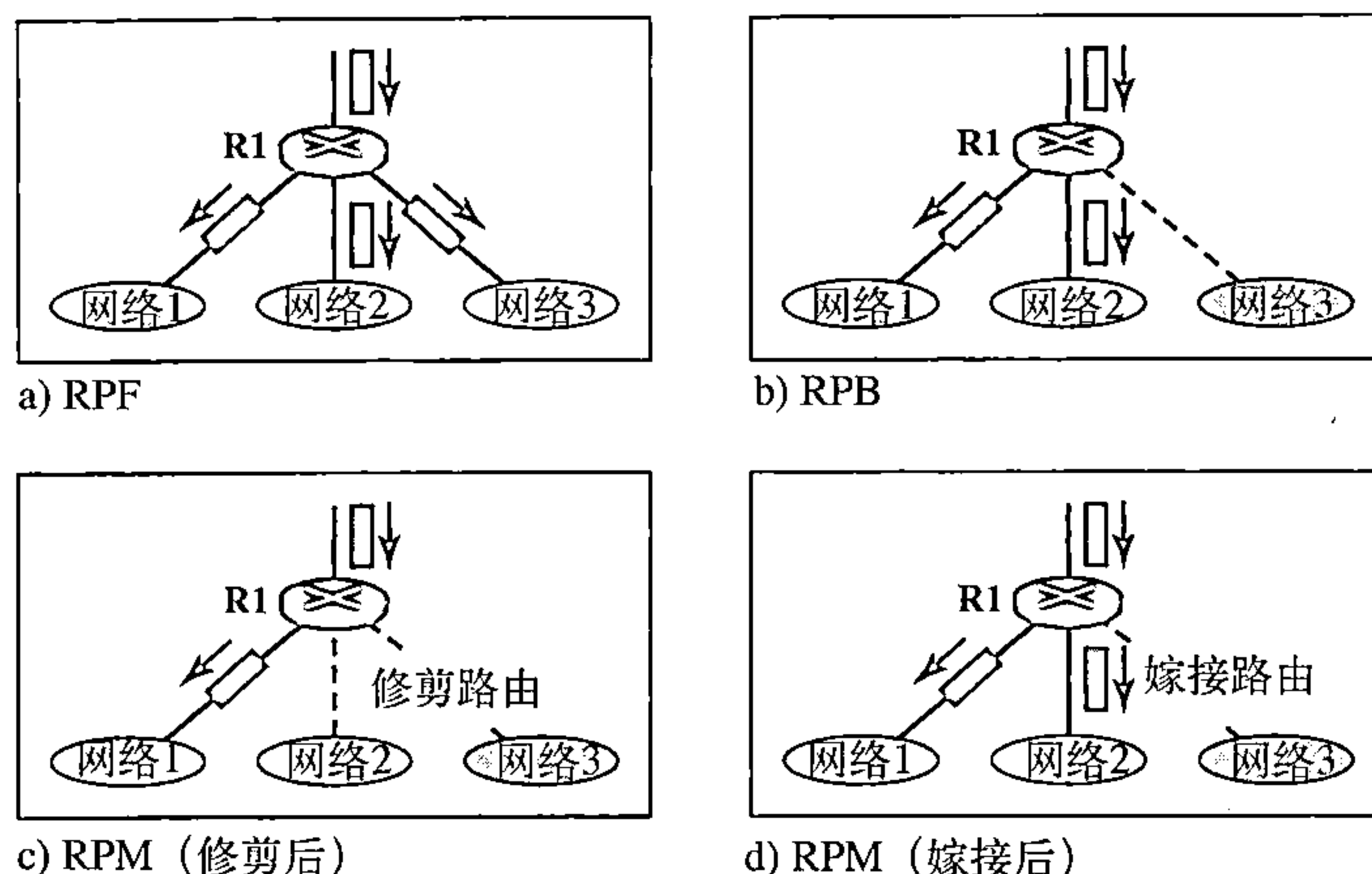


图22.43 RPF、RPB和RPM

如果一个叶子路由器（在树底端的路由器）已发送一个修剪报文，但通过IGMP突然了解到其中一个网络重新参与接收该多播分组。将怎样做？它可发送嫁接报文（graft message），该报文强制上行路由器恢复发送多播报文。

RPM将修剪和嫁接添加到RPB中，以创建一个多播最短路径树，该树支持动态成员关系的改变。

DVMR距离向量多播路由选择协议（Distance Vector Multicast Routing Protocol）是多播距离向量路由选择的实现。它是基于RIP的基于源树的路由选择协议。

CBT

基于核的树（core-Based Tree, CBT）协议是一种组共享协议，它使用核心路由器作为树的根。自治系统（AS）被分成多个区域，而每个区域选择一个核心路由器（中心路由器或会合路由器）。

树的生成 在选择了一个会合路由器之后，就要将所选择路由器的单播地址通知给每个路由器，然后每个路由器就发送一条单播加入报文来表明它想加入该组。这条报文通过位于发送方和会合路由器之间的所有路由器。每个中间路由器就从这条报文中取出所需的信息，如发送方的单播地址和分组到达所经过的端口等。然后它就将该报文转发到路径中的下一个路由器。当会合路由器已经收到了来自组中每个成员的所有加入报文时，树就生成完毕。此时，每个路由器都知道了它的上行路由器（通向根的路由器）和下行路由器（通向叶子的路由器）。

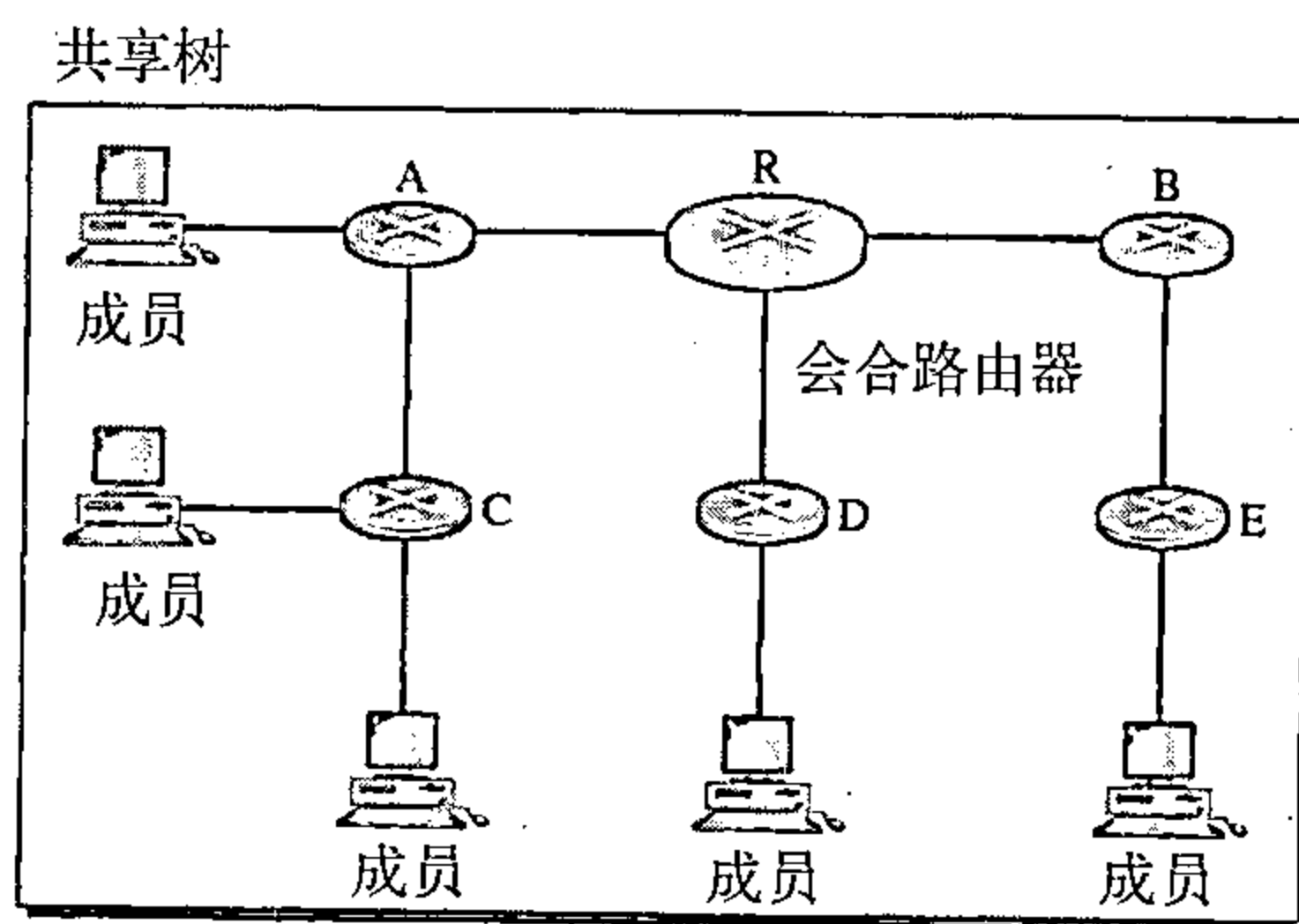


图22.44 具有会合路由器的组共享树

如果一个路由器想离开该组，它就给它的上行路由器发送一条离开报文。上行路由器就从树中除去与那台路由器的链路，并将该报文转发到它的上行路由器，依此类推。图22.44说明了一个具有会合路由器的组共享树。

读者可能已注意到了DVMRP、MOSPF及CBT之间的差异。首先，前面两种树是从根生成的，而CBT的树是从叶子生成的。其次，在DVMRP中，首先生成树（广播），然后对其进行修剪；而在CBT中，开始并没有生成树，树是通过逐步加入（嫁接）节点而形成的。

发送多播分组 在树生成之后，任何（属于组或不属于组的）源端都会给组中所有的成员发送一个多播分组。它仅使用会合路由器的单播地址将该分组发送到这个路由器。然后这个会合路由器将该分组分发到该组的所有成员。图22.45说明了一个主机如何能将一个多播分组发送到该组的所有成员。注意：源主机可以是共享树内（或外）的任何主机。在图22.45中，说明了位于共享树外的源主机。

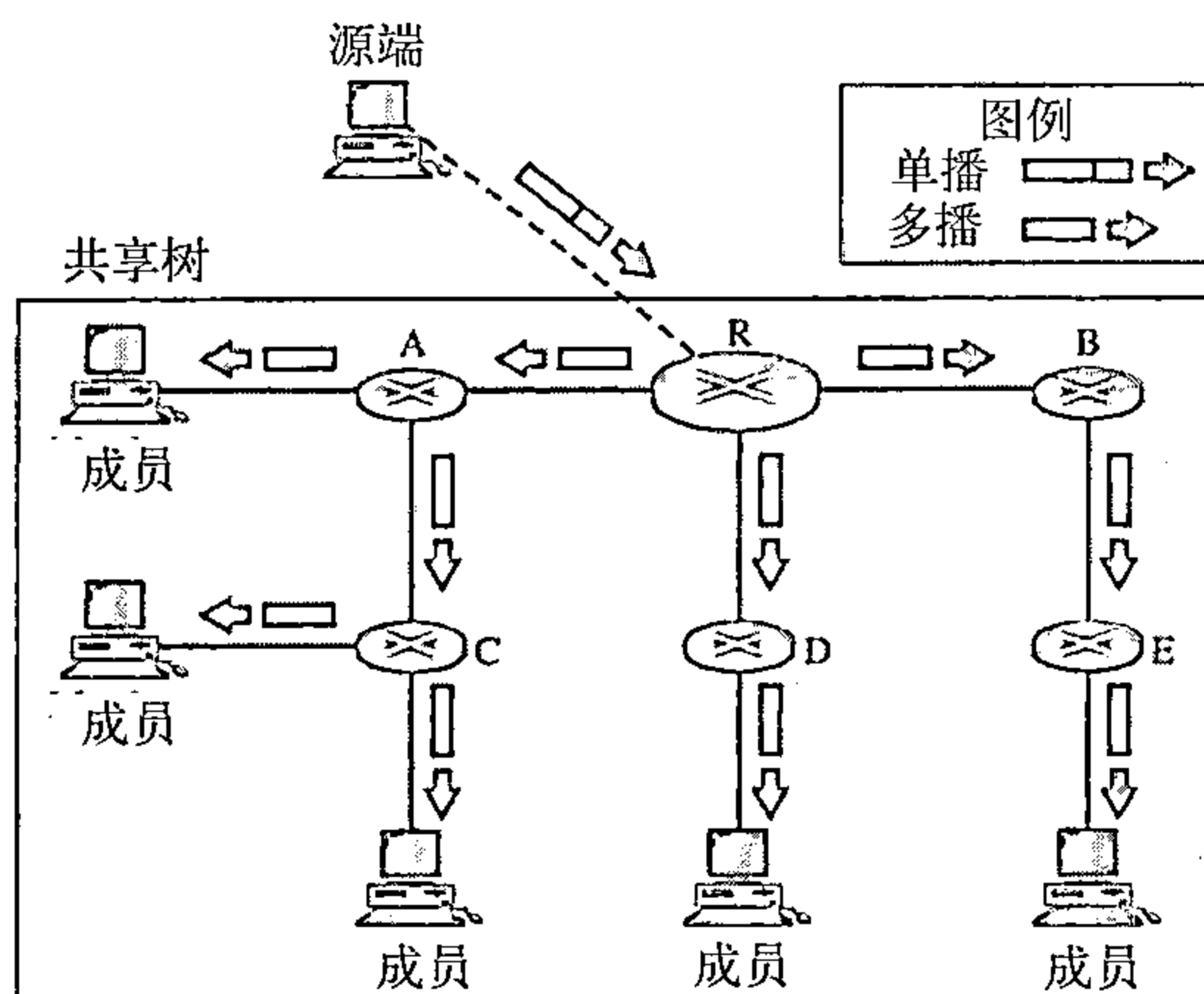


图22.45 发送一个多播分组到会合路由器

选择会合路由器 除了一点之外，这个方法是简单的。应该如何选择会合路由器以优化处理过程和多播呢？已经有了多种实现方法，但这个问题已超出了本书的讨论范围，留给更高级的书讨论。

总之，基于核的树（CBT）是一个组共享树，一种每组使用一棵树的基于中心的协议。树中的一个路由器称为核心路由器。遵循如下过程将一个分组从源端发送到组的成员：

1. 可能是（也可能不是）树的一部分的源端将多播分组封装到一个具有核心路由器单播目的地址的单播分组中，并将其发送给这个核心路由器。这一阶段的传递是使用一个单播地址来实现的，并且唯一的接收方是核心路由器。
2. 核心路由器将单播分组分解出来，并将其转发到所有加入请求到达的端口。
3. 每个接收到多播分组的路由器接着又将其转发给所有有加入请求的端口。

在CBT中，源端给核心路由器发送多播分组（封装在一个单播分组中）。核心路由器分解该分组，并将其转发到所有有加入请求的主机。

PIM

协议独立多播（Protocol Independent Multicast, PIM）是两个独立多播路由选择协议的合称，它们是PIM密集模式（PIM Dense Mode, PIM-DM）和PIM稀疏模式（PIM Sparse Mode, PIM-SM）。这两个协议都是依赖于单播协议，在这里就不讨论其相似性了，下面分别对其进行讨论。

PIM-DM 当每个路由器存在参与多播路由选择的可能性时，就使用PIM-DM。在此情况下，使用一种广播分组的协议是合理的，因为几乎所有的路由器都参与到该过程中。

PIM-DM用于密集多播环境，如一个局域网中。

PIM-DM是一种以基于源树的路由选择协议，它使用RPF和修剪/嫁接的多播路由选择策略。其操作与DVMRP相似。然而，与DVMRP不同的是，它不依赖于一种特定的单播路由选择协议。它假定自治系统使用一种单播协议，并且每个路由器有一个表，利用该表就能找到通向目的端的最优路径的外发端口。这种单播协议可能是一种距离矢量协议（RIP）或链路状态协议（OSPF）。

PIM-DM使用RPF和修剪/嫁接策略来处理多播路由选择。然而，它还是独立于底层的单播协议。

PIM-SM 当每个路由器存在参与多播路由选择的可能性很小时（稀疏模式），就使用PIM-SM。在此情况下，使用广播分组的协议是不合理的，但使用一种诸如CBT之类的组共享树的协议就更加合理了。

PIM-SM用于稀疏多播环境，如广域网。

PIM-SM是一种组共享树路由选择协议，它使用一个会合点（RP）作为树的源端。其操作与CBT相似，但它更简单。因为它不要求对加入报文的确认。此外，它还为每个区域创建一个RP备份集，用于RP故障恢复。

PIM-SM的一个特点是，在需要时它能从组共享树策略转换到基于源树策略。如果存在一个远离RP的密集活动区域，就会发生这种情况。在此情况下，就能利用基于源树策略，而不是组共享树策略对该区域进行更高效地处理。

PIM-SM与CBT类似，但其实现过程更简洁。

MBONE

多媒体和实时通信使因特网上对多播路由选择的需求增大了。然而，只有一小部分因特网路由器是多播路由器。换句话说，一个多播路由器（multicast router）可能在其附近找不到另一个多播路由器来转发多播分组。虽然在近几年内可通过增加越来越多的多播路由器解决此问题，但可以采用另一种方法，这种方法就是隧道技术（tunneling）。多播路由器可视为紧接着单播路由器的一组路由器。多播路由器在物理上可能没有连接在一起，但它们在逻辑上是连在一起的。图22.46说明了这种思想。在图22.46中，只有阴影圆圈内的路由器能进行多播路由选择。如果没有隧道技术，这些路由器是一些孤立的“岛屿”。为了使它们能进行多播路由选择，可运用隧道技术的概念将这些孤立的路由器组成一个多播主干网（multicast backbone, MBONE）。

通过将多播分组封装于一个单播分组中可以创建一个逻辑隧道（logical tunnel）。此时，多播分组就成为单播分组的有效载荷（数据）。中间（非多播）路由器作为单播路由器来路由分组，并将其从一个“岛屿”传递到另一个。这样做好像单播路由器不存在，而两个多播路由器是相邻的一样。目前唯一支持MBONE和隧道技术的协议是DVMRP。图22.47说明了这种概念。

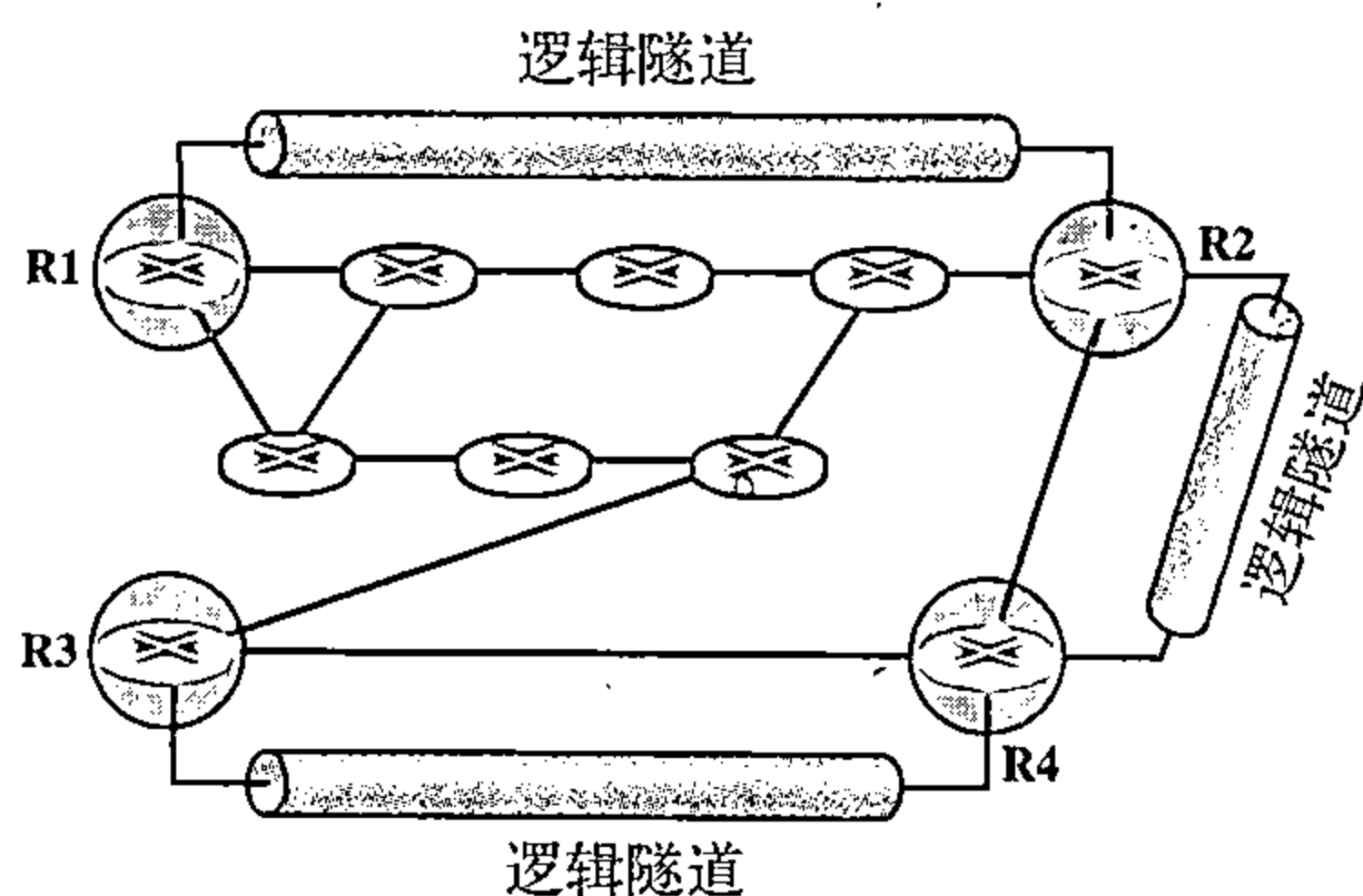


图22.46 逻辑隧道

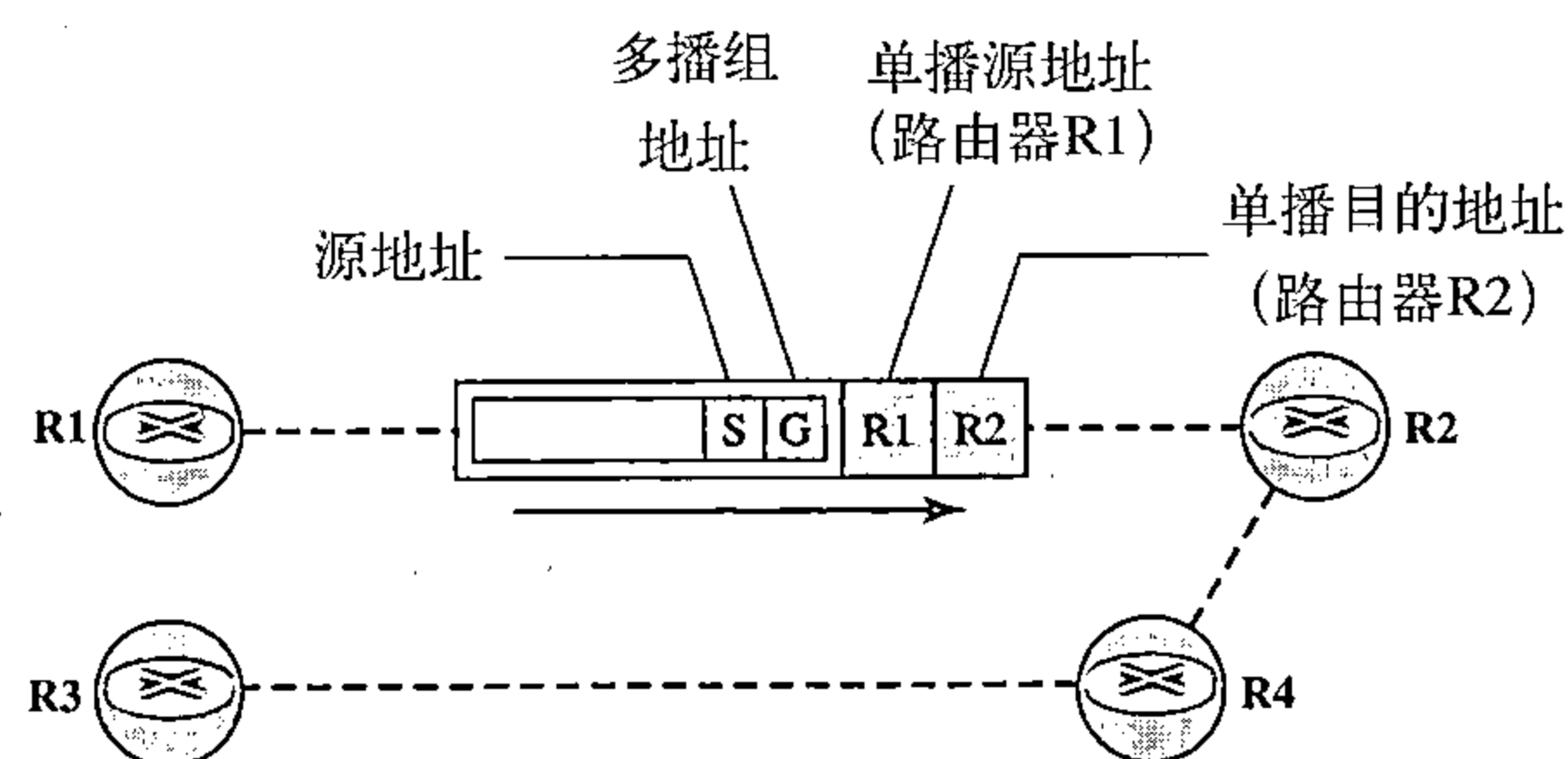


图22.47 MBONE

推荐读物

作为深入讨论与本章有关的主题，推荐阅读下列读物和网站，在本书末列出方括号[···]中的参考资料。

读物

传递和转发在[For06]的第6章中有讨论。单播路由选择协议可参看[For06]的第14章，而多播与多播路由选择可参看同书的第15章。[WZ01]有完整多播讨论。路由选择协议可参见[Hui00]。OSPF在[Moy98]中有讨论。

网站

- www.ietf.org/rfc.html 有关请求评论。

请求评论

RIP的讨论可从下列请求评论中找到：

1131, 1245, 1246, 1247, 1370, 1583, 1584, 1585, 1586, 1587, 1722, 1723, 2082, 2453

OSPF的讨论可从下列请求评论中找到：

1131, 1245, 1246, 1247, 1370, 1583, 1584, 1585, 1586, 1587, 2178, 2328, 2329, 2370

BGP的讨论可从下列请求评论中找到：

1092, 1105, 1163, 1265, 1266, 1267, 1364, 1392, 1403, 1565, 1654, 1655, 1665, 1771, 1772, 1745, 1774, 2283

本章小结

- 如果一个分组的发送方（主机或路由器）与目的端都连接在同一网络上，则分组的传递就称为直接传递；如果分组的发送方（主机或路由器）和目的端连接在不同的网络上，则分组的传递就称为间接传递。
- 在下一跳方法中，不需要将分组所经过的每个路由器全部列在路由表中，而只需要将下一跳地址写在路由表中。在特定网络方法中，在一个网络上的所有主机都共享路由表中的一个项目。
- 在特定主机方法中，路由表中包含了一个主机的完整地址。
- 在默认方法中，指定一个路由器接收所有在路由表中无匹配的分组。
- 对于无类寻址，路由表至少需要4列。
- 在无类寻址中，地址聚合简化了转发过程。
- 在无类寻址中，要求最长的掩码匹配。

- 为了防止极大的路由表，无类寻址需要分层结构的和地域的路由选择。
- 静态路由表中的项目是由网络管理人员手工更新。而动态路由表中的项目是由路由选择协议自动更新的。
- 度量是指通过网络传输分组的代价。
- 自治系统（AS）是在一个单独的管理机构管辖之下的一组网络和路由器。
- RIP是距离向量路由选择协议，每个路由器与它的邻站在固定时间间隔内共享整个关于AS的知识。
- RIP协议的两个缺点是慢收敛和不稳定性。解决RIP不稳定性的方法有触发更新、分割范围和毒性逆转。
- OSPF将自治系统划分为一些区域，这些区域定义为一些网络、主机和路由器的集合。
- OSPF是链路状态路由选择协议，每一个路由器向区域内的每个其他路由器发送其邻站的状态，只有当邻站有变化时才发送分组。
- OSPF路由表是使用Dijkstra算法计算出来的。
- BGP是用在自治系统之间的路由选择协议，用于更新路由表。
- BGP是基于称为路径向量路由选择一种路由选择协议。在这种协议中，经过自治系统的分组必须显式地列出。
- 在多播路由选择的基于源树方法中，源端/组对决定了树的结构；在多播路由选择的组共享树方法中，组决定了树的结构。
- MOSPF是一种多播路由选择协议。它使用多播链路状态路由选择来创建一个基于源树方法的最小代价树。
- 在逆路径转发（RPF）中，路由器只转发那些从源端沿着最短路径到达它的分组。
- 逆路径广播（RPB）创建一个从源端到每个目的端的最短路径广播树。它能保证每个目的端只接收到一个而只有一个分组副本。
- DVMRP是一种多播路由选择协议，它用距离向量路由选择协议创建一个基于源的树。
- 基于核树（CBT）协议是一种利用一个路由器作为树根的多播路由选择协议。
- PIM-DM是一种基于源树的路由选择协议，它使用RPF和修剪以及嫁接策略来处理多播路由选择。
- PIM-SM是一种与CBT类似的组共享树路由选择协议，它使用会合路由器作为树的源。
- 对于两个不邻的多播路由器之间的多播，我们建立一个多播主干网（MBONE）以实现隧道技术。

习题

复习题

1. 直接传递与间接传递有什么不同？
2. 列出本章所讨论的三种转发技术，对每种技术给出简要说明。
3. 比较本章讨论的两种不同的路由表。
4. RIP的目的是什么？
5. RIP报文的功能是什么？
6. 为什么截止计时器的值是周期计时器的值6倍？
7. 跳数限制如何缓解RIP问题？
8. 列举RIP的缺点及其相应的补救办法。